# ABSTRACT

Small boarding houses often run on spreadsheets, paper receipts and messages that are not connected, and the general-purpose property management systems on the market do not fit them. This study designed, developed and evaluated Hivelet, a web-based apartment management system delivered as a Progressive Web Application, for the Fe Galang Da Silva Boarding House, a 33-unit property in Legazpi City. The boarding house's existing practices were analyzed, and transferring its records, 937 income rows and 1,327 expense allocations, into the system exposed problems the spreadsheet had hidden. The system was developed in two Agile iterations with Vue, Express and PostgreSQL. It provides tenant and room management, a public list of units with inquiries the landlady answers, financial tracking with optional GCash payment through Adyen, repair requests with notifications, role-based access control, and installation on a phone or computer without an app store. The GCash payment works end to end in Adyen's test environment and takes no real money. It is not live, because going live needs the owner's merchant onboarding documents. In pilot testing at the unit and integration, system, and acceptance levels, 82 of the 83 tests executed passed (98.8 percent), with no critical defect. Three tenants completed every task they attempted, 31 of 34 attempts without help, and on the redesigned system ten tenants and four prospective tenants completed every task. The system was evaluated on the eight characteristics of ISO/IEC 25010:2011 by the process of ISO/IEC 25040, using measures from the tests and a survey. The landlady, ten tenants and three prospective tenants rated the redesigned system 4.84 overall,

Very High Quality, with every characteristic Very High Quality. Three technical evaluators rated it 4.90, also with every characteristic Very High Quality. The measurements supported the ratings on six characteristics and were less favorable on Security and Maintainability. The results show that an integrated system can be fitted to a small, cash-based property while keeping its landlady's own report layout, invoice numbers and cash payments. The study met its general objective: Hivelet was designed, developed and evaluated as a web-based apartment management system that improves administrative workflow integration, communication and operational control at the boarding house.

# TABLE OF CONTENTS

ABSTRACT ..... i
1 INTRODUCTION ..... 1
1.1 Background of the Study ..... 1
1.2 Objectives of the Study ..... 3
1.3 Significance of the Study ..... 4
1.4 Scope and Delimitations ..... 5
2 THEORETICAL FRAMEWORK ..... 7
2.1 Review of Related Literature ..... 7
2.1.1 Fragmentation and Operational Inefficiencies in Property Management ..... 7
2.1.2 Management Information Systems as the Foundation of Centralization ..... 8
2.1.3 Financial Analytics and Administrative Decision Support ..... 10
2.1.4 Digital Communication and Structured Issue Resolution ..... 11
2.1.5 Progressive Web Applications as an Accessibility and Integration Framework ..... 13
2.1.6 Synthesis of Literature ..... 14
2.2 Gap Bridged by the Study ..... 15
2.3 Concept of the Study ..... 16
2.4 Definition of Terms ..... 18
3 MATERIALS AND METHODS ..... 23
3.1 Materials ..... 23
3.1.1 Software ..... 23
3.1.2 Hardware ..... 25
3.2 Software Methodology ..... 26
3.2.1 Requirements ..... 28
3.2.2 Design ..... 28
3.2.3 Development ..... 29
3.2.4 Testing ..... 29
3.2.5 Deployment ..... 31
3.2.6 Review and Maintenance ..... 31
3.3 Evaluation Procedure ..... 31
3.4 Ethical Considerations ..... 37
4 RESULTS AND DISCUSSION ..... 40

4.1 Analysis of Existing Apartment Management Practices and System Requirements ..... 40
4.1.1 Existing Practices and Identified Problems ..... 40
4.1.2 System Requirements ..... 43
4.2 Development of the Hivelet System ..... 46
4.2.1 System Architecture and Technology Stack ..... 46
4.2.2 Tenant and Room Management Module ..... 55
4.2.3 Booking and Reservation Management Module ..... 56
4.2.4 Financial Tracking and Payment Recording Module ..... 58
4.2.5 Maintenance Ticketing and Notification Module ..... 60
4.2.6 Role-Based Access Control ..... 61
4.2.7 Progressive Web Application Implementation ..... 63
4.3 Pilot Testing of the Developed System ..... 64
4.3.1 Automated Verification ..... 65
4.3.2 Use in the Live Environment ..... 67
4.3.3 Screen-versus-Database Audit ..... 67
4.3.4 Functional Walkthrough Testing ..... 69
4.3.5 Responsiveness and Operational Performance ..... 69
4.3.6 Offline Use, Installation, Weak Connections and Simultaneous Users ..... 71
4.3.7 User Acceptance Testing with the Landlady and Tenants ..... 72
4.4 Evaluation of the System Based on ISO/IEC 25010 ..... 76
4.4.1 Respondents ..... 77
4.4.2 Interpretation of Scores ..... 79
4.4.3 Results by Characteristic ..... 80
4.4.4 Security ..... 84
4.4.5 Measured and Rated Evidence Together ..... 85
4.4.6 Optimization in Response to the Evaluation ..... 87
4.5 Deployment Plan and Strategies ..... 91
5 SUMMARY, CONCLUSIONS, AND RECOMMENDATIONS ..... 94
5.1 Summary ..... 94
5.2 Conclusions ..... 96
5.3 Recommendations ..... 98
REFERENCES ..... 100
APPENDICES ..... 107

APPENDIX A: LETTER OF REQUEST ..... 108
APPENDIX B: APPOINTMENTS ..... 109
APPENDIX C: GANTT CHART ..... 110
APPENDIX D: CONTEXT DIAGRAM ..... 112
APPENDIX E: LEVEL 1 DATA FLOW DIAGRAM ..... 113
APPENDIX F: DATA FLOW DIAGRAMS (LEVEL 2) ..... 114
APPENDIX G: USE CASE DIAGRAM ..... 121
APPENDIX H: ENTITY-RELATIONSHIP DIAGRAM ..... 122
APPENDIX I: INITIAL DESIGN ..... 123
APPENDIX J: EVALUATION INSTRUMENT ..... 127
Section 1: ABOUT YOU ..... 128
Section 2: OWNER/LANDLADY ..... 129
Section 3: TENANTS ..... 131
Section 4: TECHNICAL EVALUATORS ..... 133
Section 5: LOOKING FOR A ROOM ..... 136
APPENDIX K: USER'S MANUAL ..... 139
Part 1. For tenants / Para sa mga nangungupahan ..... 139
1. Signing in for the first time / Unang pag-sign in ..... 139
2. Overview / Pangunahing pahina ..... 140
3. Payments and billing / Bayad at singil ..... 141
4. Repairs / Pagpapaayos ..... 142
5. My details / Aking detalye ..... 143
6. Install it on your phone / I-install sa cellphone ..... 143
7. Signing out / Pag-sign out ..... 144
Part 2. For the landlady (administrator) ..... 144
1. Signing in, your number and your password ..... 146
2. Overview ..... 147
3. Recording a payment (Monthly Income) ..... 149
4. Recording an expense (Monthly Expenses) ..... 151
5. Tenants ..... 152
6. Rooms and rates ..... 153
7. Repairs ..... 154
8. Inquiries ..... 154
9. The record of who did what ..... 155

10. When the connection is weak or gone ..... 155
11. Looking after the records ..... 156
12. Quick answers ..... 156
APPENDIX L: TEST CASES ..... 158
Part A. Landlady (administrator) (A-05 to A-32 are the 26-step walkthrough) ..... 158
Part T. Tenant tasks (testers coded T1 to T3, with P done alone, PH done with help, F not done and NT not tried) ..... 161
Part P. Public site ..... 164
Part C. Simultaneous use ..... 164
Part O. Offline, weak signal and installing ..... 165
Part S. Security ..... 167
Part CO. Browsers and devices ..... 168
Part PF. Performance timings (seconds) ..... 169
Part N. Changes shipped on 30 September ..... 170
The re-run after the redesign, 4 to 13 November 2026 ..... 171
APPENDIX M: SUPPORTING EVIDENCE FOR CHAPTER 4 ..... 175
APPENDIX N: DATA DICTIONARY ..... 221
Property ..... 221
rooms ..... 221
clusters ..... 223
property_areas ..... 223
room_photos ..... 224
room_price_history ..... 225
People ..... 225
profiles ..... 225
room_assignments ..... 227
Money ..... 227
monthly_income_records ..... 228
monthly_expense_entries ..... 230
expense_property_allocations ..... 230
fixed_expense_categories ..... 231
bills ..... 231
payments ..... 232
system_settings ..... 233

Requests ..... 234
maintenance_tickets ..... 234
ticket_messages ..... 235
ticket_attachments ..... 236
inquiries ..... 236
inquiry_messages ..... 237
System ..... 238
notifications ..... 238
audit_logs ..... 239
Backup ..... 240
copied_period_backup_031 ..... 240
date_paid_import_backup_032 ..... 241
rent_period_drift_backup_030 ..... 241
APPENDIX O: USER ACCEPTANCE SIGN-OFF ..... 243
APPENDIX P: INFORMED CONSENT FORM ..... 244
APPENDIX Q: SOURCE CODE ..... 245
APPENDIX R: LITERATURE MAP ..... 246

# LIST OF TABLES

**Table 3-1.** Software requirements ..... 23
**Table 3-2.** Workstation hardware specifications ..... 25
**Table 3-3.** Mobile testing hardware specifications (baseline) ..... 26
**Table 3-4.** Likert scale ..... 35
**Table 3-5.** Interpretation of mean scores ..... 35
**Table 3-6.** Survey items and tests assigned to the ISO/IEC 25010:2011 sub-characteristics ..... 36
**Table 4-1.** Existing practices and identified problems ..... 41
**Table 4-2.** Identified gaps, system requirements and implementing modules ..... 44
**Table 4-3.** Non-functional requirements and where each is measured ..... 45
**Table 4-4.** Technology stack of the developed system ..... 47
**Table 4-5.** Data dictionary extract: monthly income records ..... 54
**Table 4-6.** Business rules for computed figures, with a worked example ..... 60
**Table 4-7.** Role and privilege matrix ..... 62
**Table 4-8.** Results of automated verification ..... 66
**Table 4-9.** Results of the screen-versus-database audit ..... 68
**Table 4-10.** Page load and response times (seconds) ..... 70
**Table 4-11.** Tenant task results ..... 73
**Table 4-12.** Summary of test execution by level ..... 75
**Table 4-13.** The evaluation process, following ISO/IEC 25040 ..... 77
**Table 4-14.** Distribution of respondents ..... 78
**Table 4-15.** Instrument administration and response rate ..... 78
**Table 4-16.** Worked computation of a weighted mean ..... 80
**Table 4-17.** Summary of evaluation results ..... 81
**Table 4-18.** Measured and rated evidence by characteristic ..... 86
**Table 4-19.** Optimizations made in response to evaluation, by source ..... 88
**Table 4-20.** Defects found, by activity, severity and disposition ..... 90
**Table 4-21.** Software and hardware requirements for deployment ..... 92
**Table 4-22.** Deployment plan and strategies ..... 92
**Table M-1.** Results of the functional walkthrough (30 September 2026) (summarized in Chapter 4, Section 4.3.4) ..... 175
**Table M-2.** Evaluation results for Functional Suitability, by item and group (summarized in Chapter 4, Section 4.4.3, Table 4-17) ..... 178

**Table M-3.** Measured evidence for Functional Suitability (summarized in Chapter 4, Section 4.4.5, Table 4-18) ..... 180

**Table M-4.** Evaluation results for Performance Efficiency, by item and group (summarized in Chapter 4, Section 4.4.3, Table 4-17) ..... 181

**Table M-5.** Measured evidence for Performance Efficiency (summarized in Chapter 4, Section 4.4.5, Table 4-18) ..... 182

**Table M-6.** Evaluation results for Compatibility, by item and group (summarized in Chapter 4, Section 4.4.3, Table 4-17) ..... 183

**Table M-7.** Measured evidence for Compatibility (summarized in Chapter 4, Section 4.4.5, Table 4-18) ..... 184

**Table M-8.** Evaluation results for Usability, by item and group (summarized in Chapter 4, Section 4.4.3, Table 4-17) ..... 184

**Table M-9.** Measured evidence for Usability (summarized in Chapter 4, Section 4.4.5, Table 4-18) ..... 186

**Table M-10.** Evaluation results for Reliability, by item and group (summarized in Chapter 4, Section 4.4.3, Table 4-17) ..... 186

**Table M-11.** Measured evidence for Reliability (summarized in Chapter 4, Section 4.4.5, Table 4-18) ..... 188

**Table M-12.** Evaluation results for Security, by item and group (summarized in Chapter 4, Section 4.4.3, Table 4-17) ..... 189

**Table M-13.** Measured evidence for Security (summarized in Chapter 4, Section 4.4.5, Table 4-18) ..... 190

**Table M-14.** Evaluation results for Maintainability, by item and group (summarized in Chapter 4, Section 4.4.3, Table 4-17) ..... 191

**Table M-15.** Measured evidence for Maintainability (summarized in Chapter 4, Section 4.4.5, Table 4-18) ..... 192

**Table M-16.** Evaluation results for Portability, by item and group (summarized in Chapter 4, Section 4.4.3, Table 4-17) ..... 192

**Table M-17.** Measured evidence for Portability (summarized in Chapter 4, Section 4.4.5, Table 4-18) ..... 194

**Table M-18.** Every optimization made in response to evaluation, with its finding (summarized in Chapter 4, Section 4.4.6, Table 4-19) ..... 194

**Table M-19.** The two iterations, phase by phase (summarized in Chapter 4, Section 4.2.1). The first working version, from which the iterations started, was built from 24 July to 28 August 2026 (95 commits): the public site, the tenant and landlady portals, the ledgers, repairs, inquiries, notifications and audit trail, with 937 income rows and 1,327 expense allocations transferred from the landlady's workbook. ..... 200

**Table M-20.** Quality in use (30 September 2026) (summarized in Chapter 4, Section 4.4.5) ..... 204

**Table M-21.** The 2011 characteristics and their place in ISO/IEC 25010:2023 (summarized in Chapter 4, Section 4.4)..... 205

**Table M-22.** Simultaneous use by the landlady and tenants (30 September 2026, two tenants) (summarized in Chapter 4, Section 4.3.7)..... 206

**Table M-23.** Test activities and the ISO/IEC 25010:2011 characteristics they measured (summarized in Chapter 4, Section 4.3) ..... 207

**Table M-24.** Offline and installation behavior (29 September 2026) (summarized in Chapter 4, Section 4.3.6)..... 208

**Table M-25.** Simulated simultaneous users, read operations (29 September 2026) (summarized in Chapter 4, Section 4.3.6) ..... 208

**Table M-26.** Simultaneous saves (19 and 24 September 2026) (summarized in Chapter 4, Section 4.3.6) ..... 209

**Table M-27.** Corrections after the testing activities (13 October 2026, referred to in Chapter 4, Section 4.4.6)..... 209

**Table M-28.** Respondents before and after the redesign (summarized in Chapter 4, Section 4.4.1) ..... 210

**Table M-29.** Composite means before and after the redesign (summarized in Chapter 4, Section 4.4.3) ..... 211

**Table M-30.** Task results, second round (summarized in Chapter 4, Section 4.3.7) ..... 212

**Table M-31.** Page timings before and after the redesign (summarized in Chapter 4, Section 4.3.5) ..... 214

**Table M-32.** The 44 functional requirements, graded against the code in September 2026 and against the final system on 13 November 2026 (summarized in Chapter 4, Section 4.1.2) ..... 215

# LIST OF FIGURES

*Figure 2-1. Management Information System (MIS) framework, adapted from Laudon et al. (2026).* ..... 9
*Figure 2-2. Conceptual Framework.* ..... 17
*Figure 3-1. Agile Software Development Life Cycle.* ..... 27
*Figure 4-1. The landlady's process before the system.* ..... 42
*Figure 4-2. System architecture.* ..... 48
*Figure 4-3. Context diagram.* ..... 49
*Figure 4-4. Level 1 data flow diagram.* ..... 50
*Figure 4-5. Use case diagram.* ..... 51
*Figure 4-6. Entity-relationship diagram.* ..... 53
*Figure 4-7. Room and rate directory.* ..... 56
*Figure 4-8. Public unit catalogue (top) and inquiry form (bottom).* ..... 57
*Figure 4-9. Monthly Income, with a GCash payment waiting to be verified.* ..... 59
*Figure 4-10. Maintenance requests board.* ..... 61
*Figure 4-11. The tenant portal on a mobile phone.* ..... 64
*Figure C-1. Development schedule, July to November 2026.* ..... 111
*Figure D-1. Context diagram, as built (the same diagram as Figure 4-3).* ..... 112
*Figure E-1. Level 1 data flow diagram, as built (the same diagram as Figure 4-4).* ..... 113
*Figure F-1. Level 2 diagram of process 1.0, handle inquiries.* ..... 115
*Figure F-2. Level 2 diagram of process 2.0, manage units and rates.* ..... 115
*Figure F-3. Level 2 diagram of process 3.0, manage tenants and tenancies.* ..... 116
*Figure F-4. Level 2 diagram of process 4.0, bill and record payments.* ..... 117
*Figure F-5. Level 2 diagram of process 5.0, manage repair requests.* ..... 118
*Figure F-6. Level 2 diagram of process 6.0, record expenses.* ..... 119
*Figure F-7. Level 2 diagram of process 7.0, produce reports.* ..... 119
*Figure F-8. Level 2 diagram of process 8.0, sign in and manage one's own account.* ..... 120
*Figure G-1. Use case diagram, as built (the same diagram as Figure 4-5).* ..... 121
*Figure H-1. Entity-relationship diagram, as built (the same diagram as Figure 4-6).* ..... 122
*Figure I-1. Public landing page, 28 August 2026.* ..... 123
*Figure I-2. A unit category on the public site, 28 August 2026.* ..... 124
*Figure I-3. Tenant directory, with sample tenants, 28 August 2026.* ..... 124

*Figure I-4. Tenant overview on a phone, 28 August 2026.* ..... 125
*Figure R-1. Literature map of the study.* ..... 246

*If a list is empty, press Ctrl+A and then F9 to update every field. After pasting into the final document, give each pasted caption the Table Caption or Figure Caption style (or keep this file's), or the lists will not find them.*

# 1 INTRODUCTION

## 1.1 Background of the Study

Around the world, apartment management involves tenant record keeping, financial tracking, and maintenance coordination. Residential tenants often ask for minor repairs, and the software apartment managers use often does not connect with their other financial and operational tools, which leaves information in separate silos and makes consolidated reporting difficult (Burke, 2026). Property management and rental platforms aim to address these inefficiencies through centralized data storage and real-time processing (Kotkar et al., 2026), yet a review of 213 studies found that real estate as a whole still needs to improve its adoption of digital technologies and online platforms to move from traditional to smart practice (Ullah et al., 2018). These studies, however, describe properties far larger than a boarding house: the apartment complexes Burke (2026) examined have 50 to 500 units. A small, resource-constrained boarding house does not share the conditions of properties of that size.

In the Philippines, a study of nine micro, small, and medium-sized enterprises during the COVID-19 pandemic found that what held back their adoption of digital tools was less a lack of money, infrastructure, or skills than the owners' hesitation and readiness, including doubt that a tool fits how the business already works (Mia et al., 2024). A small boarding house is such an enterprise. Magno et al. (2024) developed an apartment rental management system to address disorganized data management, difficulty in tracking pending and paid bills, missed payment deadlines, and reliance on paper-based records. They report

that the system improved data management and reduced the errors of manual processes, but the system focuses on financial transactions and task management, without integrating communication, booking, and tenant interaction into a unified system. Boarding house studies elsewhere in the country show the same pattern. In Ormoc City, Pernate et al. (2025) built a mobile application for finding and booking boarding house rooms, and in Surigao City, Mangca (2023b) built a web-based booking system with SMS notifications. Both center on how a prospective tenant finds and books a room rather than on the administration that follows.

In the Bicol Region, Pavia et al. (2022) developed a property management system for the mock hotel in which Hospitality Management students at Bicol College take their practical examinations, replacing third-party software, and its users rated its design, functionality, and user-friendliness positively. That setting is a controlled academic environment with predefined workflows and institutional support, unlike a small boarding house, where administration is informal, decentralized, and constrained by limited resources.

Locally, the Fe Galang Da Silva Boarding House, a 33-unit property in Legazpi City near Bicol University, shows these conditions directly. Its records were kept in standalone spreadsheet files on a USB flash drive, exposing them to accidental deletion, file corruption, duplication, and lack of version control. Rent is collected in cash, and tenants report repair requests and other concerns through messaging applications or verbally, so requests go untracked, responses are delayed, and follow-up is inconsistent. Because records, communication, and

transactions sit in separate tools, the landlady, who runs the boarding house for its owner, has difficulty keeping accurate records, monitoring financial transactions, and resolving tenant concerns on time.

Taken together, these studies show that digital systems can improve specific parts of property management, but they were built either for one function or for a controlled environment, and they do not address the fragmented administration of a real, resource-constrained boarding house. This is the gap this study addresses: the absence of an integrated platform that brings tenant management, booking and reservation management, financial tracking, and communication into one system suited to a small boarding house.

### 1.2 Objectives of the Study

The primary objective of this study is to address the mismatch between existing property management systems and the operational realities of small-scale, resource-constrained boarding houses by designing, developing, and evaluating Hivelet, a web-based apartment management system that improves administrative workflow integration, communication, and operational control. Specifically, from July to November 2026, the study aims:

1. To analyze the existing apartment management practices of the Fe Galang Da Silva Boarding House to determine the challenges in tenant management, financial tracking, communication, and booking processes, and establish system requirements based on the identified operational gaps and user needs.

2. To develop the Hivelet system to feature tenant and room management, booking and reservation management, financial tracking with optional online payment recording, a maintenance ticketing and notification system, role-based access control for administrators and tenants, and Progressive Web Application (PWA) support for responsive cross-device accessibility.
3. To pilot test the developed Hivelet system for functionality, responsiveness, and operational performance.
4. To evaluate and optimize the system based on the ISO/IEC 25010 software quality model in terms of: Functional suitability, Performance efficiency, Compatibility, Usability, Reliability, Security, Maintainability, Portability.

### 1.3 Significance of the Study

Hivelet, a web-based apartment management system delivered as a PWA, addresses the fragmented, manual administration of a small, resource-constrained boarding house. The study is significant to the following groups.

For the landlady, the study provides one system in place of spreadsheets kept on a USB flash drive, which are prone to inconsistency, loss, and duplication. Keeping tenant records, financial transactions, and repair requests in one system makes the records more reliable and the administration easier to coordinate, even where rent is paid in cash.

For tenants, the system gives one place to submit and follow their repair requests and to view their own bills and receipts, in place of messaging applications and verbal reports, which leave requests untracked and responses delayed.

For future researchers, the study offers a PWA-based property management system built for a small residential property. Its architecture and its evaluation against the ISO/IEC 25010 model may serve as a reference for systems meant for small, resource-constrained residential operations rather than for enterprise-level property management.

### 1.4 Scope and Delimitations

This study covers the design, development, and evaluation of Hivelet, a PWA that moves the 33-unit Fe Galang Da Silva Boarding House, at 32 Sampaguita Street, Barangay 4 Sagpon, Old Albay District, Legazpi City, from spreadsheet- and paper-based administration to one centralized system. Its tenant and room management moves tenants in, stores their records, and shows the current availability of every unit. Its booking and reservation management takes the form the landlady's practice needs. Prospective tenants see the units on a public page and send an inquiry about one without an account. The landlady answers the inquiry, can mark the unit Reserved, and moves the person in from it. Nothing is booked or paid for online. Financial tracking records tenant payments and expenses and monitors cash flow. It mainly records cash payments entered by hand, and it offers an optional online payment through GCash, described below. Maintenance ticketing lets tenants send repair requests, which the landlady assigns, tracks, and updates until they are resolved. Hivelet was evaluated against the eight characteristics of the ISO/IEC 25010 software quality model: functional suitability, performance efficiency, compatibility, usability, reliability, security, maintainability, and portability.

The study has three delimitations. First, the PWA caches resources for limited offline access and adapts to different devices, but entering data, keeping it synchronized in real time, and sending or answering inquiries require an internet connection, so that every device works from the same records. Second, the online payment feature lets a tenant pay a bill through GCash using the Adyen payment gateway. Each such payment is recorded as pending and counts only after the landlady verifies it. The feature was built and works end to end, from the tenant's checkout to the gateway's signed notification and the landlady's verification, but only in Adyen's test environment, so it is not live and takes no real money. Going live needs the owner's merchant onboarding documents, which were outside this study. Rent therefore continues to be paid in person and recorded by the landlady. The feature also excludes refunds (made from the gateway's own dashboard), automated reconciliation against bank records, and direct banking integration. Third, electricity is outside the system, because each unit has its own meter and its tenant pays the electric company directly.

# 2 THEORETICAL FRAMEWORK

This chapter sets out the theoretical and empirical basis of the Hivelet system from the literature on property management systems, Management Information Systems (MIS), financial analytics, digital communication systems, and Progressive Web Applications (PWA). It treats these domains as connected parts of one apartment management operation, and argues that the inefficiencies of small-scale residential operations come mainly from fragmented administration, in which records, financial monitoring, communication, and coordination sit in disconnected tools and processes. The reviewed literature supports the need for an integrated, centralized, and accessible management system for resource-constrained apartment operations.

## 2.1 Review of Related Literature

### 2.1.1 Fragmentation and Operational Inefficiencies in Property Management

The reviewed studies identify fragmented administrative workflows as one of the primary operational issues in residential property management (Burke, 2026; Magno et al., 2024). As tenants, transactions, and maintenance activities increase, manual documentation, spreadsheet-based tracking, and informal communication become difficult to manage consistently and accurately.

Burke (2026), examining apartment complexes of 50 to 500 units, finds operational, tenant, regulatory, labor, and technology challenges that differ from those of commercial properties. Among them, residential tenants often ask for minor repairs, and the software in use often does not connect with other financial

and operational tools, which leaves information in separate silos. Thevaraju et al. (2019) describe the same condition at an apartment of eight blocks and more than 1,000 units whose maintenance transactions were recorded by hand. They built a web system, developed with the Agile methodology, through which management handles apartment and tenant data, maintenance requests, notices, and complaints. Across the sector, Ullah et al. (2018), reviewing 213 studies of smart real estate technology, find that real estate still needs to improve its adoption of digital technologies and online platforms to move from traditional to smart practice.

Local studies show the same condition. Magno et al. (2024) observed that apartment rental management suffers from tracking pending and paid bills across units, missed payment deadlines, and informal, paper-based records, and that their system improved data management and reduced the errors of manual processes. Pavia et al. (2022) built a property management system for the mock hotel in which hospitality students take their practical examinations, and its users rated its design, functionality, and user-friendliness positively. That is an institutional setting with predefined workflows and technical support, which differs significantly from resource-constrained apartment operations.

Together, these studies show that the main challenge in apartment management is not the absence of digital tools but the absence of a structure that consolidates administrative functions into one environment.

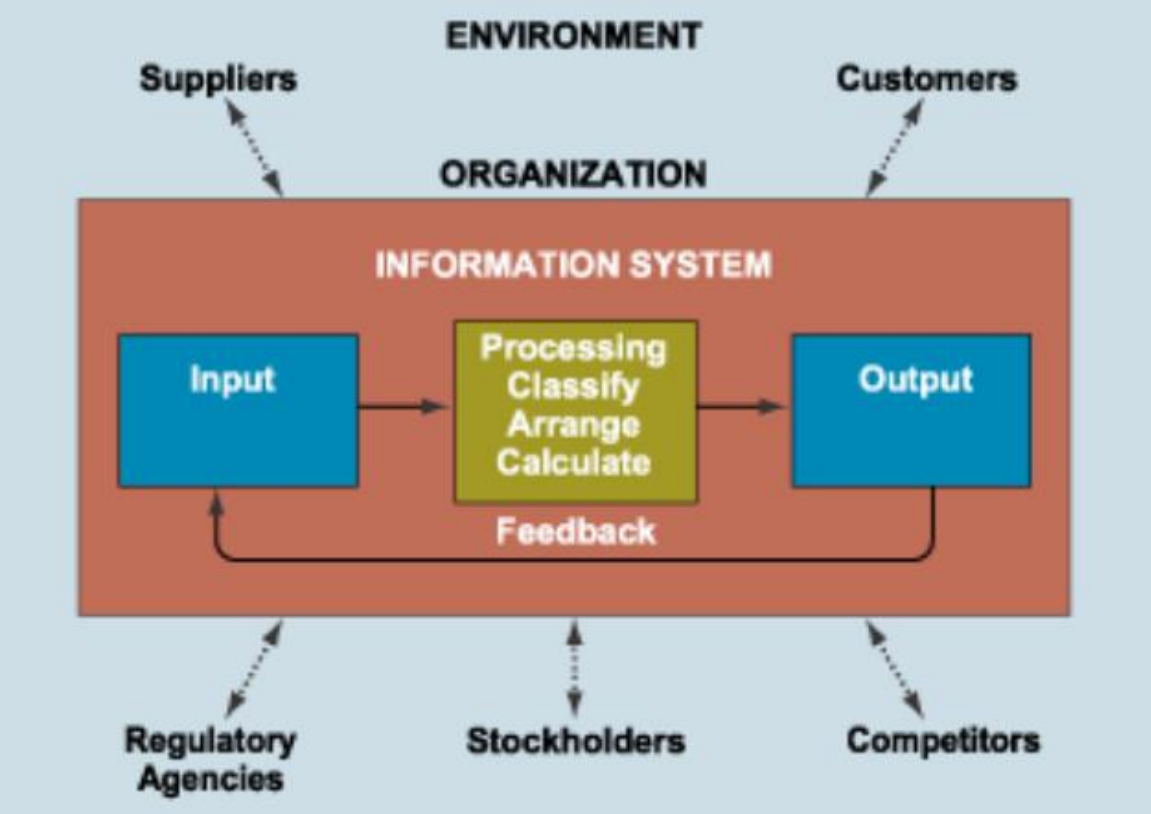
### 2.1.2 Management Information Systems as the Foundation of Centralization

The study's theoretical foundation is MIS. According to Laudon et al. (2026), an MIS is an organized system that collects, processes, stores, and distributes

information to support operational control, coordination, and administrative decision-making. Within apartment management, this framework explains why manual and semi-digital operations become inefficient: information is distributed across isolated records, disconnected tools, and inconsistent communication channels. Figure 2-1, adapted from Laudon et al. (2026), illustrates the flow from operational inputs through processing to outputs that support decision-making.

*Figure 2-1. Management Information System (MIS) framework, adapted from Laudon et al. (2026).*

[IMAGE FILE: images/Figure 2-1 - Management Information System (MIS) framework, adapted from Laudon et al. (2026).png]




Ghasemaghaei et al. (2018) find that a firm's data analytics competency, which includes the quality of its data, improves its decision-making performance.

Pressman and Maxim (2020) explain that integrated software systems improve consistency, maintainability, and operational reliability by reducing redundancy and streamlining data flow across system components.

Within apartment management, these principles support a centralized design that keeps tenant information, maintenance records, communication, and financial transactions in one system. Property management studies point the same way, locating operational inefficiency where information systems lack interoperability and centralized accessibility (Burke, 2026; Magno et al., 2024).

### 2.1.3 Financial Analytics and Administrative Decision Support

Financial management is central to apartment administration, since payment monitoring, expense tracking, and cash flow visibility shape managerial decisions. The reviewed studies emphasize that financial inefficiencies emerge when payment records are kept through fragmented or manual processes (Encarnacion et al., 2025; Magno et al., 2024).

Encarnacion et al. (2025) built a web-based payment tracking system for boarding houses near a university in Butuan City, where rent had been tracked through notes and verbal reminders, with delayed payments and little transparency as a result. In their system, landlords create bills, track payments, and monitor each tenant's balance, and tenants see their own bills and payment history. Evaluated with ISO/IEC 25010, it simplified payment tracking, reduced manual work, and improved communication between landlords and tenants. Magno et al. (2024) likewise built their system to compute monthly bills, track billing transactions, and generate a billing statement for each unit in place of paper-based

records. Caparos et al. (2025) developed a cloud-based property management system with automated billing, secure access controls, and real-time tenant communication for a real estate business in Region XII, which its staff rated positively on usability, security, operational efficiency, and tenant experience, security highest.

Read with Ghasemaghaei et al. (2018), these studies treat financial records not merely as reports but as decision support, most useful when they are integrated with the other operational records rather than kept as a standalone feature.

### 2.1.4 Digital Communication and Structured Issue Resolution

Operational efficiency in apartment environments also depends on communication and maintenance coordination. The reviewed studies identify informal communication as a contributor to delayed responses, inconsistent issue tracking, and reduced tenant satisfaction (Jing & Lim, 2021; Saputra et al., 2025).

Jing and Lim (2021) found, among residents of high-rise apartments in China, that management service quality affects residential satisfaction and the willingness to recommend, and that the quality of the management process matters most. Interviewing 150 dormitory residents, Altinay et al. (2024) found that dormitory management needed improvement in information management, communication, and service quality, and proposed digital communication platforms, regular maintenance schedules, and more transparent financial management.

Saputra et al. (2025) built a web-based room management system for a boarding house that handles room data, tenant registration, payment invoices, and complaints. In acceptance testing with its administrator and ten tenants, 90 percent were satisfied, and the system reduced administrative workload and data entry errors.

Communication with prospective tenants has followed the same pattern. In Ormoc City, where rooms in boarding houses were found through newspapers, posters, friends, and social media, Pernate et al. (2025) built a mobile application with real-time listings, booking, and digital transactions, developed with Agile and evaluated with ISO/IEC 25010, which owners rated Excellent and tenants Very Good. In Surigao City, Mangca (2023b) developed a web-based boarding house booking system with SMS notifications, and its tenants and owners credited the notifications with reducing miscommunication (Mangca, 2023a). Setty (2022) built a website on which homeowners list their properties and prospective tenants register, search for a home, and contact its owner. Each centers on how a prospective tenant finds and books a room.

Closest to this study, Canlas et al. (2026) developed FixHub, a maintenance reporting system for dormitories in the Philippines, using the Agile methodology, and evaluated it with ISO/IEC 25010 and the Technology Acceptance Model through surveys of tenants, administrators, and IT professionals, who rated it very highly on every criterion. Its scope is maintenance concerns, emergency alerts, and complaints.

The literature therefore treats structured communication and maintenance ticketing as integral parts of an apartment management system rather than supplementary features: a concern recorded in one system can be traced, coordinated, and followed to completion.

### 2.1.5 Progressive Web Applications as an Accessibility and Integration Framework

A centralized system also has to be reachable on the devices and connections its users have. PWAs provide a technological framework for meeting that need in resource-constrained environments (Biørn-Hansen et al., 2017; Muawwal, 2024).

Muawwal (2024) shows that a PWA, built from a web app manifest and a service worker, gives a website the features of an installed application without a separate native application for each platform: it could be installed on mobile and desktop devices and remained accessible offline or on an unstable connection. Biørn-Hansen et al. (2017) likewise explain that PWAs reduce development complexity by letting a single application operate across multiple platforms without separate native implementations. Because a PWA is used like an installed application but delivered as a website, its usability has also been given heuristics of its own (Anuar & Othman, 2024).

These characteristics matter in small-scale apartment operations, where limited infrastructure, inconsistent connectivity, and varied devices can keep conventional enterprise systems out of reach. Mia et al. (2024) found that small Philippine enterprises adopted the digital tools they already knew and could use at

no upfront cost, such as social media, and that what held back adoption was less money or skill than the owners' hesitation and readiness, including doubt that a tool fits how the business already works. Accessibility, simplicity of deployment, and a fit with existing practice are therefore critical considerations. A PWA serves as the delivery layer that keeps the centralized functions of an MIS accessible across devices and conditions.

### 2.1.6 Synthesis of Literature

The reviewed literature shows that the inefficiencies of apartment management come mainly from fragmented administration rather than from a complete absence of digital technologies. Property management studies find systems that do not connect with one another and records that are disorganized and inaccurate (Burke, 2026; Magno et al., 2024). Information systems and software engineering literature links better decisions to the quality of the data an organization holds and analyzes (Ghasemaghaei et al., 2018) and credits integrated systems with reducing redundancy and streamlining the flow of data (Pressman & Maxim, 2020).

Financial studies further show that structured payment records improve administrative oversight and transparency (Encarnacion et al., 2025; Magno et al., 2024). Communication and maintenance studies emphasize that accountability improves when issues are resolved through structured digital workflows rather than informal communication channels (Jing & Lim, 2021; Saputra et al., 2025).

PWA literature shows how these integrated functions can stay accessible across varying device conditions (Biørn-Hansen et al., 2017; Muawwal, 2024),

which matters for small enterprises that hesitate over tools that do not fit their practice (Mia et al., 2024). Recent housing systems have also been evaluated with ISO/IEC 25010 (Canlas et al., 2026; Pernate et al., 2025), the quality model this study uses.

Yet the systems these studies describe are mostly built in isolation. They frequently address one function, such as financial management, communication, or tenant tracking, and so remain partial solutions that do not address how apartment management workflows depend on one another. The literature therefore points to the need for one system that centralizes administrative processes, formalizes communication, supports financial monitoring, and remains accessible in resource-constrained residential settings. Appendix R maps these studies by theme, from the problem to the gap this study fills.

### 2.2 Gap Bridged by the Study

The synthesis shows a research gap: the absence of an integrated apartment management system designed for small-scale, resource-constrained residential operations. Existing systems are commonly developed either as enterprise-oriented platforms or around one part of the work. Some center on how a prospective tenant finds and books a room (Mangca, 2023b; Pernate et al., 2025; Setty, 2022). Others digitize the owner's records, such as rental records and payments (Magno et al., 2024), or room data, tenant registration, invoices and complaints (Saputra et al., 2025). Puspasari et al. (2025), for example, developed a web-based boarding house information system that monitors room status and sends billing notifications through WhatsApp, but its only user is the boarding

house manager, so tenants cannot see their own bills or report a concern through it.

While these systems improve specific operational areas, small apartment and boarding house operations face conditions they were not built around: small portfolios that find it hardest to pay for better software and so keep systems that do not integrate (Burke, 2026), and owners who hesitate over a tool that does not fit how the business already works (Mia et al., 2024). As a result, many small operations continue to rely on manual documentation, spreadsheet-based records, informal communication channels, and disconnected payment records.

This study addresses this gap through Hivelet, a PWA-based apartment management system built for a small boarding house. It brings tenant and room management, booking and reservation management, financial tracking, and maintenance ticketing into one system, organized on MIS principles and usable across devices.

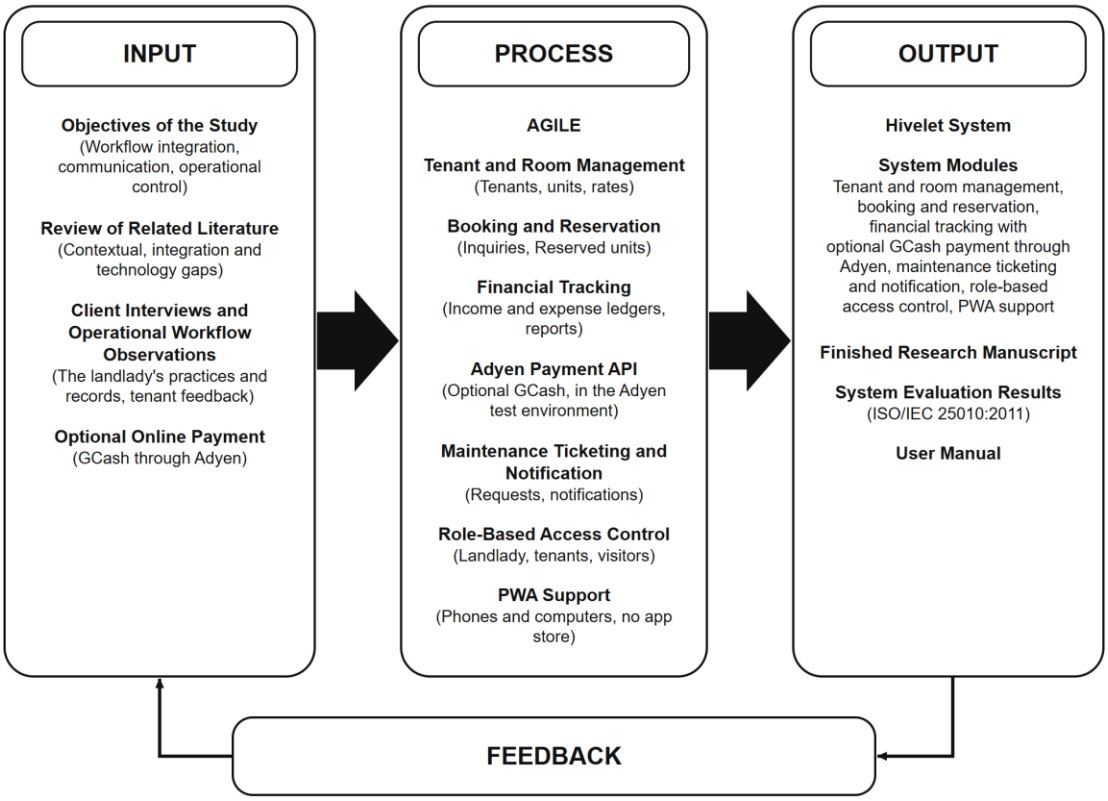
### 2.3 Concept of the Study

The concept of this study is that apartment management improves when its connected processes, namely tenant management, payment tracking, inquiries and reservations, and maintenance handling, move from separate manual handling into one centralized digital platform.

Figure 2-2 illustrates the study's conceptual framework using the Input-Process-Output (IPO) model, which maps how inputs are transformed into outputs.

*Figure 2-2. Conceptual Framework.*

[IMAGE FILE: images/Figure 2-2 - Conceptual Framework.png]




In this study, the Input phase consists of the study objectives, review of related literature, client interviews, needfinding observations, analysis of the landlady's existing administrative and payment workflows and records, and the requirement for an optional online payment through GCash. From these inputs, the researchers identified the operational gaps, user requirements, and system constraints of the boarding house.

The Process phase applies the Agile Software Development Life Cycle, with iterative stages of requirements, design, development, testing, deployment, and review (Section 3.2). Within this phase, the six features named in Objective 2 are implemented: tenant and room management (tenants, units and rates), booking

and reservation management (inquiries and reserved units), financial tracking (income and expense ledgers and reports) with an optional online payment integration using the Adyen API with GCash, maintenance ticketing and notification, role-based access control for the landlady, tenants and visitors, and Progressive Web Application support.

The Output phase includes the developed Hivelet system, its functional modules, and generated outputs such as financial reports exportable as Excel files, inquiry and tenancy records, and payment records. It also includes the completed research manuscript, system evaluation results based on ISO/IEC 25010:2011, and the user manual. Feedback from the outputs, chiefly the users' evaluation and the system's operational performance, returns to the input, so that each iteration starts from what the last one found.

### 2.4 Definition of Terms

**Hivelet**: The web-based apartment management system, delivered as a PWA, that this study developed for the Fe Galang Da Silva Boarding House to keep tenant and room management, inquiries and reservations, financial tracking, and repair requests in one system.

**Progressive Web Application (PWA)**: The form in which Hivelet is delivered: a website that the landlady or a tenant can add to a phone's home screen from the browser, without an app store, and that shows the figures last loaded when there is no connection.

**Resource-Constrained Environment**: In this study, a small residential operation such as the Fe Galang Da Silva Boarding House, with limited money,

staff, and technical infrastructure, whose management relies on a mix of manual and basic digital tools rather than an integrated system.

**Fragmented Workflow**: The handling of tenant records, finances, and repair requests in disconnected tools (spreadsheets, paper records, and messaging applications), which leaves data inconsistent and coordination delayed.

**Centralized System**: In this study, one database and one website holding every record the landlady had kept in separate files and channels: tenants, units and rates, payments, expenses, repair requests and inquiries.

**Small-Scale Apartment Management**: The administration of a resource-constrained environment as defined above: in this study, the 33 units of the Fe Galang Da Silva Boarding House, run by the landlady as the system's only administrator.

**Management Information System (MIS)**: The view of Hivelet taken in Chapter 2: a system that turns the landlady's raw records, such as receipts, expenses and repair requests, into the summaries she reads on the Overview and in the downloaded workbooks.

**Inquiry**: A message a prospective tenant sends about a unit through the public website, without an account. The landlady answers it in the system, may mark the unit Reserved so that it takes no other inquiry, and if the person rents, moves them in from it. This is the form booking and reservation management takes in this study: nothing is booked or paid for online.

**Financial Tracking**: The Hivelet module in which the landlady records tenant payments and operating expenses, the figures that Cash Flow Monitoring summarizes.

**Cash Flow Monitoring**: In this study, the landlady's view of money in and out: income, operating expenses and net income by month on the Overview, and the monthly income and expense workbooks she downloads.

**Maintenance Ticketing**: The Hivelet module through which a tenant sends a repair request, which the landlady assigns, tracks, and updates until it is resolved.

**User Authentication**: Signing in with a login ID or phone number and a password. The landlady creates each tenant's account with an individual starting password, which the tenant replaces at the first sign-in.

**Role-Based Access Control (RBAC)**: The rule that each of Hivelet's three kinds of user, the public, tenants and the administrator, can use only the functions given to that role, checked by the server on every request (Section 4.2.6).

**Optional Online Payment Feature**: The Hivelet function through which a tenant may pay a bill with GCash through Adyen. A payment made this way is recorded as pending and counts only after the landlady verifies it. It supplements rent paid in person and recorded by the landlady, and does not replace it.

**ISO/IEC 25010:2011**: The international software quality standard (International Organization for Standardization & International Electrotechnical Commission [ISO/IEC], 2011a) against which this study evaluated Hivelet. It defines eight characteristics of product quality: functional suitability, performance

efficiency, compatibility, usability, reliability, security, maintainability and portability, divided into thirty-one sub-characteristics. It also defines a model of quality in use, with five characteristics: effectiveness, efficiency, satisfaction, freedom from risk and context coverage. The 2023 revision renamed usability as interaction capability and portability as flexibility, added safety as a ninth characteristic, and moved quality in use into a separate standard (ISO/IEC, 2023).

**Adyen**: The payment gateway through which a tenant may pay a bill online with GCash. In this study it runs in Adyen's test environment, where it works end to end but takes no real money. Going live needs the owner's merchant onboarding documents, which were outside the study.

**GCash**: A Philippine mobile wallet, offered in Hivelet as the optional online way to pay a bill. It is not live, for the reason given under Adyen, so rent is paid in person and recorded by the landlady.

**Landlady**: The person who runs the Fe Galang Da Silva Boarding House day to day on behalf of its owner, and the system's only administrator.

**Owner**: The proprietor of the Fe Galang Da Silva Boarding House, for whom the landlady runs it. Taking the GCash payment live needs the owner's merchant onboarding documents (Section 1.4).

**Tenant**: A person renting a unit, who uses the system through their own account to view their bills, receipts and repair requests and to pay online.

**Prospective Tenant**: A person looking for a unit who uses only the public website, without an account, to view the units and their rates and to send an inquiry.

**Technical Evaluator**: An information technology professional from outside the research team who rated the system against ISO/IEC 25010:2011.

**User Acceptance Testing (UAT)**: Testing in which the intended users perform their own tasks on the live system, to decide whether it meets their needs (Section 4.3.7).

**Weighted Mean**: The average rating of a survey item: the sum of each rating multiplied by the number of respondents who chose it, divided by the number of respondents (Section 3.3).

**50% Share**: A system-computed figure equal to half that row's Rent Amount, kept for ledger parity with the owner's historical spreadsheet.

# 3 MATERIALS AND METHODS

This chapter presents the tools and methods used to develop the Hivelet system, and the instruments and procedures used to test and evaluate it.

## 3.1 Materials

This section presents the software and hardware used to design, develop, test and deploy Hivelet, a Progressive Web Application (PWA).

### 3.1.1 Software

Table 3-1 lists the software used to develop the system and the version of each.

**Table 3-1.** Software requirements

| Software | Version |
|---|---|
| Visual Studio Code | 1.85+ |
| Vue 3 with TypeScript | 3.5 / TypeScript 5.7 |
| Vite (build tool) | 5.4 |
| Tailwind CSS | 4.3 |
| Node.js with Express.js (TypeScript) | 22 LTS / Express 4.22 |
| PostgreSQL, hosted on Supabase | 17.6 |
| Adyen Web Drop-in / API Library | 6.44 / 32.0 |

The development environment ran on Windows 10 or higher.

The user interface was written in Vue 3 with TypeScript, built with Vite and styled with Tailwind CSS, so that its screens adapt to both phones and laptops. Node.js and Express.js, also written in TypeScript, formed the back end, which

holds the server-side logic for inquiries and reservations, financial tracking and repair requests. TypeScript was used on both sides so that errors in the shape of a record are caught before the system runs, which matters in a system that holds the landlady's actual financial records.

PostgreSQL, hosted on Supabase, was the database, holding tenant records, transaction history and the audit log. PostgreSQL was chosen for its row-level security, which lets the database itself enforce who may read or change each record, and for database triggers, which keep a history of every unit rate change however the change is made. Google Chrome was the main browser for development and testing.

Additionally, the user interface was prototyped directly in code rather than in a separate design tool. Each screen was built as a working prototype in Vue and Tailwind CSS, generated with the help of an AI coding assistant and reviewed by the researchers, so that the landlady and the tenants could try it on their own phones with real data. The design rules the screens follow, covering color, type, layout and wording, are recorded in the system's design guideline.

The landlady's past records were transferred from her own workbook rather than retyped. A transfer script written by the researchers read the workbook's two sheets, Monthly Income and Monthly Expenses, and loaded them into the database, and a verification script counted the rows loaded. Later months were added by a second script that compares every row with the database and adds only the rows not yet there. That script and the later corrections were written with the help of the same AI coding assistant and reviewed by the researchers. Each

correction was made as a numbered database migration, run after a backup, that records its evidence and the values before and after. The corrections make the records match what she wrote. Her amounts, and the five invoice numbers she had used twice, were kept as written.

### 3.1.2 Hardware

Because Hivelet is a PWA used on both computers and phones, its development and testing needed a workstation and a mobile device. Table 3-2 gives the minimum specification of the workstation used for local development, database management and testing.

**Table 3-2.** Workstation hardware specifications

| Component | Minimum specification |
|---|---|
| Processor | Intel Core i5 or AMD Ryzen 5 |
| Memory (RAM) | 8 GB DDR4 |
| Storage | 256 GB Solid State Drive (SSD) |
| Internet connection | 25 Mbps broadband or fiber |

The baseline for mobile testing, in Table 3-3, was a current mid-range Android phone, on which layout and response were evaluated.

**Table 3-3.** Mobile testing hardware specifications (baseline)

| Component | Specification |
|---|---|
| Operating system | Android 13 or higher |
| Processor or chipset | Octa-core (MediaTek Helio G99 or equivalent) |
| Memory (RAM) | 8 GB |
| Storage | 256 GB |
| Display | 6.78-inch FHD+ (for UI/UX layout testing) |

During development and testing, the workstations were the researchers' two Acer Nitro V15 laptops with Google Chrome, and the landlady recorded and tested on one of them. Response times were taken on a laptop over the boarding house's Wi-Fi and on an iPhone 15 in Safari over mobile data, and the offline, installation and browser tests also used an Infinix GT20 with Android and Chrome. The tenants used their own phones. Tables 3-2 and 3-3 are what developing and testing the system needed. Using it needs only a current web browser and an internet connection (Chapter 4, Table 4-21).

### 3.2 Software Methodology

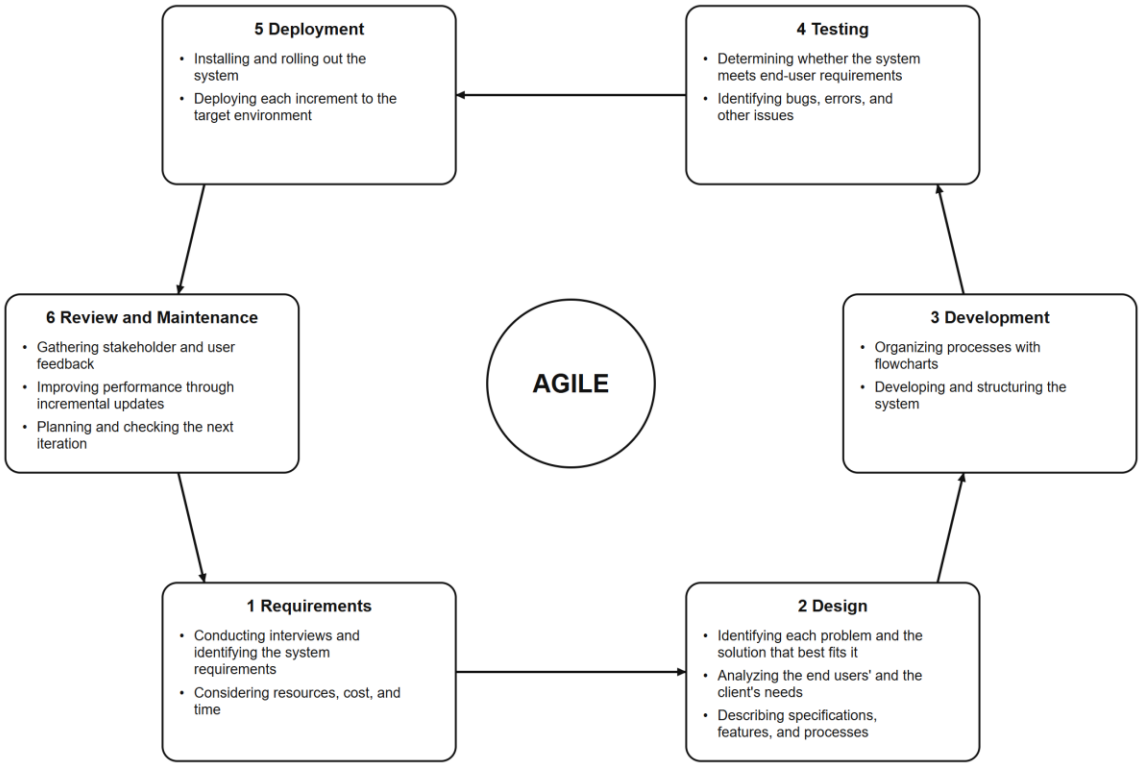
The researchers developed Hivelet with the Agile Software Development Life Cycle (SDLC). Agile is an iterative and flexible approach that emphasizes continuous improvement, stakeholder collaboration and incremental delivery rather than a rigid, linear process (Pressman & Maxim, 2020).

Agile was chosen because the boarding house's needs could change while the system was being built, and because it allowed continuous interaction with the landlady, so that the features followed her actual work: tenant management,

financial tracking, inquiries and reservations, and repair requests. Figure 3-1 shows the cycle's six phases, described in Sections 3.2.1 to 3.2.6.

*Figure 3-1. Agile Software Development Life Cycle.*

[IMAGE FILE: images/Figure 3-1 - Agile Software Development Life Cycle.png]




Unlike linear SDLC models, Agile lets issues be found early, because each iteration includes its own testing. The researchers refined the system from the feedback and test results of each iteration, and the system was evaluated during its development rather than only at the end (Section 3.3).

A first working system was built and the landlady's records were transferred into it. It was then developed in two iterations, each passing through the phases

described below (Appendix C). The first verified the system and its documents against each other and corrected the errors found, then put the system in the hands of the landlady and her tenants and refined it from their feedback, the testing and the first round of evaluation. The second redesigned the screens and added no module, and a second round of users and technical evaluators tried and rated the redesigned system.

### 3.2.1 Requirements

This phase served the first objective of the study, identifying the boarding house's needs and operational challenges. The researchers gathered data through interviews with the landlady, through observation, and through a review of her income and expense workbook. They examined her existing practices: manual ledgers, fragmented financial tracking and unstructured communication about maintenance requests. These showed inconsistent record-keeping, delayed communication and no central place to monitor the business. Written follow-up questions settled what the interviews had left open. From these findings the researchers defined Hivelet's requirements and scope: one central system for inquiries and reservations, financial tracking and maintenance requests.

### 3.2.2 Design

In this phase the requirements were turned into the system's architecture and into working prototypes of its screens, built in code as described in Section 3.1.1, for three kinds of user: the public, tenants and the administrator. The system was modeled with a context diagram, data flow diagrams in the Gane and Sarson

notation down to the second level, a use case diagram and an entity-relationship diagram (Chapter 4, Figures 4-3 to 4-6, and Appendix F). Each prototype was reviewed with the landlady on the device she uses, and the findings were applied in the next iteration (Chapter 4, Table 4-19). Because the boarding house has limited infrastructure, unstable internet and a single administrator, Hivelet was designed as a lightweight PWA with simple navigation and low device requirements. The workflows for inquiries and reservations, financial tracking and maintenance requests were mapped first and then refined from the feedback of the landlady and the tenants.

### 3.2.3 Development

This phase served the second objective of the study, developing the system. The researchers built it with the software described in Section 3.1.1, refining it in each iteration from feedback and test results. An optional online payment was added through the Adyen payment gateway, so that a tenant can pay a bill with GCash, while cash paid in person remains the main way to pay. It works end to end in Adyen's test environment, where it takes no real money. It is not live, because going live needs the owner's merchant onboarding documents.

### 3.2.4 Testing

This phase served the third objective of the study, pilot testing the system. Testing was done in the four parts below, at the test levels defined in ISO/IEC/IEEE 29119-1 (International Organization for Standardization, International Electrotechnical Commission, & Institute of Electrical and Electronics Engineers

[ISO/IEC/IEEE], 2022): unit and integration, system, and acceptance. Each test was planned as a measure of an ISO/IEC 25010 sub-characteristic listed in Table 3-6.

**Automated verification.** Twenty-one check suites were written alongside the system. They verify every endpoint and the separation of roles, re-derive the landlady's ledger and exported reports from the database, check the payment gateway's message signatures, and confirm that no screen presents stored or empty data as a live figure. They read the live system without writing to it, and were run after every significant change.

**Screen-versus-database audit.** Each figure shown on each screen was compared with a read-only query of the live database for the same records, to confirm that a screen shows a person what the database holds for them.

**Offline, installation, accessibility and simultaneous-use tests.** With an automated browser, the installed application was opened without a connection and on throttled slow connections, and the live system was read by several simulated simultaneous users, with every save blocked in the browser so that no record was written. The same browser scanned the screens against the Web Content Accessibility Guidelines, levels A and AA (World Wide Web Consortium, 2023).

**Functional walkthrough and user acceptance testing.** A 26-step walkthrough exercised once every function that writes data, on the one vacant unit (Appendix L). The landlady and three tenants then used the live system for their own tasks on their own devices, observed by the researchers, and the landlady and two of the tenants used it at the same time. One prospective tenant used the

public website on their own phone. In the second round, ten tenants and four prospective tenants did the same tasks on the redesigned system. Results are reported in Chapter 4, Section 4.3.

### 3.2.5 Deployment

The Hivelet system was deployed to Vercel, a cloud hosting service, with its database hosted on Supabase, and is reached through a public web address. On the adviser's advice the researchers hosted it themselves rather than on the university's server, so the boarding house needs no equipment beyond the landlady's own laptop and phone. The testing and the evaluation took place on this deployment, as a pilot in which the system runs beside her own records, which it has not yet replaced (Chapter 4, Section 4.5). The researchers did the configuration, database migrations and environment setup.

### 3.2.6 Review and Maintenance

In this phase the researchers corrected the issues found by testing and by the ISO/IEC 25010 evaluation. Chapter 4, Table 4-19 lists the changes made in response to the evaluation.

## 3.3 Evaluation Procedure

This phase directly supports the fourth objective of the study, evaluating and optimizing the system with the ISO/IEC 25010:2011 software quality model (ISO/IEC, 2011a). So that performance and usability were measured as the users meet them, the evaluation was conducted during actual use of the deployed system, on its eight characteristics: functional suitability, performance efficiency,

compatibility, usability, reliability, security, maintainability and portability. It followed the five steps of the evaluation process in ISO/IEC 25040: establishing the evaluation requirements, specifying the evaluation, designing it, executing it and concluding it (ISO/IEC, 2011b).

Each characteristic was judged on two kinds of evidence. Measured quality came from the tests in Section 3.2.4, each expressed in the manner of ISO/IEC 25023 as a ratio X = A / B, such as the number of test steps that gave the correct result over the number executed (ISO/IEC, 2016b). Rated quality came from the survey questionnaire, in which each item was assigned to one sub-characteristic (Table 3-6). Quality in use was measured during the user acceptance test as the tasks completed and the time and wrong turns per task, following ISO/IEC 25022 (ISO/IEC, 2016a).

**Respondents.** The survey questionnaire was given to four groups: the landlady as the system's only administrator, the tenants, prospective tenants, and technical evaluators with a background in information technology, none of whom was a member of the research team. The landlady runs the boarding house day to day on behalf of its owner. Tenants were chosen by purposive and convenience sampling (Etikan et al., 2016) to include tenants who pay in cash, tenants less used to mobile applications, users of both Android and iPhone handsets, and more than one cluster. Three were observed, since the number of new usability problems found falls quickly with each additional tester (Nielsen & Landauer, 1993). A prospective tenant used the public website as a person looking for a unit would. Five technical evaluators reviewed the system, four of them on a demonstration

copy holding sample data, so that they did not see the boarding house's records. After the screens were redesigned in the second iteration, the tasks and the questionnaire were repeated with a second round of tenants, prospective tenants and the landlady, and three more technical evaluators, IT professionals and developers, reviewed the live system and its source code for about 30 minutes each and then answered the questionnaire. Each group rated only the characteristics it was in a position to judge: tenants and prospective tenants did not rate Security, and only the technical evaluators rated Maintainability.

**Instruments.** Four instruments were used. Task cards were read aloud to each tester in English and Filipino. An observation sheet recorded, for each task, whether it was completed without help, with help or not at all, the time taken and the number of wrong turns. A defect log recorded the defects found, and the ISO/IEC 25010 survey questionnaire had a section for each respondent group (Appendix J). The research adviser reviewed the questionnaire before it was used. Responses were given on the 5-point Likert scale of Table 3-4 (Likert, 1932).

**Data gathering.** Each participant took part after giving consent (Section 3.4). Each tester used their own device, the landlady and the tenants on their own accounts, while a facilitator read the tasks without indicating where to tap and an observer completed the observation sheet. When a tester was stuck, help followed a fixed order: a neutral prompt, then a hint, then a demonstration. A task finished after the prompt counted as completed without help, one finished after the hint as completed with help, and one that needed a demonstration as not completed. Each respondent answered the survey after using the system, without the researchers

viewing their answers. Page load times were measured on the devices named in Section 3.1.2, with the browser's cache cleared for a first load.

**Statistical treatment.** For each survey item the weighted mean was computed as

WM = Σ(f × w) / N

where *f* is the number of respondents choosing a rating, *w* the rating (1 to 5) and *N* the number of respondents, and interpreted using Table 3-5. The composite mean of each characteristic is the mean of its group means, so that the landlady is not outweighed by several tenants on functions only she uses. The users' composites, from the landlady, the tenants and the prospective tenants, are reported beside the technical evaluators', and each overall mean is the mean of the composites its respondents rated. An answer was counted only when it came from a respondent and was given in the section of the questionnaire written for that respondent's own group. For the acceptance test, the task completion rate is the number of tasks completed without help divided by the number attempted, reported with the median time per task and the mean number of wrong turns. The results are descriptive, and no inferential test was made.

**Table 3-4.** Likert scale

| Scale | Meaning | Description |
|---|---|---|
| 5 | Strongly Agree | The respondent strongly agrees with the statement |
| 4 | Agree | The respondent agrees with the statement |
| 3 | Neutral | The respondent neither agrees nor disagrees with the statement |
| 2 | Disagree | The respondent disagrees with the statement |
| 1 | Strongly Disagree | The respondent strongly disagrees with the statement |

Table 3-5 divides the range from 1 to 5 into five equal intervals of 0.80.

**Table 3-5.** Interpretation of mean scores

| Mean score | Verbal interpretation |
|---|---|
| 4.21 – 5.00 | Very High Quality |
| 3.41 – 4.20 | High Quality |
| 2.61 – 3.40 | Moderate Quality |
| 1.81 – 2.60 | Low Quality |
| 1.00 – 1.80 | Very Low Quality |

Table 3-6 assigns every survey item and every test to the sub-characteristic it measures. Item numbers are those of each group's section of the questionnaire (O = administrator, T = tenants, E = technical evaluators, P = prospective tenants).

**Table 3-6.** Survey items and tests assigned to the ISO/IEC 25010:2011 sub-characteristics

| Characteristic | Sub-characteristic | O | T | E | P | Measured by (Section 3.2.4) |
|---|---|---|---|---|---|---|
| Functional Suitability | Functional completeness | 1 | 1, 3 | 1 | 1 | Requirements traceability; walkthrough |
| | Functional correctness | 2, 3, 4, 6 | | 2, 3, 5 | | Automated verification; screen-versus-database audit; walkthrough |
| | Functional appropriateness | 5 | 2 | 4 | 2, 3 | User acceptance test |
| Performance Efficiency | Time behaviour | 7, 8 | 9, 10 | 6, 7 | 7, 8 | Load and response times on real devices |
| | Resource utilization | | | 8 | | Lighthouse; size of the code downloaded |
| | Capacity | 9 | | | | Simultaneous-user test |
| Compatibility | Co-existence | 12 | 15 | 11 | | (rated only) |
| | Interoperability | 10, 11 | 14 | 9, 10 | 10 | Browser matrix; gateway checks; export checks |
| Usability | Appropriateness recognizability | 13 | 4 | 12 | 4 | User acceptance test |
| | Learnability | 17 | 8 | 16 | | User acceptance test |
| | Operability | 14 | 5, 6 | 13 | 5 | User acceptance test |
| | User error protection | 15, 16 | 7 | 14, 15 | 6 | Walkthrough |
| | User interface aesthetics | | | | | (not evaluated) |
| | Accessibility | | | | | Accessibility scans (axe-core; WCAG 2.0 to 2.2, A and AA) |

| Characteristic | Sub-characteristic | O | T | E | P | Measured by (Section 3.2.4) |
|---|---|---|---|---|---|---|
| Reliability | Maturity | | 12 | | 9 | Defect log |
| | Availability | 18 | 11 | | | (rated only) |
| | Fault tolerance | 20 | 13 | 17, 19 | | Offline and weak-connection tests; walkthrough |
| | Recoverability | 19, 21 | | 18, 20 | | Live use; offline tests |
| Security | Confidentiality | 22, 23 | | 21, 22 | | Walkthrough; manual protection checks; code review |
| | Integrity | | | | | Gateway signature checks; code review |
| | Non-repudiation | | | | | Audit trail |
| | Accountability | 24 | | 23 | | Audit trail |
| | Authenticity | 25 | | 24, 25 | | Manual protection checks; passive external scans |
| Maintainability | Modularity | | | 30 | | Permission matrix check |
| | Reusability | | | | | (not evaluated) |
| | Analysability | | | 26, 27 | | Migration history |
| | Modifiability | | | 28 | | Configurable settings |
| | Testability | | | 29 | | Automated verification |
| Portability | Adaptability | 26 | 16, 17 | 31 | 11 | Screen widths; devices used |
| | Installability | 27, 28 | 18 | 32, 33 | 12 | Installation tests |
| | Replaceability | | | | | Export in the landlady's layout |

## 3.4 Ethical Considerations

The study used the landlady's real records and the tenants' real accounts, so it followed the Data Privacy Act of 2012 (Republic Act No. 10173, 2012). Participation was voluntary and could be withdrawn at any time without any effect on a tenant's tenancy. The landlady and each tenant signed the informed consent form (Appendix P), and the prospective tenants and the technical evaluators agreed through the data privacy notice that opens the questionnaire. The four technical evaluators reached through Panelista, a platform that matches capstone projects with volunteer evaluators, reviewed a demonstration copy holding sample data, so none of them saw the landlady's records. The three of the second round reviewed the live system, which a member of the research team signed into as the test tenant and as the administrator and handed to them ready to use, so they could see the boarding house's records but were given no sign-in details. That member stayed with them throughout. The landlady is the mother of a member of the research team. Her answers are therefore reported as one respondent's, beside those of the tenants, the prospective tenants and the technical evaluators, and every characteristic was also measured by tests that do not depend on what any respondent said. Each tenant used only their own account, which shows only their own records, and received an individual starting password in person, which they replaced at first sign-in. Respondents could give a name so that the researchers could tell the responses apart. The names were removed before the responses were analyzed, and none appears in this paper. Names, contact details and amounts are hidden in every figure in this paper, and faces appear only with

consent. Photographs and notes were kept by the researchers and will be deleted after the defense.

# 4 RESULTS AND DISCUSSION

This chapter presents the results of the study in the order of the four specific objectives in Section 1.2: the analysis of existing practices (4.1), the development of the system (4.2), pilot testing (4.3), and evaluation and optimization against ISO/IEC 25010 (4.4), followed by the deployment plan (4.5). As described in Section 3.3, every quality characteristic is judged on two kinds of evidence: what was measured in testing and how the system's users rated it.

## 4.1 Analysis of Existing Apartment Management Practices and System Requirements

*Answers Objective 1.*

The Fe Galang Da Silva Boarding House has 33 rentable units in five clusters: the Boarding House (22 units), the Back Apartment (5), the Front Apartment (3), the Penthouse (1) and Linda (2), on three residential floors and a rooftop penthouse level (12, 11, 9 and 1 units). Thirty-two of the 33 units are let to the boarding house's tenants.

### 4.1.1 Existing Practices and Identified Problems

Before the system, the landlady kept a spreadsheet workbook with two sheets, a Monthly Income Report and a Monthly Expenses Report, on a USB flash drive, together with a paper invoice book. Rent was collected in cash on site, and repair requests arrived by messaging application or in person. Table 4-1 compares each practice with the problem it caused. Inquiries arrived much as Pernate et al.

(2025) found for boarding houses in Ormoc City, through social media and word of mouth, and left no record.

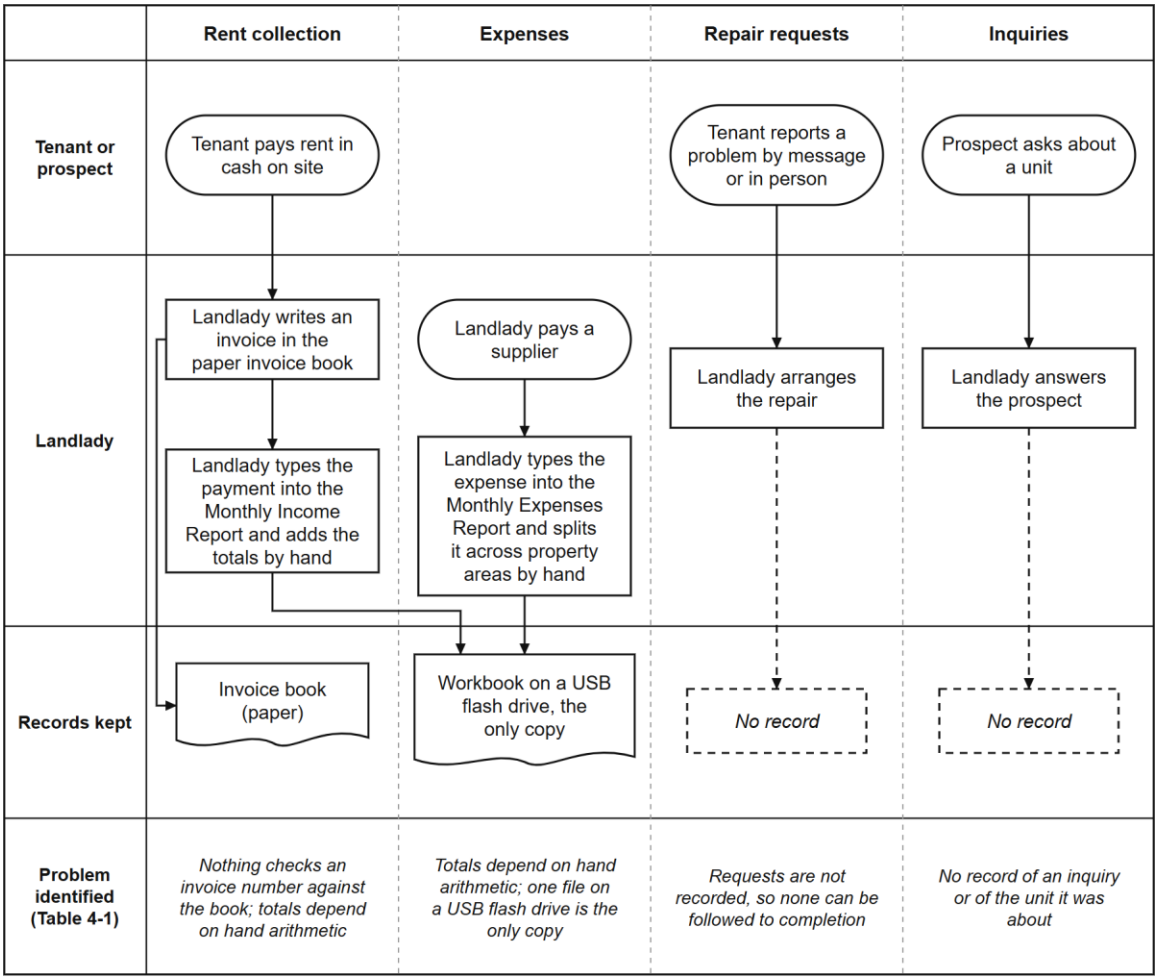
**Table 4-1.** Existing practices and identified problems

| Area | Existing practice | Problem identified |
|---|---|---|
| Tenant management | Tenant details kept in the income sheet and in the landlady's memory | No single record of who lives in which unit, since when, or how many occupants |
| Financial tracking | Income and expense sheets typed by hand; invoices written in a paper book | Totals depend on manual arithmetic; nothing checks an invoice number against the book; one file on a USB flash drive is the only copy |
| Communicati on | Requests sent by messaging application or said in person | Requests are not recorded, so there is no way to follow a request to completion |
| Booking | Inquiries arrived through Facebook, referrals from other people, contact and text messages, or walk-ins in person | No record of inquiries or which unit an inquiry was about |

Figure 4-1 follows each area through the hands that touched it. Every path ends in a record kept apart from the others, or in no record at all, which is where each problem in Table 4-1 begins.

*Figure 4-1. The landlady's process before the system.*

[IMAGE FILE: images/Figure 4-1 - The landlady's process before the system.png]




The strongest evidence came from the landlady's own records. Transferring her workbook into the system (937 income rows and 1,327 expense allocations, from January 2024 to July 2026) exposed problems the spreadsheet had hidden. **402 of the 937 income rows (43%)** had no anniversary date and no deposit, and **five invoice numbers** were each used for two different payments, once for two tenants on the same day (Section 4.3.1). The main finding is that the problems did not sit in one area. **Each area kept its own informal record, and nothing**

**connected one to another**, the fragmentation described in Chapter 2 (Burke, 2026; Magno et al., 2024).

### 4.1.2 System Requirements

The analysis produced **44 functional requirements** and **49 business rules**, each traced in a traceability matrix to the part of the system that implements it. Graded against the code early in the first iteration, 25 of the 44 were implemented as worded. Graded again by the same criteria against the final system, on 13 November 2026, 33 are implemented as worded, 9 in part and 2 not as worded (Appendix M, Table M-32). What those two describe, the income, expenses and net income and their trend by month, is shown on the landlady's Overview, worked out on the screen rather than by the server. Table 4-2 traces each gap to the requirement and the module that meets it.

**Table 4-2.** Identified gaps, system requirements and implementing modules

| Identified gap | System requirement | Module (Section) |
|---|---|---|
| No single tenant and unit record | Keep one record per unit and per tenant, with occupancy and move-in dates | Tenant and Room Management (4.2.2) |
| Inquiries not recorded | Let the public view units and send an inquiry about a specific unit | Booking and Reservation (4.2.3) |
| Manual arithmetic and unchecked receipts | Compute charges, record payments against the landlady's own invoice numbers, reproduce the landlady's reports | Financial Tracking (4.2.4) |
| Cash only, no digital option | Offer an optional online payment that the landlady verifies before it counts | Financial Tracking (4.2.4) |
| Requests not followed to completion | Record each request and its status until the landlady closes it | Maintenance Ticketing and Notification (4.2.5) |
| Anyone with the file sees everything | Give the landlady and each tenant access to only what their role needs | Role-Based Access Control (4.2.6) |
| One file on one device | Make the system reachable from any phone or computer | Progressive Web Application (4.2.7) |

The analysis also set ten non-functional requirements, one for each ISO/IEC 25010 characteristic and two for auditability and exportability. Table 4-3 lists them with the section that reports how each was met.

**Table 4-3.** Non-functional requirements and where each is measured

| Code | Requirement | Measured in |
|---|---|---|
| NFR-001 | The system shall provide the required business functions accurately | Sections 4.3.1 to 4.3.4; Section 4.4.3 |
| NFR-002 | The system shall be understandable to a non-technical administrator | Section 4.3.7; Section 4.4.3 (Usability) |
| NFR-003 | The system shall protect personal and financial information | Section 4.2.6; Section 4.4.4 |
| NFR-004 | Common operations should respond efficiently under the expected workload | Table 4-10; Appendix M, Tables M-25 and M-31 |
| NFR-005 | The system shall function on modern desktop and mobile browsers | Table 4-21; Section 4.4.3 (Compatibility) |
| NFR-006 | The system shall preserve data consistency and avoid silent data loss | Sections 4.3.1, 4.3.3 and 4.3.6 (Appendix M, Table M-26) |
| NFR-007 | The codebase shall use a modular, documented, maintainable architecture | Section 4.4.3 (Maintainability): partly met |
| NFR-008 | The system shall be deployable to a hosting environment the researchers can maintain | Section 4.2.1; Table 4-21 |
| NFR-009 | Important financial and administrative changes shall be traceable | Section 4.2.6 (audit trail) |
| NFR-010 | Relevant business data shall be usable outside the application | Section 4.2.4 (Excel downloads) |

NFR-008 originally named the university's server. On the adviser's advice the researchers deployed the system on its own hosting instead (Section 3.2.5), and the requirement was reworded to match.

Where a requirement needed the landlady's answer, it was asked rather than assumed. She confirmed that a bill is overdue from the day after its due date, and that rates are set by hand, so no increase is applied automatically. She also confirmed the rate of every unit, each of which agreed with her receipts, and that tenants do not report cash payments, because the landlady records every payment.

## 4.2 Development of the Hivelet System

*Answers Objective 2. Each feature the objective names has its own subsection.*

Figures 4-2 to 4-6 model the system as built, and Figures 4-7 to 4-11 show its screens after the redesign of the second iteration. The names, payments, expenses and repair requests in the screen figures are sample records, so that no real tenant's name or payment appears in any figure.

### 4.2.1 System Architecture and Technology Stack

The system was developed in two Agile iterations (Section 3.2; Appendix M, Table M-19). The first checked the system and its documents against each other, corrected 22 recorded errors and added automated checks, then put the system in the hands of the landlady and her tenants and refined it from their feedback, the testing and the evaluation. The second redesigned the screens, added no module, and added two rules: a record dated in a closed year can no longer be changed, and a tenant can no longer cancel a repair once someone is sent. After the redesign, the 114 test cases were run again: 99 passed, the 10 timings were taken, 5 could not be run, and none failed, so the results of the first iteration stand (Appendix L).

The evaluation against ISO/IEC 25010 was carried out twice, each time in actual use as Section 3.3 requires: in the first iteration, and again in the third on the redesigned system, with a second round of users and of technical evaluators (Sections 4.3.7 and 4.4). Section 4.4 reports the second and gives the first for comparison. Table 4-4 lists the technology used.

**Table 4-4.** Technology stack of the developed system

| Layer | Technology | Purpose |
|---|---|---|
| Presentation | Vue 3.5 with TypeScript 5.7, built with Vite 5.4, styled with Tailwind CSS 4.3 | The screens used by the public, tenants and the landlady |
| Application | Node.js with Express 4.22 in TypeScript | Business rules, calculations and access checks |
| Data | PostgreSQL 17.6, hosted on Supabase | Storage of all records, with row-level security |
| Payment gateway | Adyen (Web Drop-in 6.44, API Library 32.0) with GCash | The optional online payment, in Adyen's test environment, which takes no real money |
| Client delivery | Progressive Web Application (vite-plugin-pwa) | Installation on a phone or computer without an app store |
| Hosting | Vercel | Serves the application at a public web address |

Figure 4-2 shows where each part runs. The application reaches the browser as a Progressive Web Application and talks to one server program, an Express API on Vercel. Only that API reads or writes the database. Row-level security is on for every table, so a request that does not pass the API's sign-in and role checks reaches no data. Payment details go from Adyen's own form to Adyen directly, and every change to the database is a numbered, reviewable migration. The system is hosted rather than run on a server at the property or the university (Section 3.2.5), so the boarding house needs no equipment beyond the landlady's own laptop and phone.

*Figure 4-2. System architecture.*

[IMAGE FILE: images/Figure 4-2 - System architecture.png]

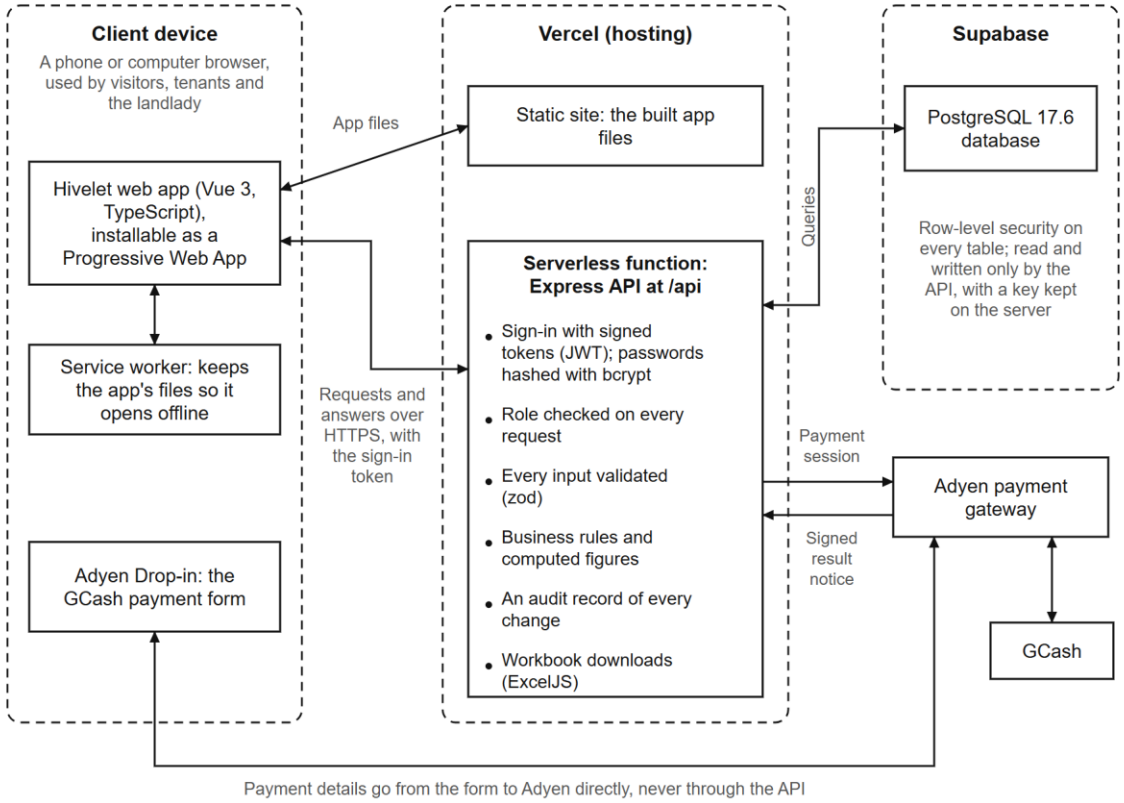



Figure 4-3 is the context diagram: the system as one process, with the three parties outside it and the data that passes between them. Figure 4-4 opens it into eight processes and the stores each reads and writes, in the Gane and Sarson notation. A rounded rectangle is a process, a rectangle open on the right is a data store, and a shaded square is a party outside the system. To trace one path, the landlady records a cash receipt through process 4.0, Bill and record payments, which writes it to store D3, Bills and receipts. Process 7.0, Produce reports, reads it from there for the Overview and the Excel workbooks. The notifications and the audit record are written by nearly every process and are left out of Figure 4-4 for readability. The units store, D1, is drawn twice, the second time under processes

5.0 and 4.0, so that its lines cross no others. Adyen, the payment gateway, is drawn as part of the payment process rather than as a party outside the system, as in the data flow diagrams the researchers' Systems Analysis and Design professor approved. Taking a GCash payment through Adyen is step 4.3 of process 4.0 (Appendix F), and Figure 4-2 shows where the gateway sits.

*Figure 4-3. Context diagram.*

[IMAGE FILE: images/Figure 4-3 - Context diagram.png]

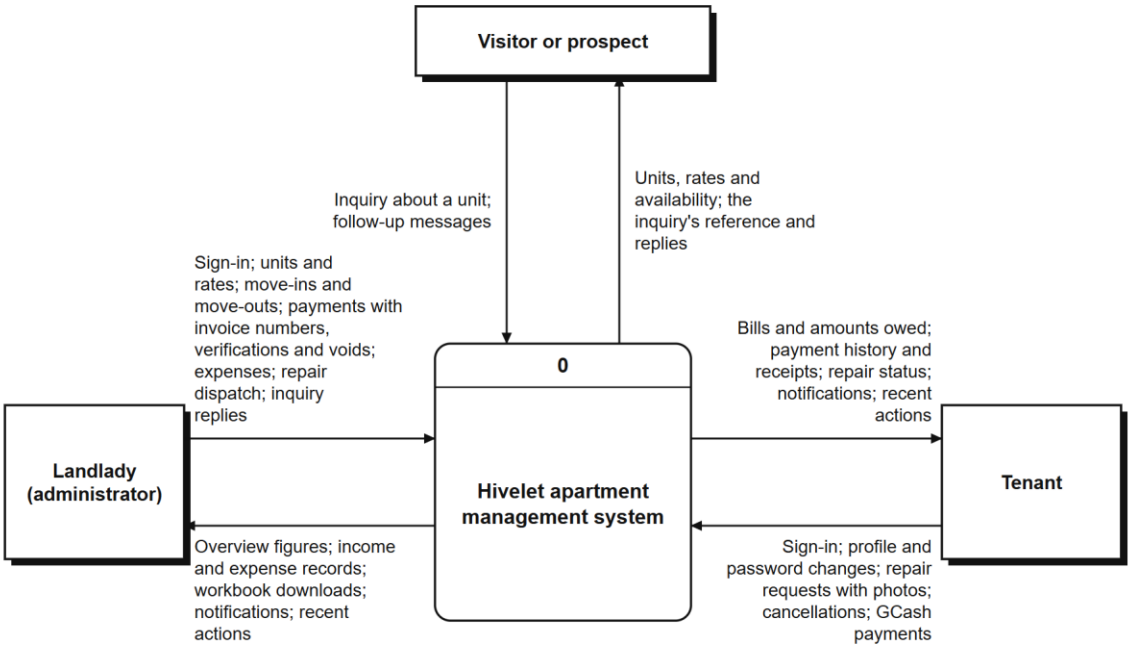


*Figure 4-4. Level 1 data flow diagram.*

[IMAGE FILE: images/Figure 4-4 - Level 1 data flow diagram.png]

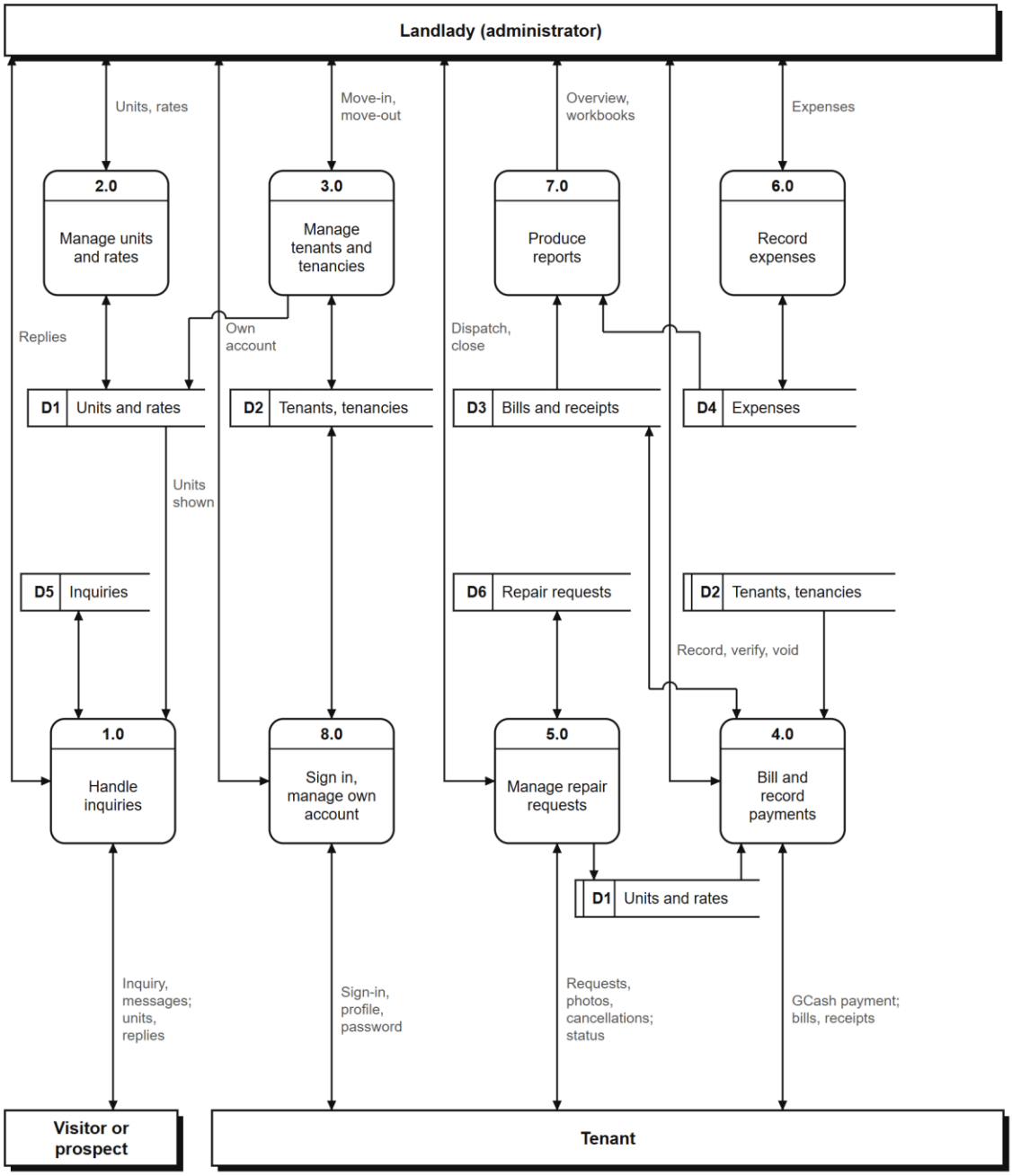



Figure 4-5 shows the same functions from the side of the people who use them. Each use case is a function the server grants to that role and refuses to every other (Table 4-7).

*Figure 4-5. Use case diagram.*

[IMAGE FILE: images/Figure 4-5 - Use case diagram.png]

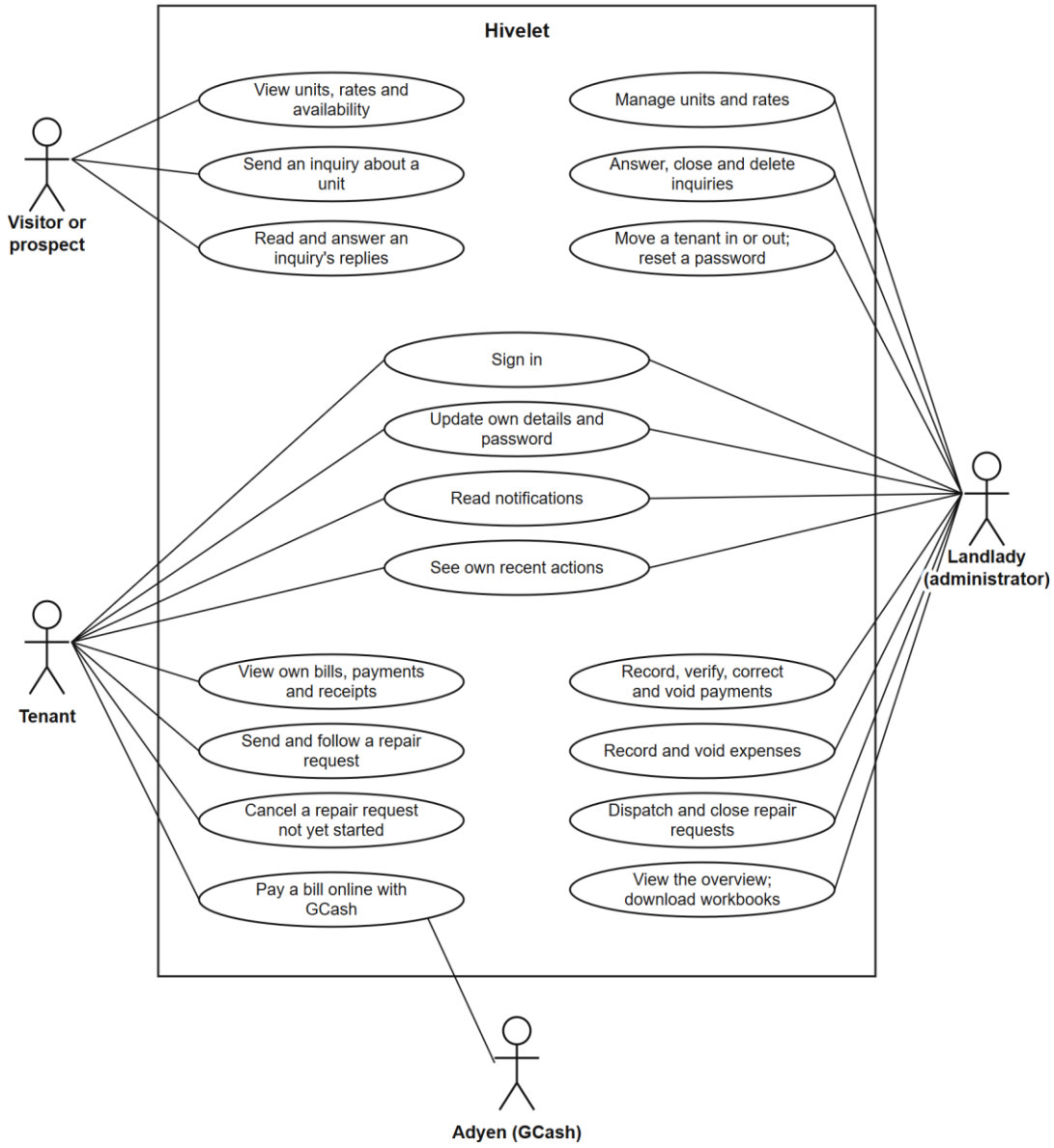
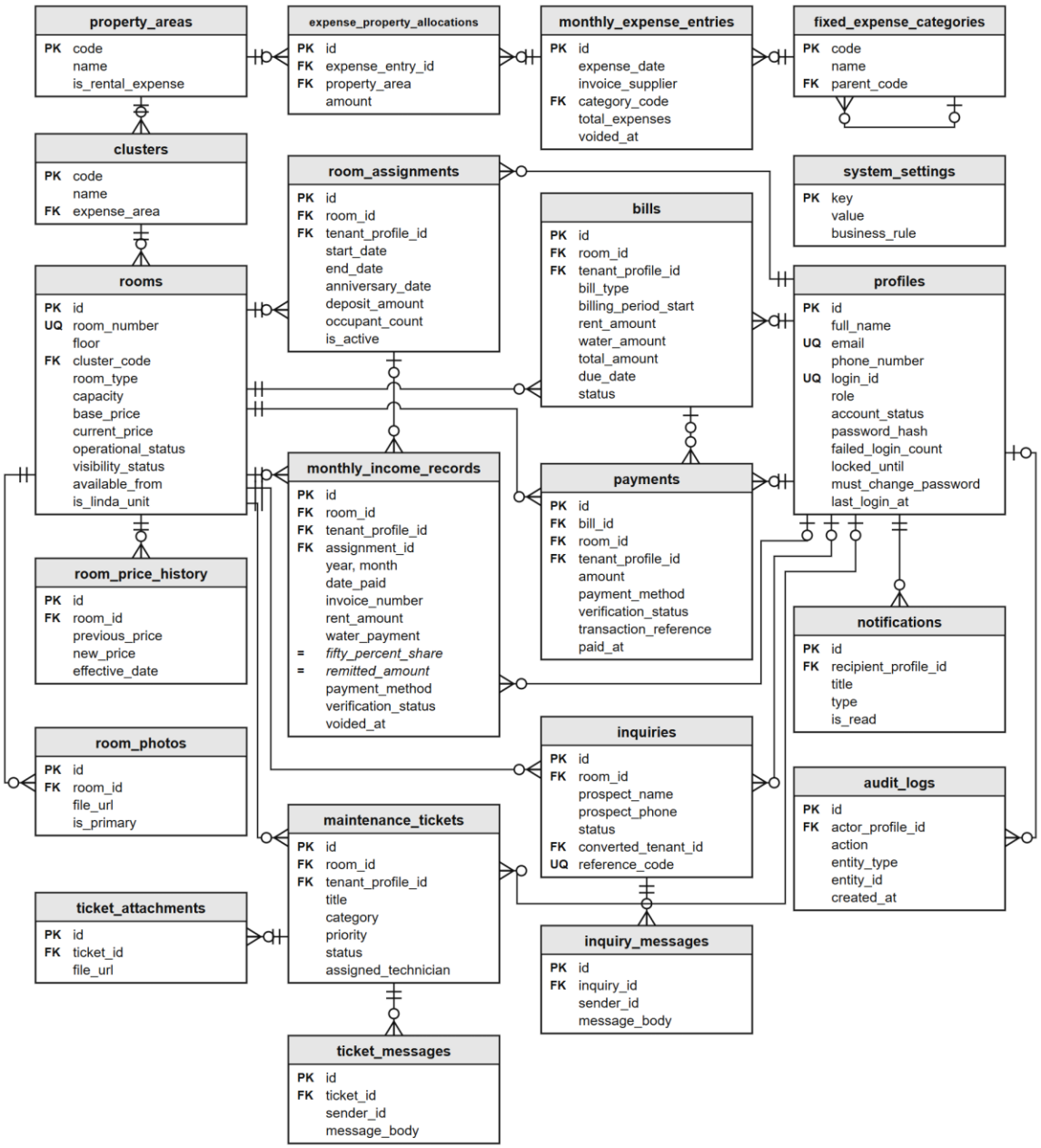



Figure 4-6, in Crow's Foot notation, is the entity-relationship diagram drawn from the database's own catalogue: 21 tables, each with a primary key, and 37 foreign keys. It draws 27 of the keys. The other ten only record who made, changed, closed, verified or sent a row, and are left out so that the figure stays readable. Its two centers are the rooms table, which holds the units, and the

profiles table, which holds the people. Every bill, payment, receipt, tenancy, repair request and inquiry belongs to one unit, and every tenancy, bill and payment to one person. The two ledgers are kept as the landlady kept them. The tables follow the third normal form, each column describing only its own table's key, with deliberate exceptions so that a record stays as it was made. A receipt keeps the name, occupants and amounts written on it, and a bill the amounts it was raised with. Receipts name their unit and tenant directly, since the 937 transferred from the workbook have no tenancy to point to, payments name theirs beside their bill, and a few totals are stored beside the figures they add up. Several rules live in the database itself, so no screen or script can bypass them. One unit cannot have two active tenancies, one invoice number cannot be used twice for the same unit and month, a gateway reference cannot be recorded twice, and the 50% Share and the remitted amount are computed columns. Table 4-5 is an extract of the data dictionary for the table the landlady reads most. The complete dictionary, every column of all 24 tables, is in Appendix N. It holds the 21 tables drawn in Figure 4-6 and three that keep backups made before corrections to the data.

*Figure 4-6. Entity-relationship diagram.*

[IMAGE FILE: images/Figure 4-6 - Entity-relationship diagram.png]




PK primary key, FK foreign key, UQ unique, = computed by the database from other columns of the row. Two bars: exactly one; bar and ring: zero or one; crow's foot and ring: zero or more. Columns ending in _by, and sender_id in the two message tables, also refer to profiles; they are left out for readability, as are three correction-backup tables.

**Table 4-5.** Data dictionary extract: monthly income records

| Column | Type | Null | Key | Description |
|---|---|---|---|---|
| id | uuid | No | PK | Identifies the receipt |
| room_id | uuid | No | FK | The unit the payment is for (rooms) |
| tenant_profile_id | uuid | Yes | FK | The tenant who paid, where one is recorded (profiles) |
| assignment_id | uuid | Yes | FK | The tenancy the payment belongs to, where known (room_assignments) |
| year, month | integer | No | | The ledger month the receipt is filed under |
| date_paid | date | No | | The day the money was received |
| contact_name | varchar(255) | No | | The name as written on the receipt |
| invoice_number | varchar(100) | Yes | | The landlady's own invoice number; not repeated for the same unit and month among receipts that are not voided |
| rent_period_start, rent_period_end | date | No | | The period the rent covers (BR-033) |
| rent_amount | numeric(10, 2) | No | | The rent received |
| occupants | integer | No | | The number of occupants that month |
| water_payment | numeric(10, 2) | No | | The water fee, ₱200 per occupant (BR-014) |
| fifty_percent_share | numeric(10, 2) | | Computed | A system-computed figure equal to half the row's Rent Amount, kept for ledger parity with the owner's historical spreadsheet (BR-035) |
| remitted_amount | numeric(10, 2) | | Computed | Rent Amount plus Water Payment (BR-038) |
| payment_method | enumeration | No | | Cash, GCash, Bank Transfer or Adyen Online |
| verification_status | enumeration | No | | Verified, Pending Verification or Rejected |

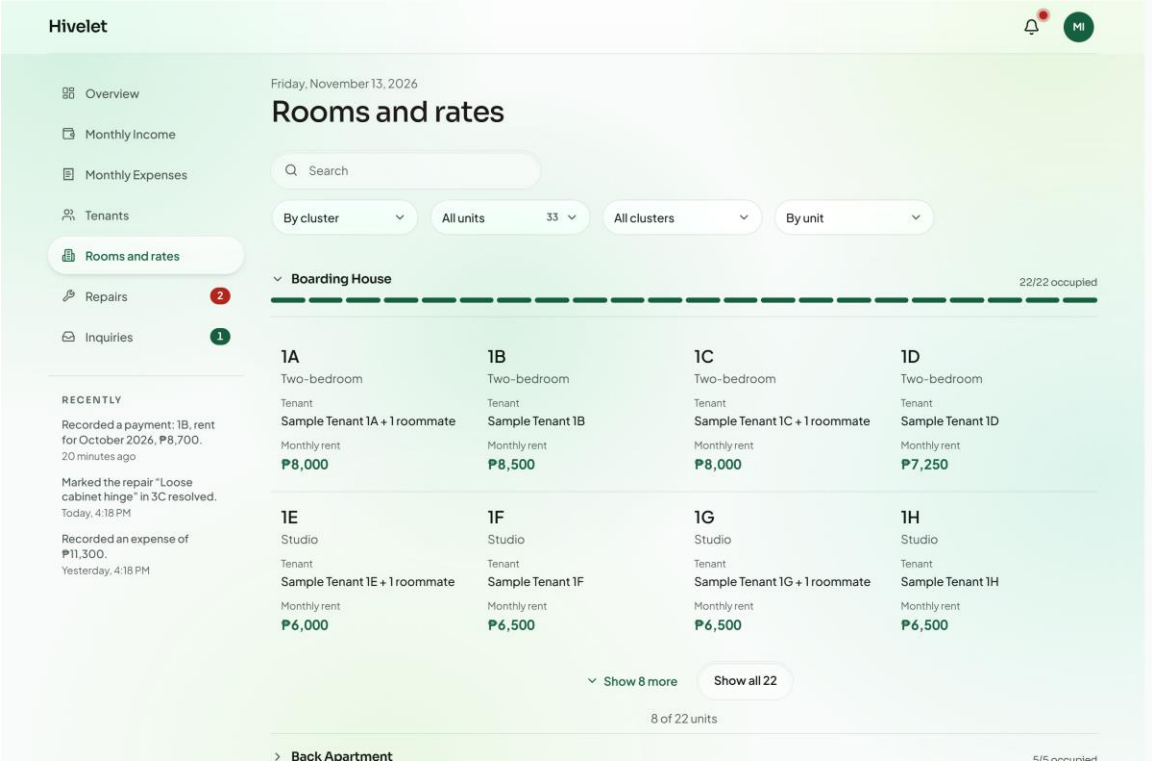
| Column | Type | Null | Key | Description |
|---|---|---|---|---|
| transaction_reference | varchar(120) | Yes | | The gateway's reference for an online payment; never recorded twice |
| is_linda_billing, linda_water_charge, linda_electricity_charge | boolean, numeric(10, 2) | Yes | | The two Linda units' charges, recorded separately (BR-040) |
| voided_at, voided_by, void_reason | timestamp, uuid, text | Yes | voided_by: FK | When, by whom and why a receipt was voided; a voided receipt is kept, never deleted |
| created_at, updated_at | timestamp | Yes | | When the row was written and last changed |

### 4.2.2 Tenant and Room Management Module

Figure 4-7 presents the Rooms and rates screen. Each unit carries its cluster, floor, current rate and status, and each tenancy the number of occupants, the move-in date and emergency contacts. When the landlady changes a rate, a database trigger records the old rate, the new rate, the date and who made the change, so no path can change a rate without leaving that record. The landlady's workbook did not record move-in dates, so for the transferred tenancies the screen says the date is not recorded rather than showing a guessed one. A tenant can also see the deposit held for their tenancy on their Overview, as one tenant asked in the survey (Table 4-19).

*Figure 4-7. Room and rate directory.*

[IMAGE FILE: images/Figure 4-7 - Room and rate directory.png]



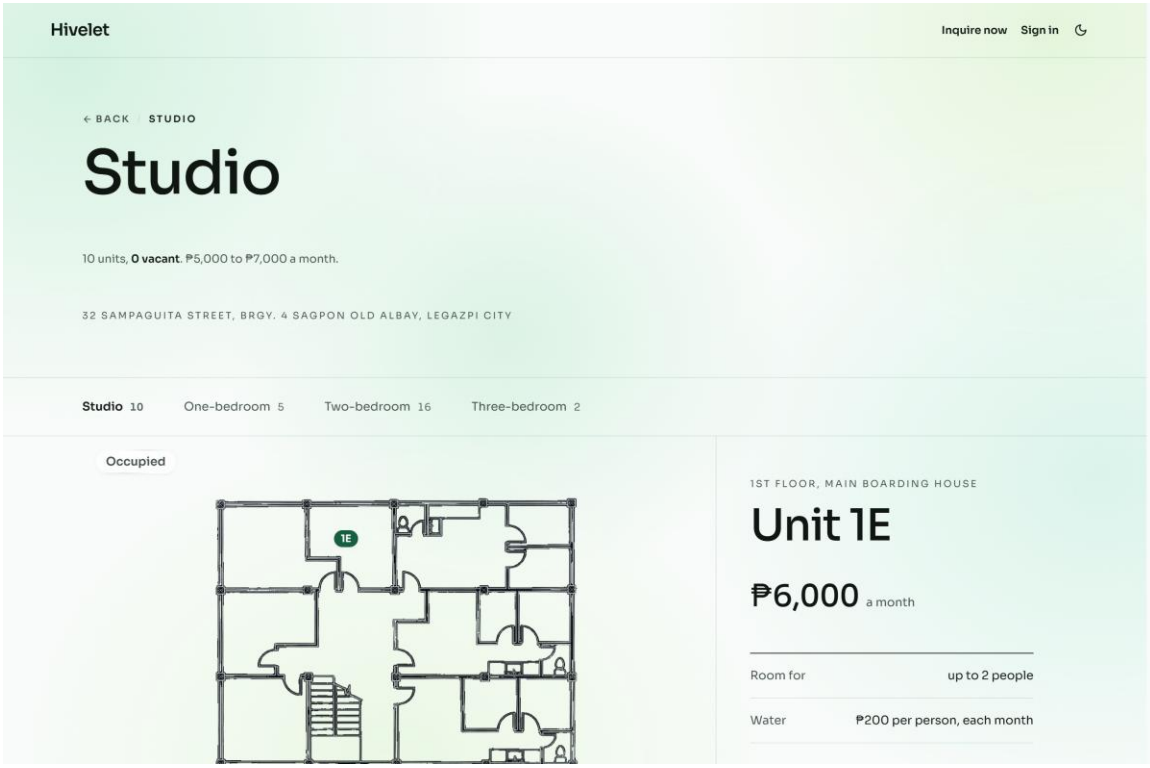

### 4.2.3 Booking and Reservation Management Module

Booking and reservation management takes the form the landlady's practice needs. She arranges every viewing herself, so nothing is booked or paid for online. A visitor sends an inquiry about a unit, the landlady answers it, may mark the unit Reserved, and moves the person in from the inquiry (Section 1.4). Figure 4-8 presents the public unit catalogue and the inquiry form. A visitor sees each unit with its rate, read live from the database, and sends an inquiry about a specific unit without an account. On sending, the visitor receives a private link and a short reference code. Either one (the code with the phone number given) opens the conversation, where the landlady's answer appears and the visitor can write back. The link's secret is stored only in hashed form. An accepted inquiry becomes

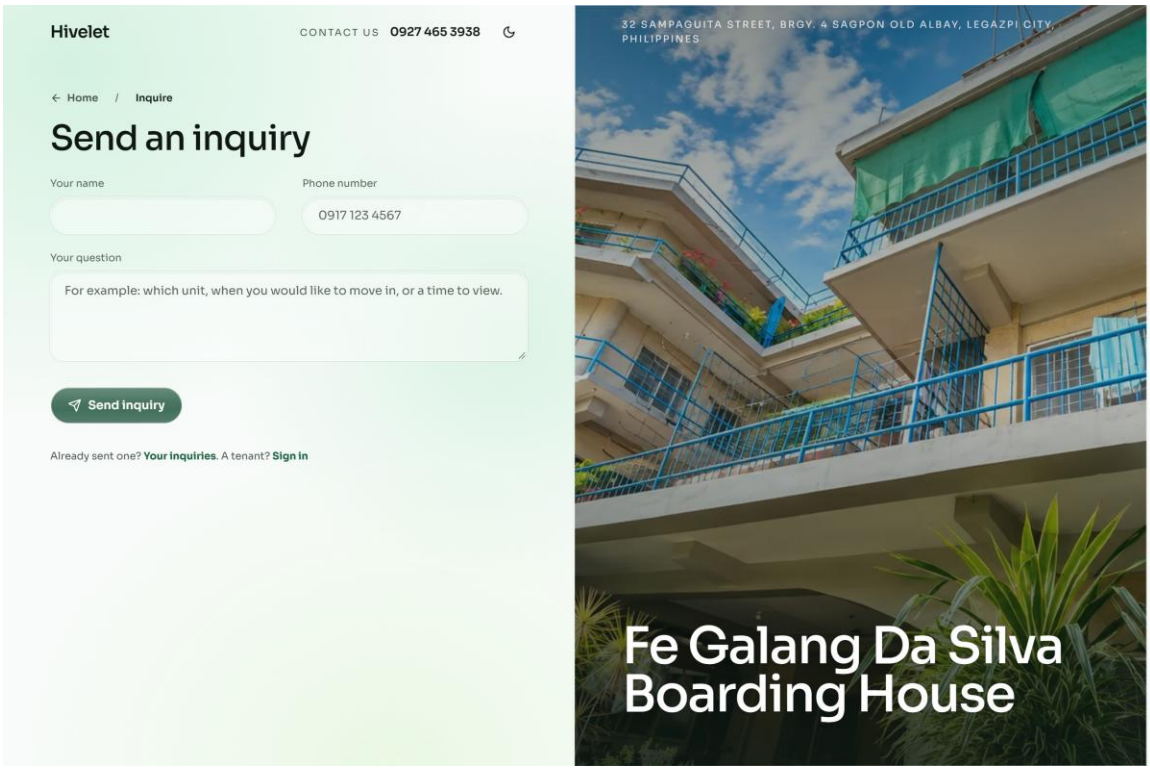
a tenancy linked to it, and a reserved unit accepts no new inquiries. The public site never shows a tenant's name, which is checked on every verification run against all 33 units.

*Figure 4-8. Public unit catalogue (top) and inquiry form (bottom).*

[IMAGE FILE: images/Figure 4-8 - Public unit catalogue (top) and inquiry form (bottom) (1).png]



[IMAGE FILE: images/Figure 4-8 - Public unit catalogue (top) and inquiry form (bottom) (2).png]



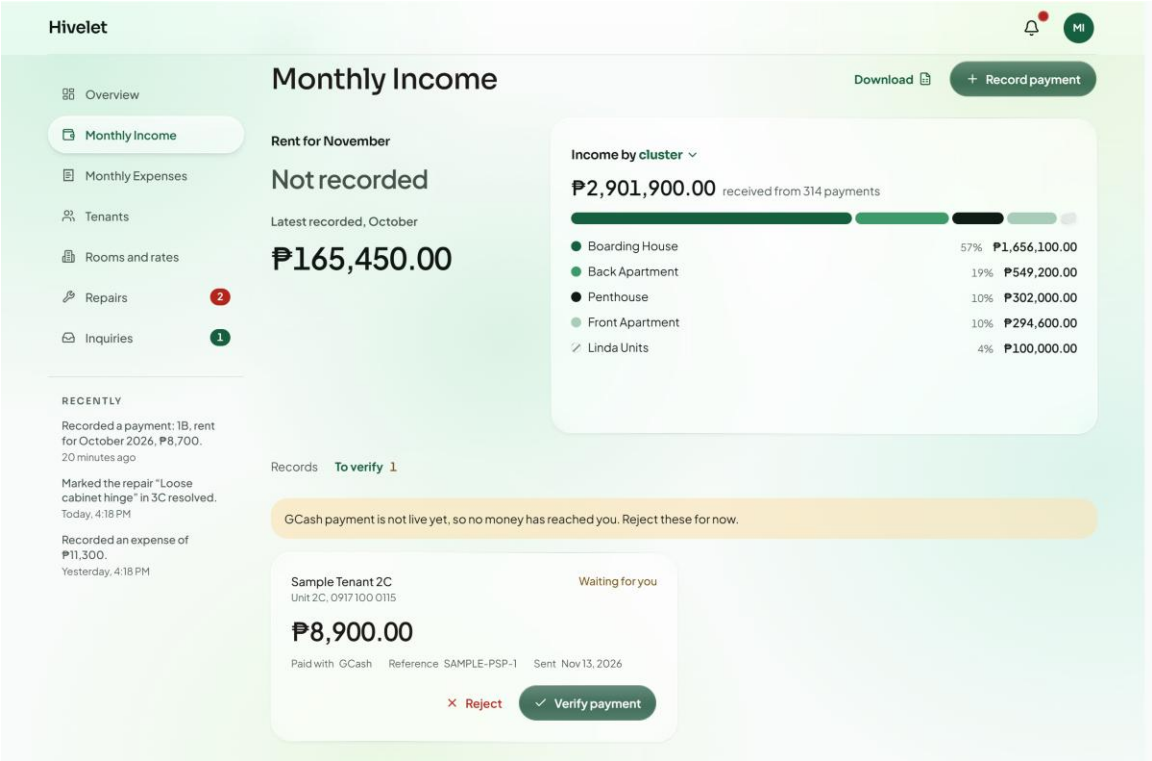

### 4.2.4 Financial Tracking and Payment Recording Module

Figure 4-9 presents Monthly Income. The landlady records a cash payment with the number of the invoice she issued, or with none, since not every payment has one. The system never makes up an invoice number, and it warns her if the unit already has a payment recorded for that period. The receipt is filed under the month the rent is for, not the day it was paid. Both the income and the expense records keep her workbook's layout and download as Excel files in it. The optional online payment goes through Adyen with GCash, in Adyen's test environment, where it takes no real money. A payment made this way enters **Pending Verification** and changes nothing on the tenant's bill until the landlady verifies it. Such payments wait on the **To verify** tab beside **Records**, and while any are waiting the page opens on that tab. Tenants see the same record from their side,

month by month. Only receipts the landlady has verified count as paid, and a month with no receipt is shown as "Not recorded", never as owed, because the records alone cannot tell an unpaid month from one not yet entered. Electricity is outside the system. Each unit has its own meter, and the tenant pays the electric company directly.

*Figure 4-9. Monthly Income, with a GCash payment waiting to be verified.*

[IMAGE FILE: images/Figure 4-9 - Monthly Income, with a GCash payment waiting to be verified.png]




These results agree with Magno et al. (2024), who found that digitizing billing and payment tracking improved data management and reduced the errors of manual processes, and with Encarnacion et al. (2025), whose system let landlords track payments and see balances. They add one observation: the consolidation also exposed errors already in the landlady's records (Section 4.1.1).

Every figure the landlady reads is computed from the rules in Table 4-6 rather than typed, so the same inputs always give the same amount.

**Table 4-6.** Business rules for computed figures, with a worked example

| Rule | What the system computes | Worked example |
|---|---|---|
| BR-014 Water fee | Water = number of occupants × ₱200 | A unit with two occupants: 2 × ₱200 = ₱400 |
| BR-038 Remitted amount | Remitted = Rent Amount + Water Payment | ₱8,500 + ₱400 = ₱8,900 |
| BR-035 50% Share | A system-computed figure equal to half that row's Rent Amount, kept for ledger parity with the owner's historical spreadsheet | ₱8,500 ÷ 2 = ₱4,250 |
| BR-033 Rent period | The period a payment covers, from the tenancy's anniversary date | Anniversary on the 13th: rent for 13 August to 12 September |
| BR-010, BR-011, BR-012 Due date and overdue | Due on the date the billing cycle sets; overdue from the next day, with no grace period | Due 13 September: overdue from 14 September |
| BR-039 Deposit | Equal to the rent in effect at move-in, set once | Moved in at ₱8,500: deposit ₱8,500 |
| BR-040 Linda's units | Water at the same ₱200 per occupant, recorded separately and kept out of the remitted amount | A Linda unit with two occupants: ₱400 recorded as Linda's water; remitted = rent only |
| BR-045 Expense total | An expense entry's total = the sum of its allocations to property areas | ₱6,000 to the Boarding House + ₱2,000 to the Main House = ₱8,000 |

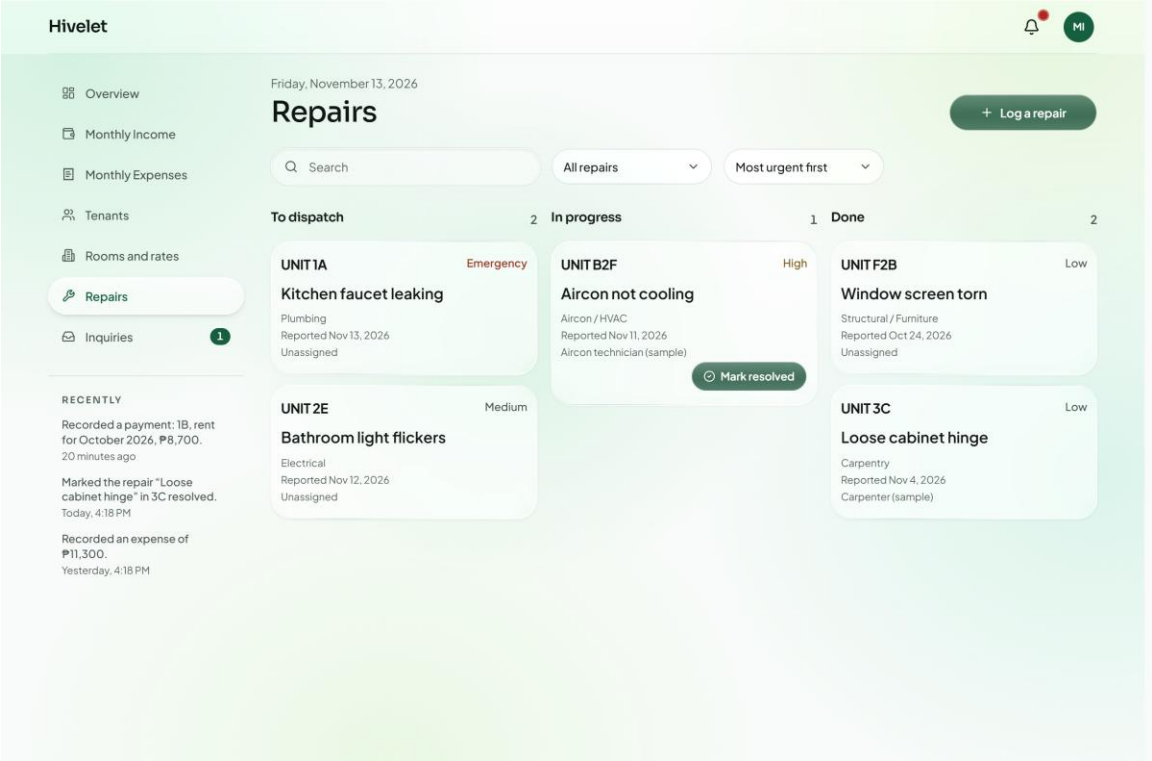
### 4.2.5 Maintenance Ticketing and Notification Module

Figure 4-10 presents the repairs board. A tenant submits a repair request with a title, description and priority, follows it through to completion, and may cancel it until someone is sent to do the work. The landlady can also log a repair she is told about in person, including one for an empty unit. Only the landlady closes a request, and the tenant is notified when it is done. In-app notifications

also tell a tenant when a payment is verified or declined, and the landlady when a payment, inquiry or comment arrives. Opening one opens the record it is about. Saputra et al. (2025) likewise brought a boarding house's complaints into the system that holds its rooms and payments, and reported less administrative work. The change also bears on the quality of the management process, which Jing and Lim (2021) found matters most to residential satisfaction. A request now carries its status from submission to closing, where before it left no record (Figure 4-1).

*Figure 4-10. Maintenance requests board.*

[IMAGE FILE: images/Figure 4-10 - Maintenance requests board.png]




### 4.2.6 Role-Based Access Control

The system has the three kinds of user designed in Section 3.2.2: the public, tenants and the administrator. It also keeps a record of each visitor who sends an

inquiry, a prospective tenant, who holds the public's rights. Every role's rights come from one permission matrix. Each of the 60 administrator and tenant routes is checked against it on every verification run. Table 4-7 sets out which role may use which function.

**Table 4-7.** Role and privilege matrix

| Function | Visitor | Tenant | Administrator |
|---|---|---|---|
| View the public site, units and rates | Yes | Yes | Yes |
| Send an inquiry and read the reply (private link or reference code) | Yes | No | No |
| View and update their own details and password | No | Own | Own |
| View a unit, its bills, payments and receipts | No | Own | All |
| Pay a bill online with GCash | No | Own | No |
| Send, follow and cancel a repair request | No | Own | All, and dispatch or close |
| Receive notifications | No | Own | Own |
| Manage units and rates | No | No | Yes |
| Manage tenants: move in, move out, reset a password | No | No | Yes |
| Record, verify, correct and void payments | No | No | Yes |
| Income and expense records, overview figures and workbook downloads | No | No | Yes |
| Answer, close and delete inquiries | No | No | Yes |

Passwords are stored as bcrypt hashes with a per-password salt. A new one needs ten characters, a letter and a number, and is refused if it appears in a public list of breached passwords. An account locks after five failed sign-ins, a new account must change its password at first sign-in, and every administrator action

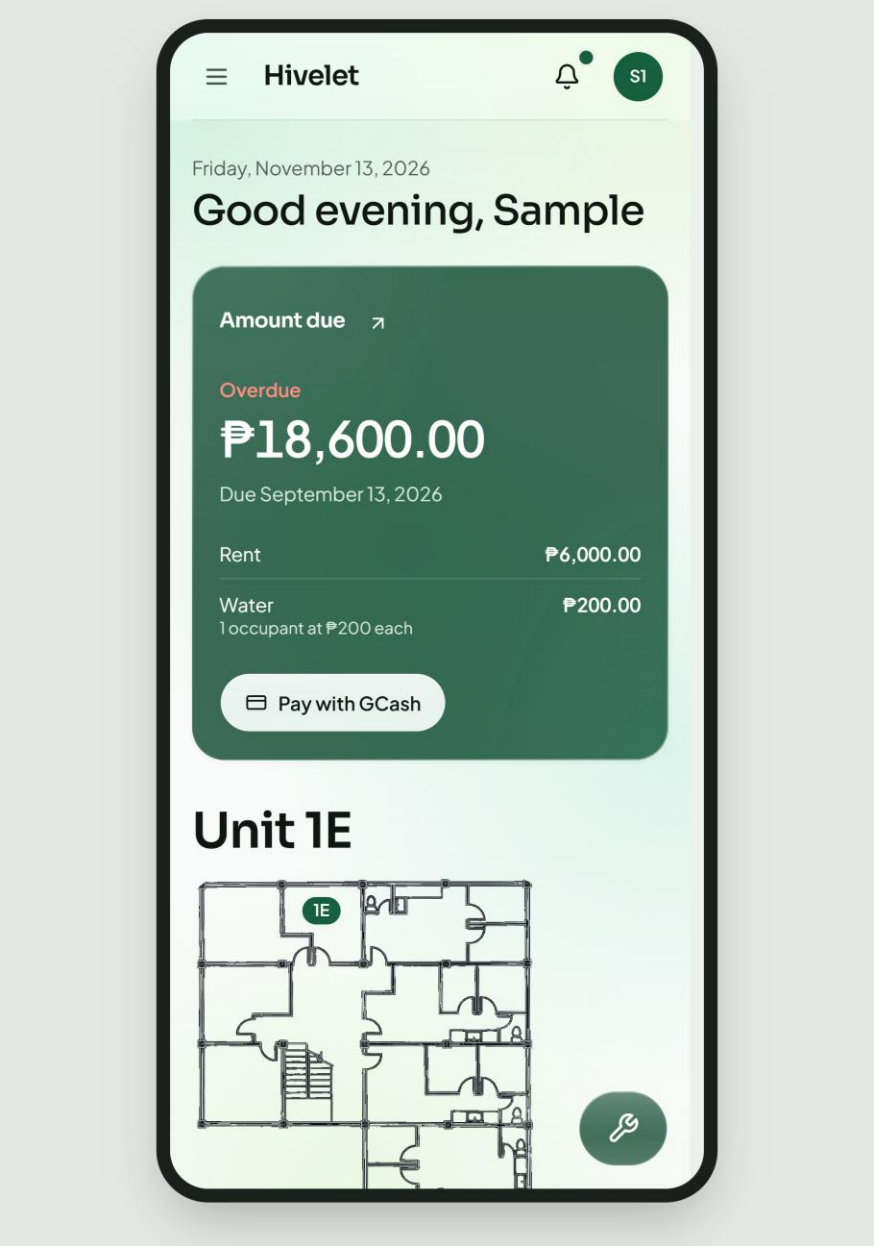
is written to an audit trail the application cannot edit, delete or empty. The trail is kept in the database and is read there. A screen that showed it was removed on the adviser's advice that it is a developer's record, and each person's side menu lists their own last three actions. The testing of these protections is reported in Section 4.4.4.

### 4.2.7 Progressive Web Application Implementation

Figure 4-11 presents the tenant portal on a phone. The client is a Progressive Web Application with a service worker and a manifest, so it installs on a phone or computer without an app store. It is installed from the browser (**Add to Home Screen** in Safari, **Install app** or **Add to Home screen** in Chrome) and opens in its own window like an application. The application files it keeps on the device come to about 1.2 MB. It keeps a read-only copy of the figures, names and notifications the signed-in person last loaded on that person's device, so the installed application can show them without a connection, under the notice "No connection · Transactions unavailable". Paying and recording still need the connection, and the copy is removed at sign-out, which stays within the delimitation in Section 1.4. Screens adapt to the device, and a table on a computer becomes a list on a phone, one row to a record. As Biørn-Hansen et al. (2017) describe, one code base served every device: the landlady's laptop, an Android phone in Chrome and an iPhone in Safari (Section 4.3.7).

*Figure 4-11. The tenant portal on a mobile phone.*

[IMAGE FILE: images/Figure 4-11 - The tenant portal on a mobile phone.png]




## 4.3 Pilot Testing of the Developed System

*Answers Objective 3: functionality, responsiveness and operational performance.*

The three qualities Objective 3 names are characteristics of ISO/IEC 25010:2011: functionality is **Functional Suitability**, responsiveness the time behavior of **Performance Efficiency**, and operational performance its capacity and **Reliability**. A test written for one quality often measures others, so each test was assigned in advance to the sub-characteristics it measures (Chapter 3, Table 3-6). Appendix M, Table M-23, lists them by test activity. The measures are brought together by characteristic in Section 4.4.

### 4.3.1 Automated Verification

Twenty automated check suites were written during development, and a twenty-first was added later (Section 4.3.6). They run against the live system and perform no writes, which is what makes them safe to run against the boarding house's real records. The evening before the first test with real users, all 20 passed. Table 4-8 shows the principal results.

**Table 4-8.** Results of automated verification

| Suite | What it checks | Result |
|---|---|---|
| Endpoint and access control | Every endpoint the client calls, the separation of roles, and the rejection of bad input | 78 of 78 passed |
| Ledger integrity | All 937 income rows against the business rules; unit, tenant and occupancy agreement | Passed; 5 receipt anomalies reported for the landlady |
| Report reconciliation | The exported Excel reports against the database, month by month | 490 of 490 passed |
| Payment gateway | Signature checking on gateway messages and how each kind of message is recorded | 73 of 73 passed |
| Record relations | That separate records agree with each other (payments with bills, tickets with units) | 30 of 30 passed |
| Billing arithmetic | Water, billing period and how a payment is split across bills | Passed |
| Data presentation | That no screen shows stored, sample or empty data as a live figure | 6 of 6 passed |
| Interface integrity | That every screen component exists, every input has a label, and every source file is used | Passed |
| Document consistency | That the requirements matrix and business rule register agree with themselves | Passed (44 requirements, 49 rules) |

The five receipt anomalies in Table 4-8 are those of Section 4.1.1. They came from the landlady's records and are reported on every run rather than corrected, because only her invoice book can say what each should read. More important, the most serious defects found in development gave no error on screen. A cash payment could be confirmed while nothing was written to the ledger, and a partial payment could be recorded while the remaining debt was lost. Each was fixed and now has a check that fails if it returns. This is the case for automated verification in a system that holds real money: the most dangerous defects are the silent ones.

### 4.3.2 Use in the Live Environment

The system has run at a public address with the boarding house's real records throughout the pilot. Its first real bill was computed exactly as the rules require, and a test receipt voided after it was recorded returned the ledger to its 937 rows. When the landlady found the system hard to use on her phone, the causes were measured and fixed the same day. A GCash test payment passed end to end, from Adyen's authorization to the landlady's verification queue, once Adyen corrected an account setting on its side. Every record the pilot tests created was then removed by one reviewed migration after a backup, leaving the 937 income rows transferred from her workbook untouched.

### 4.3.3 Screen-versus-Database Audit

The suites prove each part correct on its own, not that a screen shows a person what the database holds for them. A tenant's payment history once read "Nothing recorded" although the tenant had seven receipts that year, while every suite passed. The researchers therefore compared every figure on every administrator and tenant screen with a read-only query of the live database for the same records. Table 4-9 shows the result.

**Table 4-9.** Results of the screen-versus-database audit

| Screen | What was compared | Result |
|---|---|---|
| Administrator overview | Collections for each month of 2026, the year's total, occupancy, rent per cluster, expected monthly income, operating and personal costs | Matched to the peso |
| Monthly Income | Totals for rent and water; collections by cluster | Matched after one defect was fixed |
| Monthly Expenses | Total spent, split by kind and by area | Matched; one layout defect fixed |
| Rooms and rates | Rent and number of occupants for each unit | Matched |
| Tenants | Number of tenants per cluster; move-in dates | Counts matched; one defect fixed |
| Activity (audit trail), a screen later removed (Section 4.2.6) | Each recent entry against the recorded action | One defect fixed |
| Repairs, inquiries | Number of open items | Matched |
| Tenant payments | A tenant's receipts for 2026 | Two defects fixed |
| Tenant overview, repairs and details | Unit, rate, occupants, monthly charge (rent plus ₱200 per occupant), the period the receipts reach, amount due, requests, contact details | Matched |

The audit found six defects no automated check had caught, all fixed the same day. They were a tenant's payment history that read only online payments, rental and personal expenses in one column, a placeholder move-in date shown for every tenant, an opened checkout listed as a recorded payment, a voided receipt in a tenant's receipt list, and the Boarding House's remitted amount halved on the income screen. The lesson is the reverse side of Section 4.3.1: **passing checks show that what was tested is correct, not that everything is.** It qualifies Magno et al. (2024), who found that digitizing transaction tracking improves data

accuracy. Here every record was already digital and in one database, yet six screens disagreed with it, so digitizing alone did not make the records agree.

### 4.3.4 Functional Walkthrough Testing

A 26-step walkthrough, 30 rows counting its sub-steps, exercised every function that writes data once, on the one unit then unoccupied, so that no real record was touched. The landlady performed it herself on her laptop, signed in to her own account, with one of the researchers reading each step. The full results are in Appendix M, Table M-1. Of the 30 rows, 24 passed the first time, 1 failed and 5 were not performed as written. Every function a tenant uses passed, as did onboarding, the duplicate guards, voiding, inquiry replies, repairs, the Excel export and moving a tenant out. The one failure was step 23b. With the connection cut, the Overview showed ₱0 where the design requires "—", so that a missing figure is never read as a zero balance. Once corrected, it passed when re-tested on the landlady's laptop (Table 4-20). A rate change and its reversal were skipped, and an expense split across two areas was entered as two expenses, so splitting, editing and deleting an expense were not exercised by the landlady.

### 4.3.5 Responsiveness and Operational Performance

Response times were measured on the live system, on the laptop and phone named in Section 3.1.2, from the tap until the figures the user came for were on screen (Table 4-10).

**Table 4-10.** Page load and response times (seconds)

| Screen | Laptop (Acer Nitro V15, Chrome 154, Wi-Fi) | Phone (iPhone 15, Safari, mobile data) |
|---|---|---|
| Public home page, first load | 1.85 | 2.42 |
| Public home page, second load | 0.62 | 0.95 |
| Sign-in to dashboard | 2.10 | 2.88 |
| Income ledger, one full year | 1.45 | not measured |
| Excel export of one year | 2.30 | not measured |
| Tenant portal, first load | 2.15 | 3.05 |
| Tenant payments page | 1.20 | 1.78 |
| Sending a repair request | 1.65 | 2.10 |

Every screen was usable within about three seconds on the phone and a little over two on the laptop, the slowest being the tenant portal's first load (3.05 seconds on the phone). Google's PageSpeed Insights, emulating a mid-range phone on slow 4G, scored the public page 85 to 88, with its largest element at 3.4 to 3.6 seconds, and 100 for accessibility. On the administrator's Overview, Lighthouse found the page moving as its figures arrived (a layout shift of 0.22) and 0.7 seconds of script work on the phone (Chapter 5, recommendation 5).

After the redesign of the second iteration, the screens were timed again on another laptop and network (Appendix M, Table M-31). Every screen timed was ready sooner than before except sign-in to the Overview, at 2.63 seconds against 2.10. The two days used different laptops, networks and timing methods, so the comparison shows the direction of the change and not an exact difference. The administrator's Overview now moves by 0.08 as its figures arrive, down from 0.22 and below the 0.1 counted as good. PageSpeed Insights scored the public page

85 on a phone, as before, with its largest element at 4.2 to 4.4 seconds, and 99 on a desktop.

### 4.3.6 Offline Use, Installation, Weak Connections and Simultaneous Users

The live system was measured with no connection, on a weak one, once installed, and with many requests at once, using an automated browser in phone emulation with every save blocked so that nothing was written to the boarding house's records. Without a connection, the installed application opened on all ten addresses tried, in about 1.5 seconds. It showed no personal or financial data until sign-in and said at once what could not be done offline. On a slow connection a returning visitor saw the units after 0.3 seconds instead of 4.6. In a load test a script played 12 simultaneous users on the public pages and 6 on the administrator's screens, each pausing between requests as a person reading does. None of the 1,085 and 285 requests failed, and nineteen in twenty were answered in under a second, under loads several times the property's own. These were simulated users, not people (the landlady and tenants together are reported in Section 4.3.7). Every result is in Appendix M, Tables M-24 and M-25.

The tests found one weakness: on a weak signal a page could wait for minutes with no message. A limit was added the same day. A page now stops after 25 seconds and says it could not reach the server, and a save stops after 45 seconds and says it **cannot tell whether the save arrived, so the reader should check before sending it again**. An accessibility scan of the public pages (axe-core 4, WCAG 2 A and AA) at phone and computer widths found no violation, nor did the same scan of the administrator's eight screens.

**Simultaneous saves.** Reading at the same time tests capacity. Saving at the same time tests correctness, because two requests that arrive together can each pass a check before either has been saved. This was tested by sending the same request several times in the same instant. That reproduces a double click, a retry after a timeout, or two people pressing Save together, with closer timing than people can produce by hand. Before the fix, the same receipt sent twice or five times at once was saved every time, and two GCash checkouts could open on one bill. After it, one was saved and every other refused, and a checkout abandoned for more than 15 minutes no longer held the bill (Appendix M, Table M-26).

Neither fault showed an error, and neither could be fixed in the program alone, since a check followed by a save can always be outrun. Each fix is a rule in the database, which refuses the second save whatever order the requests arrive in, so ten requests arriving together meet the same refusal as two. A twenty-first suite now repeats these tests on every run against the system's own request handlers. It passes 15 checks, and each of four deliberate faults made it fail. Two limits remain. First, the rule covers receipts with an invoice number. Two identical receipts without one, or two acknowledgement receipts, arriving at the same instant would both be saved, which the payment form's confirmation step makes unlikely without ruling out. Second, two people changing the same record in different ways at once was not tried.

### 4.3.7 User Acceptance Testing with the Landlady and Tenants

The landlady and three tenants used the live system for their own tasks, each on their own device and account, after giving written consent (Section 3.4),

by the procedure in Section 3.3. Where an observer's sheet and the system's activity record disagreed, the activity record was used. Table 4-11 gives each task's result. The landlady accepted the system and signed the acceptance form (Appendix O).

**Table 4-11.** Tenant task results

| Task | Tenants attempting | Completed without help | Completed with help | Not completed | Median time (s) | Mean wrong turns | Completion without help |
|---|---|---|---|---|---|---|---|
| First sign-in and choosing a password (T-01, T-02) | 3 | 4 | 2 | 0 | 120 | 0.5 | 67% |
| Finding their unit, rent and balance (T-03, T-04) | 2 | 3 | 1 | 0 | 90 | 0.5 | 75% |
| Finding their payments (T-05, T-06) | 2 | 4 | 0 | 0 | 60 | 0.0 | 100% |
| Sending a repair request with a photo (T-07 to T-09) | 3 | 7 | 0 | 0 | 90 | 0.0 | 100% |
| Following up a request (T-10, T-11) | 2 | 3 | 0 | 0 | 120 | 0.0 | 100% |
| Checking their details and notifications (T-12, T-13) | 3 | 5 | 0 | 0 | 60 | 0.2 | 100% |
| Opening the GCash payment (T-14) | 2 | 2 | 0 | 0 | 120 | 0.0 | 100% |
| Staying out of the administrator's pages; signing out (T-15, T-16) | 2 | 3 | 0 | 0 | 60 | 0.0 | 100% |
| **All tasks** | 3 | 31 | 3 | 0 | 60 | 0.2 | 91% |

Every task the tenants attempted was completed, 31 of 34 attempts without help (91 percent). Help was needed only at the start. Two of the three tenants were

helped through their first sign-in, and one to find their unit and rent. Everything else, from finding payments to sending a repair request and opening the GCash payment, was done alone in one to two minutes. The repair requests they sent reached the landlady at once, and she acted on them the same evening. Mia et al. (2024) found that a lack of digital skills was not necessarily a barrier for small Philippine enterprises, which relied on tools they already knew. The tenants' results are consistent with this at the scale of one boarding house: the help they needed was confined to the first two things a new user does.

The tests that needed several tenants saving at once were not performed, because only two attended together. In the session in which the landlady and two tenants used the system at the same time, every screen loaded, each tenant saw only their own unit, the landlady's full year of Monthly Income opened in about three seconds, and no session affected another (Appendix M, Table M-22). On two phones the offline behavior matched Appendix M, Table M-24, except that the Android phone showed the "cannot tell whether it was saved" message after 60 to 80 seconds instead of 45. Once corrected, it was on time when re-tested on that phone (Table 4-20).

**Second round.** On the redesigned system of the second iteration, ten tenants and four prospective tenants did the tasks set for them, the tenants the same tasks as in Table 4-11 (Appendix M, Table M-30). Every task was completed. The tenants did 151 of their 160 attempts without help (94 percent), and the prospective tenants 31 of 32 (97 percent). As in the first round, help was needed mostly at the first sign-in, where five of twenty attempts needed help, most often

after a character on the starting-password slip was misread, and in finding the browser's address bar when asked to type in an administrator's page. Time on task was not recorded reliably in this round, so no times are reported for it.

Table 4-12 brings the testing together by the test levels of ISO/IEC/IEEE 29119 (Section 3.2.4).

**Table 4-12.** Summary of test execution by level

| Level | Activity (source) | Executed | Passed | Failed | Not performed or not applicable | Pass rate |
|---|---|---|---|---|---|---|
| Unit and integration | Automated check suites (Table 4-8) | 20 | 20 | 0 | 0 | 100% |
| System | Functional walkthrough by the landlady (Appendix M, Table M-1) | 25 | 24 | 1 | 5 | 96% |
| Acceptance | Tenant tasks (Table 4-11) | 34 | 34 | 0 | 0 | 100% (91% without help) |
| Acceptance | Simultaneous use by the landlady and tenants (Appendix M, Table M-22) | 4 | 4 | 0 | 3 | 100% |
| **All levels** | | **83** | **82** | **1** | **8** | **98.8%** |

Every level passed at least 96 percent of the tests executed, against the course standard of 90 percent. The one failure, at the system level, was not critical, and no critical defect was found in any activity. Table 4-20, in Section 4.4.6, lists the defects by activity, severity and disposition.

Table 4-12 counts the activities that correspond to the three test levels. The other cases of Appendix L, on the public site, offline use and installation, security,

browsers and timings, are reported as measures in Sections 4.3.5, 4.3.6 and 4.4.4. One of them failed, the late message on the Android phone, and it was on time when re-tested.

## 4.4 Evaluation of the System Based on ISO/IEC 25010

*Answers Objective 4.*

The evaluation followed the five steps of ISO/IEC 25040 (Table 4-13), on the quality model of ISO/IEC 25010:2011. That edition was used because the approved proposal (Section 3.3) named its eight characteristics as the evaluation criteria, and the instrument was built on them. Each characteristic was judged on **measured quality**, the results of Section 4.3 expressed as measures of its sub-characteristics in the manner of ISO/IEC 25023, and on **rated quality**, a survey in which each item belongs to one sub-characteristic (Chapter 3, Table 3-6). Each respondent group rated only what it can judge. Tenants and prospective tenants did not rate Security, and only the technical evaluators, the group with the background to judge it, rated Maintainability.

**Table 4-13.** The evaluation process, following ISO/IEC 25040

| Step | What it required | How it was carried out |
|---|---|---|
| 1. Establish the evaluation requirements | The purpose, the product, the quality model and who judges it | Objective 4; the live system at its public address, and for four of the first round's technical evaluators a demonstration copy of it with sample data; ISO/IEC 25010:2011, eight characteristics; the landlady, tenants, prospective tenants and technical evaluators (Section 3.3) |
| 2. Specify the evaluation | The measures for each sub-characteristic, and the criteria for judging them | Measures from the tests of Section 4.3 (Appendix M, Table M-23); survey items assigned to sub-characteristics (Chapter 3, Table 3-6); criteria: Chapter 3's Table 3-5 for every mean, and for tests the course standard of at least 90 percent passed with no critical defect open |
| 3. Design the evaluation | The activities, their order and schedule | The measurements, then the walkthrough and acceptance test, the technical evaluators' reviews, and the survey after each group's use of the system; a second round, of users with their tasks and of technical evaluators, with the same survey, after the redesign |
| 4. Execute the evaluation | Take the measures and apply the criteria | Section 4.3 (measured) and the survey (rated; Appendix M, Tables M-2 to M-17 for the redesigned system, and Tables M-28 and M-29 for the first round) |
| 5. Conclude the evaluation | Review the results, report them, and act on them | Tables 4-17 and 4-18, and the changes made in response (Table 4-19) |

ISO/IEC 25010 was revised in 2023, renaming Usability as *Interaction Capability* and Portability as *Flexibility* and adding *Safety*. The survey was built and answered on the 2011 characteristics, so its results are reported on them. Appendix M, Table M-21, shows where each falls in the revised model.

### 4.4.1 Respondents

Table 4-14 shows the respondents who rated the redesigned system of the second iteration, whose ratings are reported in this section, and Table 4-15 how many forms were given out and returned.

**Table 4-14.** Distribution of respondents

| Group | Number | Percent | Basis for inclusion |
|---|---|---|---|
| Landlady | 1 | 5.88% | The system's only administrator |
| Tenants | 10 | 58.82% | Users of the tenant portal on their own accounts |
| Technical evaluators | 3 | 17.65% | Able to judge the technical characteristics |
| Prospective tenants | 3 | 17.65% | Users of the public website |
| **Total** | **17** | 100% | |

**Table 4-15.** Instrument administration and response rate

| Group | Distributed | Retrieved | Valid | Response rate |
|---|---|---|---|---|
| Landlady | Not recorded | 1 | 1 | — |
| Tenants | Not recorded | 10 | 10 | — |
| Technical evaluators | 3 | 3 | 3 | 100% |
| Prospective tenants | Not recorded | 5 | 3 | — |
| **Total** | | **19** | **17** | |

The number of forms given to the users was not recorded, so their response rate cannot be given. Two of the five forms returned by prospective tenants were answered in another group's section, the landlady's and the tenants', and are left out by the rule of Section 3.3 (Appendix M, Table M-28). Every other form returned

was complete and valid. One further answer, which the researchers made while testing the form, is not a respondent's and is not counted.

The landlady, ten tenants and three prospective tenants answered after using the redesigned system of the second iteration, and three technical evaluators, IT professionals and developers, after reviewing the live system and its source code for about 30 minutes each, each after giving consent as described in Section 3.4. No member of the research team was among them, and the landlady's relation to one is stated in Section 3.4. The landlady is one respondent, so her group's mean is one person's rating, and three technical evaluators are a small group. A member of the research team was with those three throughout (Section 3.4), which may have favored higher ratings. The results describe how these respondents rated the system, not a sample from which a wider population can be inferred. The first round, on the system of the first iteration, had ten respondents: the landlady, three tenants, a prospective tenant and five technical evaluators (Appendix M, Tables M-28 and M-29).

### 4.4.2 Interpretation of Scores

Each item was rated on the five-point scale in Table 3-4, and means are read using Chapter 3's Table 3-5. The composite of each characteristic is the mean of its group means, so that the one landlady is not outweighed by ten tenants on functions only she uses. The users' composites, from the landlady, the tenants and the prospective tenants, are reported beside the technical evaluators', whose questionnaire asks about the system from a developer's side. Each overall mean is the mean of the characteristics its respondents rated: seven for the users, since

only the technical evaluators rated Maintainability, and eight for the technical evaluators. Averaging every user's response together instead gives 4.80 rather than 4.84, with the same interpretation, Very High Quality, and the same highest and lowest characteristics. Table 4-16 works one weighted mean in full, WM = Σ(f × w) / N.

**Table 4-16.** Worked computation of a weighted mean

| Rating (w) | Meaning | Frequency (f) | f × w |
|---|---|---|---|
| 5 | Strongly Agree | 8 | 40 |
| 4 | Agree | 2 | 8 |
| 3 | Neutral | 0 | 0 |
| 2 | Disagree | 0 | 0 |
| 1 | Strongly Disagree | 0 | 0 |
| **Total** | | **N = 10** | **Σ(f × w) = 48** |

Indicator: "I can see my own unit details, my bill, and what I still owe." (Functional Suitability, rated by tenants). WM = 48 / 10 = **4.80**, which Table 3-5 reads as **Very High Quality**.

### 4.4.3 Results by Characteristic

Table 4-17 summarizes the result for each characteristic. The rating of every item by every group is in Appendix M (Tables M-2 to M-17, ratings and measured evidence for each characteristic).

**Table 4-17.** Summary of evaluation results

| Characteristic | Users' mean | Interpretation | Technical evaluators' mean | Interpretation |
|---|---|---|---|---|
| Functional Suitability | 4.80 | Very High Quality | 4.93 | Very High Quality |
| Performance Efficiency | 4.86 | Very High Quality | 4.89 | Very High Quality |
| Compatibility | 4.79 | Very High Quality | 5.00 | Very High Quality |
| Usability | 4.80 | Very High Quality | 4.87 | Very High Quality |
| Reliability | 4.78 | Very High Quality | 4.83 | Very High Quality |
| Security | 5.00 | Very High Quality | 4.93 | Very High Quality |
| Maintainability | not rated | | 4.73 | Very High Quality |
| Portability | 4.83 | Very High Quality | 5.00 | Very High Quality |
| **Overall** | **4.84** | **Very High Quality** | **4.90** | **Very High Quality** |

Users: the landlady, ten tenants and three prospective tenants (n = 14). Security is the landlady's rating, since tenants and prospective tenants did not rate it. Technical evaluators: n = 3.

The users rated the system **Very High Quality overall (4.84)**, and the technical evaluators **Very High Quality overall (4.90)**. Every composite of both is Very High Quality. This is the range Canlas et al. (2026) report for a Philippine dormitory system rated by the same kinds of respondents, tenants, administrators and IT professionals. In the first round, on the system before the redesign, the users had given 4.75 and the five technical evaluators 4.34, with Usability and Reliability at 4.20, the top of High Quality (Appendix M, Table M-29). Two of the users' composites were lower than in that round, Functional Suitability (4.94 before) and Portability (4.96 before), from a group nearly three times as large. The

differences between the characteristics are small: eleven of the fourteen users gave every item the same rating, nine of them 5 and two of them 4. Each rating is therefore also read beside the measured evidence for its characteristic (Section 4.4.5).

**Performance Efficiency** was the users' highest of the characteristics all three user groups rated (4.86, technical evaluators 4.89). It matches the measurements: no failed request, and after the redesign every screen timed again was ready sooner than before except sign-in, though screens still took two to three seconds on a phone (Section 4.3.5). **Portability** followed (4.83 and 5.00), as one code base installed on an Android phone and an iPhone without an app store. **Compatibility** (4.79 and 5.00) agrees with the browser matrix, on which every combination tried worked. **Security** (5.00 and 4.93) is discussed in Section 4.4.4.

**Usability** (4.80 and 4.87) was higher for both groups after the redesign. The landlady gave every item 5, the tenants 4.74 and the prospective tenants 4.67. Before the redesign the users had given it 4.49, and the landlady had given 3 to telling what each screen is for and to learning the system without help (Appendix M, Table M-29). The first round's technical evaluators gave it 4.20. The first of them gave 3 to all five items before the fourteen changes made for that group, eleven of them about usability, and the four who reviewed afterwards gave 4.80, 4.60, 3.60 and 5.00. The three who reviewed the redesigned system gave 4.80, 4.80 and 5.00. Here the rating and the measurement agree. At least nine tasks in ten were done without help in each round, and 76 accessibility scans of 19 screens found no violation once one defect on three screens was corrected.

**Functional Suitability** (4.80 and 4.93): every feature the objective names is present, and 96 to 100 percent of the executed checks of correctness gave the right result. The landlady gave every item

5. The users' lowest items, at 4.67, were the prospective tenants' three, and the technical evaluators'

   lowest was payment recording, also at 4.67.

**Reliability** (4.78 and 4.83) is the users' lowest. The tenants' lowest item was whether the information shown is correct and up to date (4.50), to which one tenant gave 2. The probable cause is that not every receipt in the landlady's receipt book has been entered yet, so a month a tenant has paid can still read "Not recorded" on that tenant's screen (Section 4.2.4 and Table 4-22). The technical evaluators gave 4.67 to keeping recorded data and to telling "no data" from "could not be loaded", and 5.00 to the other two. In the first round their lowest item, 3.80 for handling a failed request without presenting incorrect data, named the one test that failed, the ₱0 of step 23b, since resolved. The testing agrees: no critical defect was found, and the two offline defects found on real devices passed on re-test.

**Maintainability** (4.73), rated by the technical evaluators only, is Very High Quality and their lowest. Its three respondents, who read the source code as well as using the system, gave 4.40, 5.00 and 4.80, and its lowest items, at 4.33, were changing configurable values without changing code and automated checks that would catch a regression. In the first round five evaluators had given it 4.32, each after reviewing the running system without inspecting its source code. Here the measurement, read from the code itself, carries more weight than the rating, and

it points the same way. The system can be checked as a whole on every change, but three database calls in four are written inside the request handlers, so the rules for one record are spread across the handlers that touch it. Both are taken up in Chapter 5, recommendation 6.

### 4.4.4 Security

The landlady gave every Security item 5. The three technical evaluators, whose mean was 4.93, gave 5 to every item but the audit trail, which they gave 4.67. They gave 5 to authentication and session handling although the two low-risk session weaknesses the code review accepted remain, so on that item the rating is higher than the measurement. In the first round the five technical evaluators' mean was 4.36. They gave 4.20 to authentication and session handling, which agrees with those weaknesses, and 4.00 to the handling of credentials, for which the measured evidence gives no cause and the forms gave no reason.

Three passive external scans graded the site **A+**: Qualys SSL Labs, Mozilla HTTP Observatory (all 12 tests passed once the Content Security Policy was enforced) and securityheaders.com. Twelve checks by hand passed, among them an account locking after five wrong passwords, no personal data behind the Back button after signing out, and another tenant's repair request answering "not found". A review of all 71 request handlers found no high-severity problem, and Supabase's security advisor raised no warning. Two weaknesses remain by design. The sign-in token is kept in the browser's storage, where an injected script could read it. The enforced Content Security Policy and the absence of any raw

HTML on the screens stand against that. Sign-in also takes a password only, with no second factor even for the administrator. Both are left to Chapter 5, recommendation 3.

### 4.4.5 Measured and Rated Evidence Together

Table 4-18 sets the two kinds of evidence side by side. Of the thirty-one sub-characteristics of ISO/IEC 25010:2011, twenty-seven have a measured result and twenty-five are rated by at least one survey item. Two, user interface aesthetics and reusability, have neither, and no conclusion is drawn about them.

**Table 4-18.** Measured and rated evidence by characteristic

| Characteristic | Sub-characteristics | Measured | Rated | Measured result (Appendix M) | Users' mean (Table 4-17) | Technical evaluators' mean (Table 4-17) |
|---|---|---|---|---|---|---|
| Functional Suitability | 3 | 3 | 3 | All six features present; 96 to 100 percent of correctness checks passed | 4.80 | 4.93 |
| Performance Efficiency | 3 | 3 | 3 | No request failed under load; every screen usable within 3.05 s on a phone | 4.86 | 4.89 |
| Compatibility | 2 | 1 | 2 | 5 of 5 device and browser combinations; gateway and spreadsheet exchange correct | 4.79 | 5.00 |
| Usability | 6 | 5 | 4 | 91 and 94 percent of the tenants' tasks without help in the two rounds; 0 accessibility violations in 76 scans | 4.80 | 4.87 |
| Reliability | 4 | 3 | 4 | No critical defect; the two offline defects found on real devices, a ₱0 shown for a figure that could not load and a late message, both passed on re-test | 4.78 | 4.83 |
| Security | 5 | 5 | 3 | Every protection tried held; A+ from all three outside scanners | 5.00 | 4.93 |
| Maintainability | 5 | 4 | 4 | 20 of 20 suites passed without writing to the records; every change a numbered migration; but 154 of 204 database calls inside the request handlers | not rated | 4.73 |
| Portability | 3 | 3 | 2 | No screen wider than its window at four widths; installed on two phones without an app store | 4.83 | 5.00 |
| **All** | **31** | **27** | **25** | | **4.84** | **4.90** |

The measured results are those of the first iteration, except the tenants' tasks of the second round. On the redesigned system, 109 of the 114 test cases were run again and none failed (Section 4.2.1).

For six of the eight characteristics the two kinds of evidence agree. For Security and Maintainability the measurements are less favorable than the ratings. Two session weaknesses remain although the technical evaluators gave session handling 5 (Section 4.4.4), and three database calls in four sit inside the request handlers although they gave the organization of the code 5 (Section 4.4.3; Appendix M, Tables M-14 and M-15). Taken together, the ratings and the measurements indicate that the system is acceptable for use at the boarding house. Both groups rated every characteristic Very High Quality, no critical defect is open, and the weaknesses the measurements show are carried into the recommendations of Chapter 5.

Quality in use, which ISO/IEC 25010:2011 defines as the quality experienced by users in their own context, was measured by the acceptance test of Section 4.3.7 in the manner of ISO/IEC 25022 (Appendix M, Table M-20). For effectiveness, all 34 attempted tasks were completed, 31 of them without help. For efficiency, the median was 60 seconds a task with 0.2 wrong turns. For context coverage, the system was used on the landlady's laptop, an Android phone and an iPhone. In the second round every task was again completed (Appendix M, Table M-30), and efficiency was not measured (Section 4.3.7).

### 4.4.6 Optimization in Response to the Evaluation

Objective 4 asks for the system to be evaluated **and optimized**. Table 4-19 groups the 51 changes made in response to feedback and testing by where each finding came from. Every change, with its finding and the characteristic it serves, is in Appendix M, Table M-18. Most were made the day the finding arrived.

**Table 4-19.** Optimizations made in response to evaluation, by source

| Source | Changes | Characteristics served | Examples of what was found |
|---|---|---|---|
| The landlady's review on a phone | 4 | Usability, Portability | Edit buttons that touch screens never showed; a tenant register wider than the phone |
| Screen-versus-database audit | 7 | Functional Suitability, Reliability, Usability, Security | A payment history showing no payments; a placeholder move-in date for every tenant |
| Development reviews (performance, requirements, client, mobile, final review) | 11 | Usability, Functional Suitability, Reliability, Performance Efficiency, Portability | Repairs for an empty unit; page names that differed between menu, tab and heading |
| Pre-testing audit and Lighthouse | 5 | Usability, Security, Performance Efficiency, Reliability | One shared starting password for every tenant; no way back for a forgotten password |
| Testing day (landlady's use, walkthrough, Android phone) | 3 | Reliability, Usability | Changes not reaching the other person until a refresh; ₱0 shown with no connection |
| Evaluation account and security reviews | 3 | Security, Functional Suitability, Reliability | A request from another website answered with a server error; breached passwords accepted |
| Technical evaluators' review | 14 | Usability, Functional Suitability, Security, Reliability | Filters in a pop-up with an Apply button; no way for a tenant to withdraw a request |
| The survey | 4 | Usability, Reliability, Functional Suitability | Usability items rated 3 before the evaluators' changes; a tenant asking to see the deposit |
| **All** | **51** | | |

The four technical evaluators who reviewed the demonstration copy wrote their comments on the questionnaire after the period Table 4-19 counts. One asked for three things: figures kept rather than loaded again on Monthly Income and the dashboards, which the second iteration did (Appendix M, Table M-31), finished repair requests listed by their dates so they can be looked back on, and a record archived rather than deleted. The last two were not done in this study (Chapter 5, recommendation 4). The three technical evaluators of the second round named nothing that needed improving. Their comments praised how quickly the screens change, the billing arithmetic, the confirmation asked before a deletion, and how steady the pages hold on a phone.

Table 4-20 counts the defects found by each activity in Sections 4.3 and 4.4, by their highest severity and what became of them. **Critical** means a wrong amount in the records or lost data. **High** means a wrong amount shown, a core task blocked, or an exploitable weakness. **Medium** means a misleading display or a weakness with a workaround. **Low** covers usability, appearance or hardening.

**Table 4-20.** Defects found, by activity, severity and disposition

| Activity | Found | Highest severity | Resolved | Open or accepted |
|---|---|---|---|---|
| Screen-versus-database audit (Table 4-9) | 6 | High: a tenant's payments not shown; a voided receipt shown as paid | 6 | None |
| Functional walkthrough (Appendix M, Table M-1) | 1 | Medium: ₱0 instead of "—" with the connection cut | 1 | None: passed when re-tested on the landlady's laptop |
| Testing day on phones | 1 | Medium: the "cannot tell whether it was saved" message late on Android | 1 | None: on time when re-tested on the same phone |
| Check of the evaluation account | 1 | High: online payment refused for an amount owed | 1 | None |
| Security review and outside probe | 12 | High, operational: the hosting provider's visitor challenge could block payment notifications | 9 | 3 accepted as low risk: sign-in attempt limits counted per server instance, signing out not revoking a token, a wide connection policy |
| Technical evaluators | 14 | Medium: phone, email and photo validation | 14 | None |
| **All activities** | **35** | | **32** | **3 accepted** |

No defect found was critical. The two defects of Section 4.3.1 that lost a record are not among these: they were found and corrected in the first iteration, before pilot testing. The security audit records the reason each of the three accepted items was accepted.

One further defect, of high severity, was found after these activities, in an audit of the live records, and corrected the same day: two tenants were shown as owing a month they had already paid. It is outside the counts above, which are the

testing activities' own. Appendix M, Table M-27, describes it, its cause and its correction.

### 4.5 Deployment Plan and Strategies

The system is hosted on Vercel at a public web address, with the database on Supabase. The optional GCash payment through Adyen works end to end in Adyen's test environment, which takes no real money. It is not live, because going live needs the owner's merchant onboarding documents, and this study did not do that onboarding. Every approved change is deployed automatically from the project's main code branch. The researchers give the system to the boarding house free, with no subscription to pay. Under the memorandum of agreement signed with the owner, they provide the system, its user manual and training and maintain and support it for an agreed period after turnover, and the boarding house's records remain its own. Until turnover the hosting, database and payment gateway accounts are held by the research team. At turnover they pass to the landlady, together with the member of the team who is her son and knows the system. Table 4-21 lists what is needed to use the system, and Table 4-22 the deployment plan.

**Table 4-21.** Software and hardware requirements for deployment

| Side | Requirement | Used in testing |
|---|---|---|
| Landlady | A current web browser on a computer, and an internet connection | The researchers' Acer Nitro V15 laptops on Windows 11 with Chrome, Edge and Firefox (Table 4-10), one of which the landlady used |
| Tenants and visitors | A current web browser (Chrome, Edge, Firefox or Safari) on a phone or computer, and an internet connection | Infinix GT20 with Android and Chrome; iPhone 15 and iPhone 13 with Safari; the three tenants' own phones, models not recorded |
| Hosting | Vercel (application), Supabase (PostgreSQL database), Adyen test account with GCash (optional online payment; not live, takes no real money) | Vercel; PostgreSQL 17.6 on Supabase; Adyen Web Drop-in 6.44 and API Library 32.0 (Table 4-4) |
| Administration | A computer that can run Node.js, used only for backups and verification runs | Node.js 22 on the researchers' Windows 11 computer |

**Table 4-22.** Deployment plan and strategies

| Stage | Activity | Status |
|---|---|---|
| 1. Environment preparation | Hosting, database and gateway set up; security settings applied | Done |
| 2. Record transfer | The landlady's workbook transferred: 937 income rows and 1,327 expense allocations | Done |
| 3. Pilot use | The system used alongside the landlady's existing records; differences investigated | In progress |
| 4. Training and hand-over | The landlady and tenants shown how to use the system; user manual (Appendix K) handed over; accounts issued | Done: every tenant account issued with its own starting password; the landlady guided through the system, and the landlady and tenants using it on their own accounts (Section 4.3.7) |
| 5. Full transition | The system becomes the landlady's main record once both records agree | Not yet: the system runs beside the landlady's own records |

The plan manages three risks:

- **The records are the boarding house's real financial records.** A backup is taken before any change to the data, and every change is a numbered migration that can be reviewed before it runs.
- **Gateway and database credentials** are kept outside the code, passed between the researchers separately, and never written in any document. A check for committed credentials runs before every commit.
- **The five historical receipt anomalies** are kept exactly as the landlady wrote them and reported on every verification run, so they are never mistaken for errors made by the system.

# 5 SUMMARY, CONCLUSIONS, AND RECOMMENDATIONS

This chapter summarizes the study, states the conclusions drawn from its results, and gives recommendations for the landlady, for the continued development of the system, and for future researchers.

## 5.1 Summary

Small boarding houses that run on spreadsheets, paper receipts and messages are not served by the property management systems on the market, which assume larger operations. This study designed, developed and evaluated Hivelet, a web-based apartment management system delivered as a Progressive Web Application, for the Fe Galang Da Silva Boarding House, a 33-unit property in Legazpi City. The results are summarized below in the order of the specific objectives.

1. **The existing practices were analyzed.** The landlady kept a two-sheet workbook on a USB flash drive and a paper invoice book, collected rent in cash, and received requests by message or in person. Transferring her records, 937 income rows and 1,327 expense allocations, showed that 43% of the income rows lacked an anniversary date and deposit and that five invoice numbers had each been used for two payments. The analysis produced 44 functional requirements and 49 business rules. Graded against the final system, 33 of the requirements were implemented as worded, 9 in part and 2 not as worded, against 25 as worded early in the first iteration.

2. **The system was developed** in two Agile iterations with Vue, Express and PostgreSQL, with the six features the objective names: tenant and room management, booking and reservation management, financial tracking with optional online payment through Adyen with GCash, maintenance ticketing and notification, role-based access control, and Progressive Web Application support. It holds the boarding house's real records. The GCash payment works end to end in Adyen's test environment and takes no real money. It is not live, because going live needs the owner's merchant onboarding documents.
3. **The system was pilot tested.** Twenty automated check suites, run against the live system, all passed, including 490 report reconciliation checks. A screen-by-screen comparison with the live records found six defects the checks had missed, all corrected. The landlady performed the walkthrough herself: 24 of its 30 rows passed the first time, one failed and five were not performed as written. Three tenants completed every task they attempted, 31 of 34 attempts without help. Of the 83 tests executed across the three levels, 82 passed (98.8 percent), and no critical defect was found. Every screen measured was usable within about three seconds on a phone, and no request failed under twelve simulated simultaneous visitors or six simulated readers of the boarding house's records. After the redesign, 109 of the 114 test cases were run again and none failed, and every screen timed again was ready sooner than before except sign-in. On the

redesigned system, ten tenants and four prospective tenants completed every task, 151 of 160 and 31 of 32 attempts without help.

4. **The system was evaluated** by the five steps of ISO/IEC 25040: twenty-seven of the model's thirty-one sub-characteristics had a measured result and twenty-five were rated. In response to feedback and testing, 51 changes were made, among them each of the first technical evaluators' fourteen comments, and the second iteration then redesigned the screens. The fourteen users, the landlady, ten tenants and three prospective tenants, rated the redesigned system 4.84 overall, Very High Quality. Of the characteristics all three user groups rated, Performance Efficiency was highest (4.86) and Reliability lowest (4.78). Three technical evaluators rated it 4.90 overall, Very High Quality, with Compatibility and Portability highest (5.00) and Maintainability lowest (4.73). Both groups rated every characteristic Very High Quality. Before the redesign, five users had rated the system 4.75 and five technical evaluators 4.34.

## 5.2 Conclusions

Based on the results of the study, the following conclusions were drawn:

1. **The first objective was achieved.** The existing practices were analyzed in all four areas the objective names, and every gap found was traced to a requirement and to the module that meets it (Chapter 4, Table 4-2). The main problem at the property was not the lack of a digital tool but the lack of connection between its records.

2. **The second objective was achieved.** All six features the objective names were developed and are in use on the boarding house's real records, and they keep the landlady's own report layout, invoice numbers and cash as the main way to pay, so the system follows the way she already keeps the accounts (Chapter 4, Sections 4.2.2 to 4.2.7). The optional GCash payment is the one feature not in use with real money: it works end to end in Adyen's test environment, and going live needs the owner's merchant onboarding documents.
3. **The third objective was achieved.** The system was pilot tested for functionality, responsiveness and operational performance, and 82 of the 83 tests executed passed (98.8 percent), above the course standard of 90 percent, with no critical defect (Chapter 4, Tables 4-12 and 4-20). The one failure, a ₱0 shown with the connection cut, passed when re-tested on the landlady's laptop. The redesigned system kept the result: none of the 109 test cases rerun after the redesign failed, every task of the second round was completed, and the landlady accepted the system and signed the acceptance form (Section 4.3.7).
4. **The fourth objective was achieved.** The system was evaluated on all eight characteristics of ISO/IEC 25010:2011 by the process of ISO/IEC 25040 and optimized in response (Chapter 4, Table 4-19). Its users rated it Very High Quality overall (4.84) and its technical evaluators Very High Quality overall (4.90), with every characteristic of both Very High Quality on the scale of Chapter 3's Table 3-5, and the measured evidence agreed on six

characteristics and was less favorable on Security and Maintainability (Chapter 4, Tables 4-17 and 4-18). Both ratings are higher than the first round's, 4.75 and 4.34, which a smaller group and other technical evaluators gave on the system before the redesign, and they are the view of these seventeen respondents rather than of a wider population.

## 5.3 Recommendations

Based on the summary and conclusions of the study, the following are recommended:

**For the landlady of the Fe Galang Da Silva Boarding House**

1. Enter each receipt from her receipt book on Monthly Income, so that the system's receipts agree with the book and the system can become her main record (Appendix M, Table M-27; Chapter 4, Table 4-22).
2. Have a backup taken before any change to the records, as the researchers did throughout (Section 4.3.3).

**For the continued development of the system**

3. Take the GCash payment live, if the owner wants it, by completing Adyen's merchant onboarding with the owner's merchant onboarding documents, which this study did not do, and let the system refund a GCash payment the landlady rejects (Sections 1.4 and 4.2.4). Close the two weaknesses Chapter 4 names: keep the sign-in token in a cookie that no script can read, and add a second sign-in factor for the administrator (Section 4.4.4).
4. Add automated browser tests that sign in and perform each step of the landlady's walkthrough (Section 4.3.3). As a technical evaluator advised

(Section 4.4.6), keep a deleted repair request or inquiry as an archived record and list finished repairs by their dates, and call the Monthly Income action Void rather than Delete (Appendix L, case A-25).

5. Make the public page appear faster on phones, where its largest element took 4.2 to 4.4 seconds on slow 4G after the redesign, by pre-rendering it and serving its typefaces from the site itself (Section 4.3.5). Lighten the work the landlady's Overview does as it loads, which kept a simulated slow phone busy for 1.8 seconds (Appendix L, case PF-10).
6. Raise **Maintainability**, the technical evaluators' lowest at 4.73, by moving the database work now written inside the request handlers into service modules, one per kind of record. Help each new tenant through the first sign-in, where tenants needed help most in both rounds (Sections 4.3.7 and 4.4.3).

**For future researchers**

7. Check a system against its own real data, not only against test cases: the most important defects in this study passed every automated check and were found only by comparing each screen with the records it claimed to show (Section 4.3.3).
8. Evaluate over a longer period of real use with every tenant, measuring availability over months and the landlady's time on record-keeping, with an instrument built on ISO/IEC 25010:2023 (Section 4.4; Appendix M, Table M-21).

# REFERENCES

Altinay, Z., Arslan, R. U., & Sharma, R. C. (2024). Assessing student satisfaction and institutional efficiency in dormitory management: A qualitative analysis based on student perspectives. *Sustainability, 16*(20), 8823. https://doi.org/10.3390/su16208823

Anuar, N. N., & Othman, M. K. (2024). Development and validation of progressive web application usability heuristics (PWAUH). *Universal Access in the Information Society, 23*(1), 245–273. https://doi.org/10.1007/s10209-022-00925-4

Biørn-Hansen, A., Majchrzak, T. A., & Grønli, T.-M. (2017). Progressive web apps: The possible web-native unifier for mobile development. In *Proceedings of the 13th International Conference on Web Information Systems and Technologies (WEBIST 2017)* (pp. 344–351). SciTePress. https://doi.org/10.5220/0006353703440351

Burke, D. (2026). Property management challenges in residential multifamily apartment complexes. *American Journal of Industrial and Business Management, 16*(1), 1–19. https://doi.org/10.4236/ajibm.2026.161001

Canlas, W. J. R., Andres, I. V., Kato, K. C., Vinuya, L. V., & Rivera, M. G. (2026). FixHub: A community-based maintenance reporting system. *International Journal for Multidisciplinary Research, 8*(3). https://doi.org/10.36948/ijfmr.2026.v08i03.79323

Caparos, E., Gabriel, C., Prudente, R., & Romero, R. (2025). Cloud based property management system: Enhancing real estate operation and tenant experience. *International Journal of Innovative Science and Research Technology, 10*(5), 1782–1791. https://doi.org/10.38124/ijisrt/25may851

Encarnacion, R. E., Laulita, F. A. G., Cruiz, H. J. B., & Quinte, J. L. E. (2025). Design and evaluation of an online payment monitoring system for student boarding houses near a university campus in the Philippines. *Proceedings of International Exchange and Innovation Conference on Engineering & Sciences (IEICES), 11*, 1225–1231. https://hdl.handle.net/2324/7395667

Etikan, I., Musa, S. A., & Alkassim, R. S. (2016). Comparison of convenience sampling and purposive sampling. *American Journal of Theoretical and Applied Statistics, 5*(1), 1–4. https://doi.org/10.11648/j.ajtas.20160501.11

Ghasemaghaei, M., Ebrahimi, S., & Hassanein, K. (2018). Data analytics competency for improving firm decision making performance. *Journal of Strategic Information Systems, 27*(1), 101–113. https://doi.org/10.1016/j.jsis.2017.10.001

International Organization for Standardization & International Electrotechnical Commission. (2011a). *Systems and software engineering: Systems and software Quality Requirements and Evaluation (SQuaRE): System and software quality models* (ISO/IEC Standard No. 25010:2011).

International Organization for Standardization & International Electrotechnical Commission. (2011b). *Systems and software engineering: Systems and software Quality Requirements and Evaluation (SQuaRE): Evaluation process* (ISO/IEC Standard No. 25040:2011).

International Organization for Standardization & International Electrotechnical Commission. (2016a). *Systems and software engineering: Systems and software Quality Requirements and Evaluation (SQuaRE): Measurement of quality in use* (ISO/IEC Standard No. 25022:2016).

International Organization for Standardization & International Electrotechnical Commission. (2016b). *Systems and software engineering: Systems and software Quality Requirements and Evaluation (SQuaRE): Measurement of system and software product quality* (ISO/IEC Standard No. 25023:2016).

International Organization for Standardization & International Electrotechnical Commission. (2023). *Systems and software engineering: Systems and software Quality Requirements and Evaluation (SQuaRE): Product quality model* (ISO/IEC Standard No. 25010:2023).

International Organization for Standardization, International Electrotechnical Commission, & Institute of Electrical and Electronics Engineers. (2022). *Software and systems engineering: Software testing: Part 1: General concepts* (ISO/IEC/IEEE Standard No. 29119-1:2022).

Jing, F., & Lim, C.-K. (2021). An effect of management service quality on residential satisfaction and recommended intention of high-rise apartments. *Turkish Journal of Computer and Mathematics Education, 12*(4), 201–211. https://doi.org/10.17762/turcomat.v12i4.492

Kotkar, A. R., Kaldate, V. N., Kandekar, R. B., Andhale, S. A., & Wamane, S. J. (2026). Revolutionizing student accommodation: A MERN based rental system for seamless property management. *International Journal of Engineering Research & Technology, 15*(2). https://doi.org/10.5281/zenodo.18641825

Laudon, K. C., Laudon, J. P., & Traver, C. G. (2026). *Management information systems: Managing the digital firm* (18th ed.). Pearson.

Likert, R. (1932). A technique for the measurement of attitudes. *Archives of Psychology, 22*(140), 1–55.

Magno, J. M., Pelayo, R. S., & Soberano, K. T. (2024). Apartment rental management system for real-time transaction and task organization. *International Journal of Multidisciplinary Research and Analysis, 7*(7), 3287–3294. https://doi.org/10.47191/ijmra/v7-i07-29

Mangca, D. C. (2023a). Assessment of a web-based boarding house booking system with notification capability. *International Research Journal of Advanced Engineering and Science, 8*(2), 59–62. http://irjaes.com/wp-content/uploads/2023/04/IRJAES-V8N2P48Y23.pdf

Mangca, D. C. (2023b). Boarding house booking system: Design and development using Laravel framework with SMS notification. *International Journal of Innovative Science and Research Technology, 8*(7), 1887–1892. https://ijisrt.com/assets/upload/files/IJISRT23JUL1243.pdf

Mia, I. B., Jimenez, S., Habaradas, R., Ranieses, J. J., Javier, C., & Enriquez, J. (2024). Digital technology adoption among Philippine micro-, small-, and medium-sized enterprises: Barriers, enablers & challenges during COVID-19. *Journal of Business, Ethics and Society, 4*(1), 1–17. https://doi.org/10.61781/4-1i2024/1bmlm

Muawwal, A. (2024). The implementation of PWA (progressive web app) technology in enhancing website performance & mobile accessibility. *Buletin Pos dan Telekomunikasi, 22*(1). https://doi.org/10.17933/bpostel.v22i1.395

Nielsen, J., & Landauer, T. K. (1993). A mathematical model of the finding of usability problems. In *Proceedings of the INTERACT '93 and CHI '93 Conference on Human Factors in Computing Systems* (pp. 206–213). ACM Press. https://doi.org/10.1145/169059.169166

Pavia, J. F., Bonilla, K. M. J., Cordovilla, W. J. B., Pama, J. R. N., Baula, C. L., & Bermejo, E. O. (2022). Bicol College property management system for Bachelor of Science in Hospitality Management students: Mock hotel operation. *JPAIR Multidisciplinary Research, 50*(1), 52–66. https://doi.org/10.7719/jpair.v50i1.820

Pernate, W. J. A., Rosales, D. R. E., Aya-ay, L. A. S., & Artigas, A. L. (2025). BH Niyo App: A smart mobile solution for boarding house room rentals in Ormoc City. *Business, Education, Social Sciences, and Technology, 1*(2), 83–90. https://doi.org/10.69478/BEST2025v1n2a013

Pressman, R. S., & Maxim, B. R. (2020). *Software engineering: A practitioner's approach* (9th ed.). McGraw-Hill Education.

Puspasari, B. D., Irawan, J. D., Wicaksono, R. A., Pratama, D. R., & Ilmi, A. B. (2025). Development of information system for boarding houses. *International Journal of Computer Science and Information Technology, 2*(1), 8–12. https://doi.org/10.36040/ijcomit.v2i1.13659

Republic Act No. 10173. (2012). *Data Privacy Act of 2012*. National Privacy Commission. https://www.privacy.gov.ph/data-privacy-act/

Saputra, R. G., Susanto, R., & Pradana, A. I. (2025). Implementation of web-based room management system for boarding house operations. *Bit-Tech (Binary Digital - Technology), 8*(1), 830–839. https://doi.org/10.32877/bt.v8i1.2738

Setty, Y. (2022). *Management system for an apartment* [Master's capstone project, Governors State University]. OPUS Open Portal to University Scholarship. https://opus.govst.edu/cgi/viewcontent.cgi?article=1542&context=capstones

Thevaraju, D. P., Zakaria, Z., Sukri, K. A. M., & Kasim, S. (2019). Vista Angkasa Apartment Management System. *International Journal of Advanced Science Computing and Engineering, 1*(2), 76–84. https://doi.org/10.62527/ijasce.1.2.4

Ullah, F., Sepasgozar, S. M. E., & Wang, C. (2018). A systematic review of smart real estate technology: Drivers of, and barriers to, the use of digital disruptive technologies and online platforms. *Sustainability, 10*(9), 3142. https://doi.org/10.3390/su10093142

World Wide Web Consortium. (2023). *Web Content Accessibility Guidelines (WCAG) 2.2* (W3C Recommendation). https://www.w3.org/TR/WCAG22/

# APPENDICES

# APPENDIX A: LETTER OF REQUEST

**[TO DO: scan of the signed letter of request to the owner]**

# APPENDIX B: APPOINTMENTS

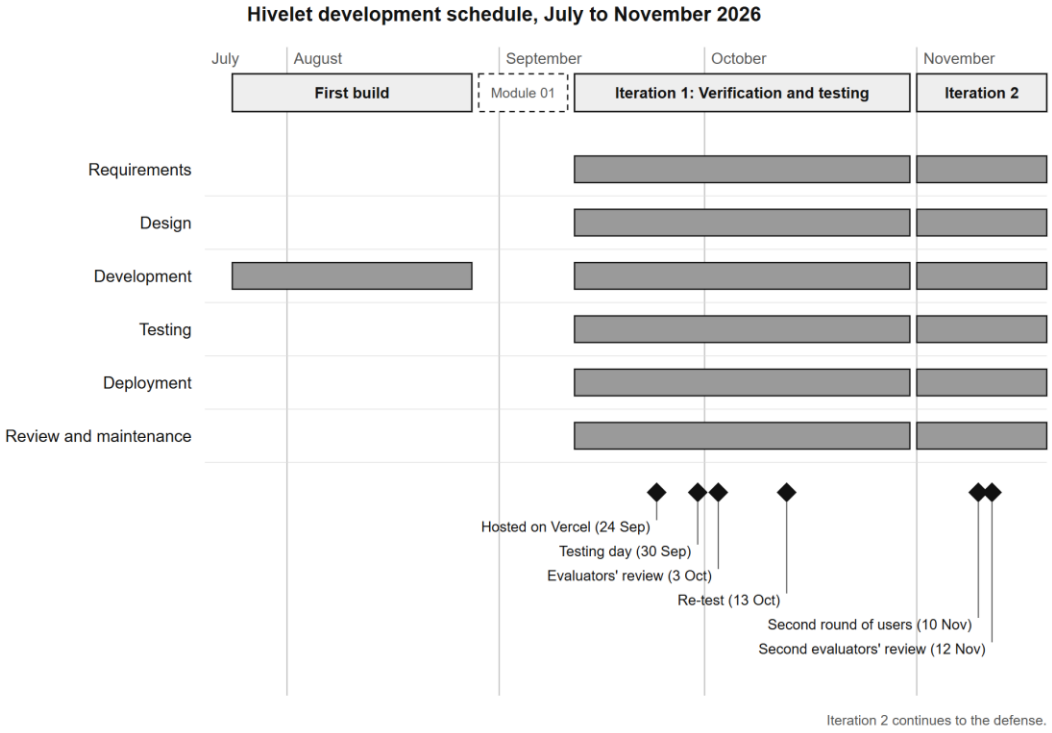
**[TO DO: scans of the adviser appointment papers]**

# APPENDIX C: GANTT CHART

Figure C-1 shows the schedule as it ran, taken from the project's version history (Appendix M, Table M-19). Each iteration passed through the phases of the Agile cycle of Section 3.2. The dashed block is the preparation and submission of the Module 01 documents, between the first build and the first iteration. The first build ran from 24 July to 28 August 2026, Iteration 1 from 12 September to 31 October, and Iteration 2 from 1 November. Iteration 1 began by checking the system and its documents against each other, from 12 to 14 September, before the system was put online and tested. Four further technical evaluators reviewed a demonstration copy of the system from 15 to 18 October. On the redesigned system, the second round of users ran from 31 October to 10 November, and three technical evaluators reviewed it on 12 November. The interviews and the first design, made for the proposal before July 2026, are outside the chart.

*Figure C-1. Development schedule, July to November 2026.*

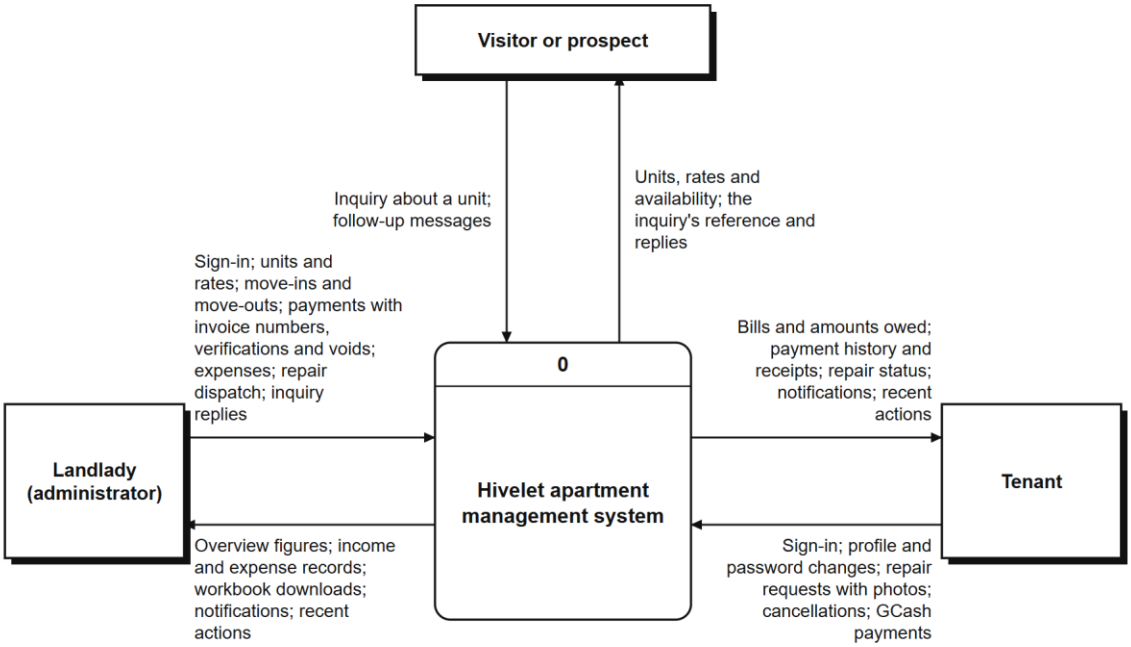
[IMAGE FILE: images/Figure C-1 - Development schedule, July to November 2026.png]



# APPENDIX D: CONTEXT DIAGRAM

*Figure D-1. Context diagram, as built (the same diagram as Figure 4-3).*

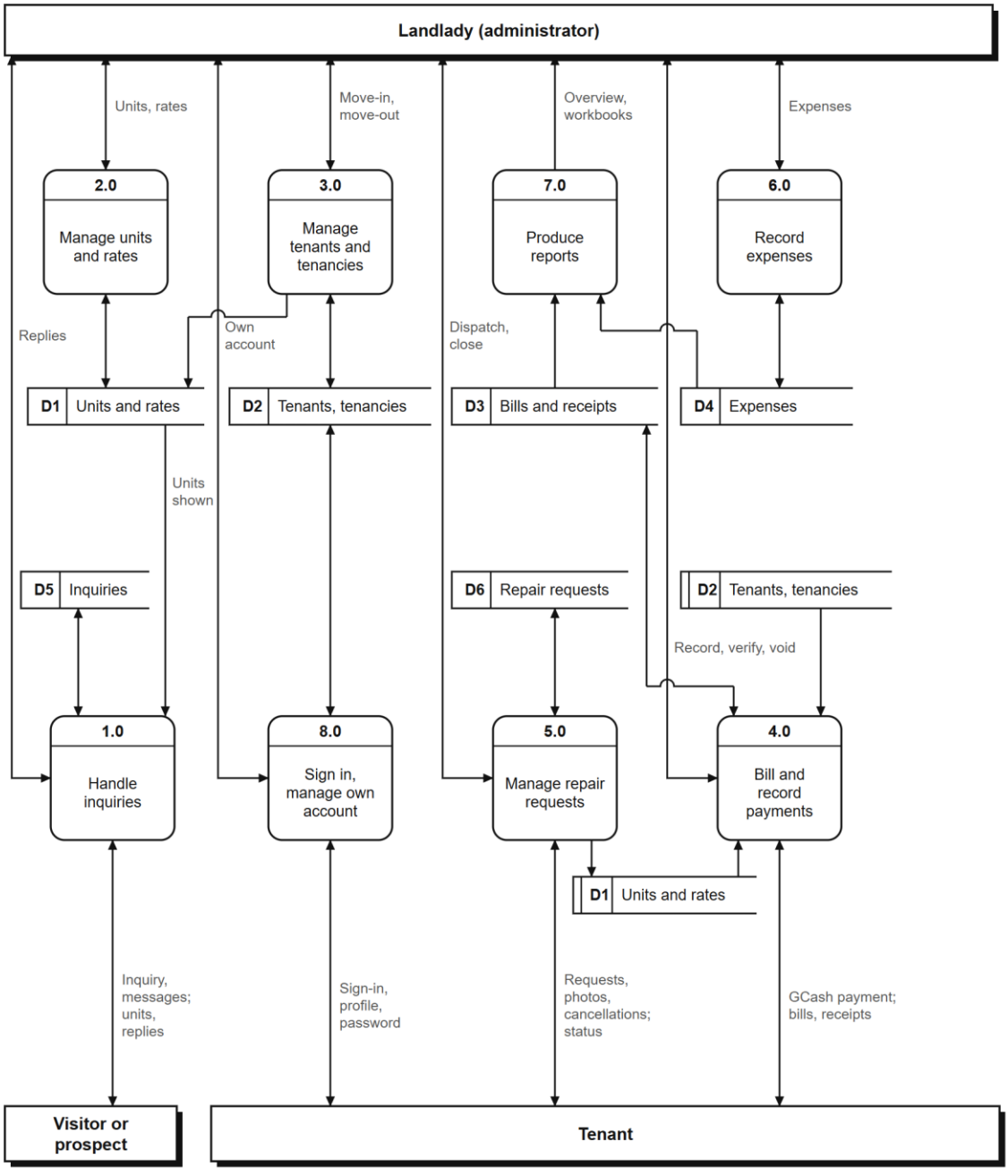
[IMAGE FILE: images/Figure D-1 - Context diagram, as built (the same diagram as Figure 4-3).png]



# APPENDIX E: LEVEL 1 DATA FLOW DIAGRAM

*Figure E-1. Level 1 data flow diagram, as built (the same diagram as Figure 4-4).*

[IMAGE FILE: images/Figure E-1 - Level 1 data flow diagram, as built (the same diagram as Figure 4-4).png]



# APPENDIX F: DATA FLOW DIAGRAMS (LEVEL 2)

Figures F-1 to F-8 open each of the eight processes of the level 1 diagram (Figure 4-4) into its steps, in the Gane and Sarson notation used for Figures 4-3 and 4-4. A rounded rectangle is a process, with its number in the band across the top. A rectangle open on the right is a data store. A shaded square is an external entity. Each step is one operation the system's screens perform. The stores and external entities keep the names of Figure 4-4. Each diagram shows every store its steps read or write, and each agrees with the stores Figure 4-4 connects to its process. Adyen, the payment gateway, is part of the payment process rather than an external entity, so it appears on no diagram as an entity. Taking a GCash payment through it is step 4.3 of Figure F-4.

*Figure F-1. Level 2 diagram of process 1.0, handle inquiries.*

[IMAGE FILE: images/Figure F-1 - Level 2 diagram of process 1.0, handle inquiries.png]

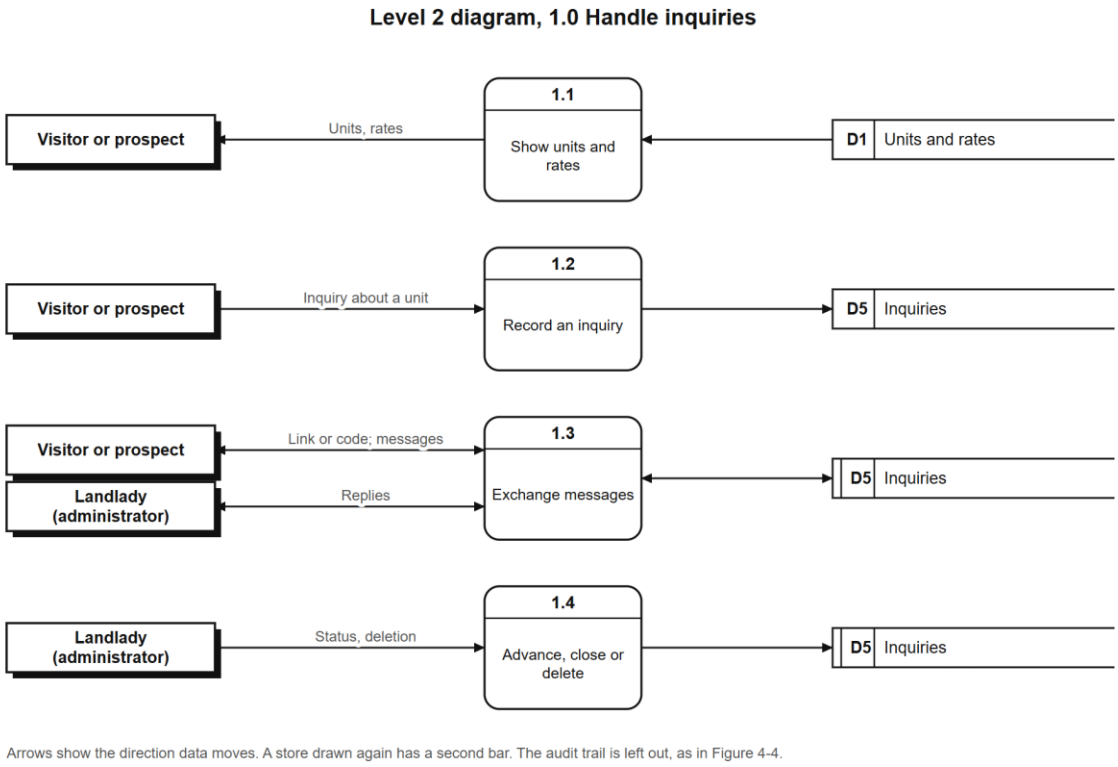



*Figure F-2. Level 2 diagram of process 2.0, manage units and rates.*

[IMAGE FILE: images/Figure F-2 - Level 2 diagram of process 2.0, manage units and rates.png]

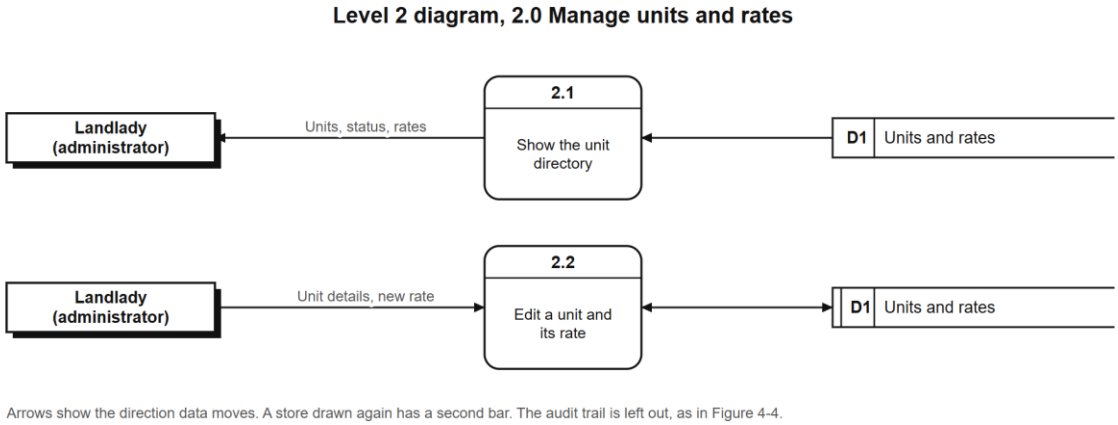
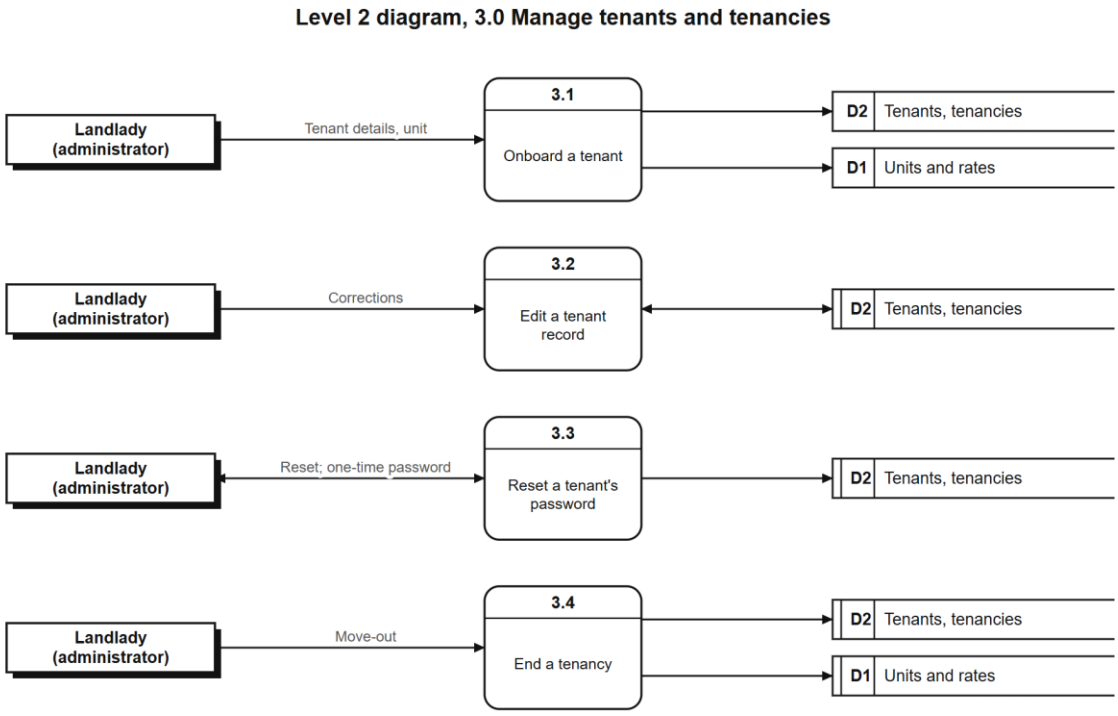


*Figure F-3. Level 2 diagram of process 3.0, manage tenants and tenancies.*

[IMAGE FILE: images/Figure F-3 - Level 2 diagram of process 3.0, manage tenants and tenancies.png]




Arrows show the direction data moves. A store drawn again has a second bar. The audit trail is left out, as in Figure 4-4.

*Figure F-4. Level 2 diagram of process 4.0, bill and record payments.*

[IMAGE FILE: images/Figure F-4 - Level 2 diagram of process 4.0, bill and record payments.png]

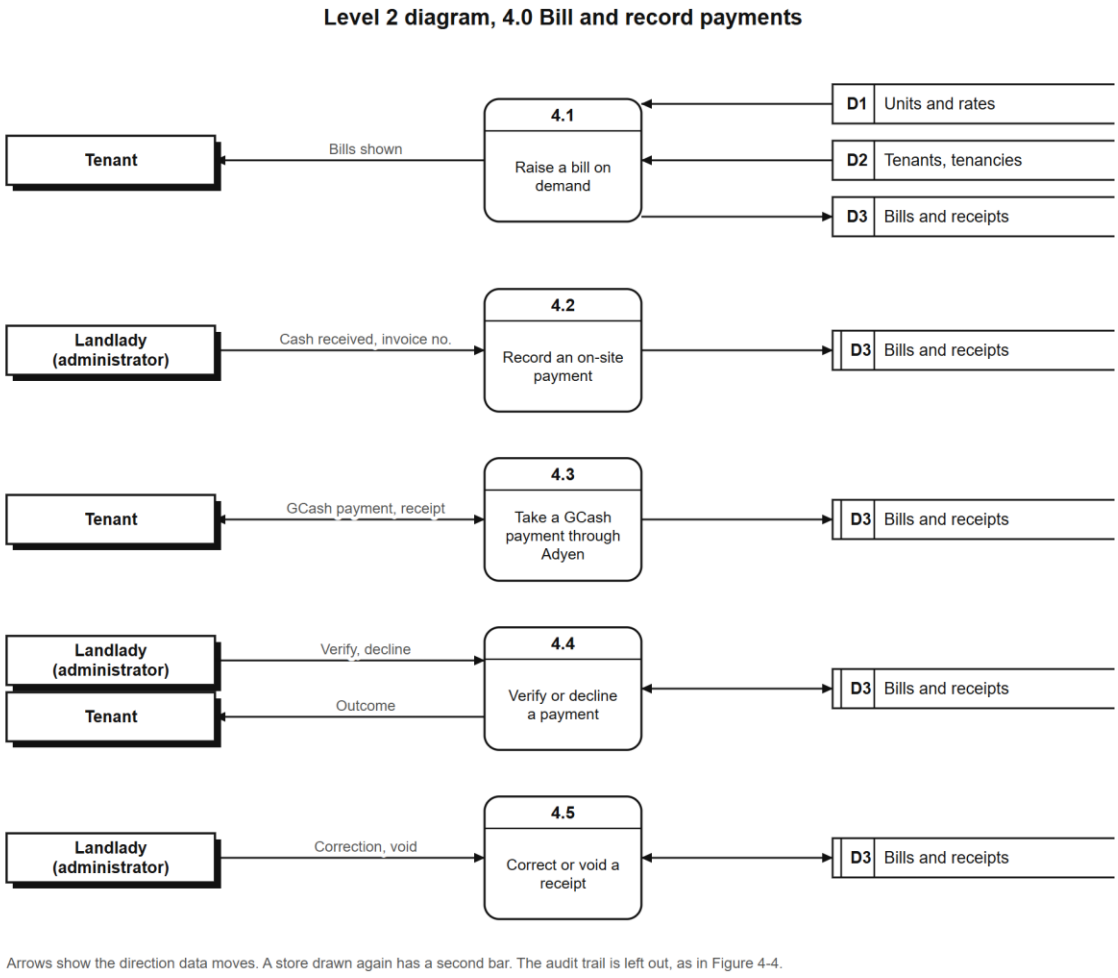
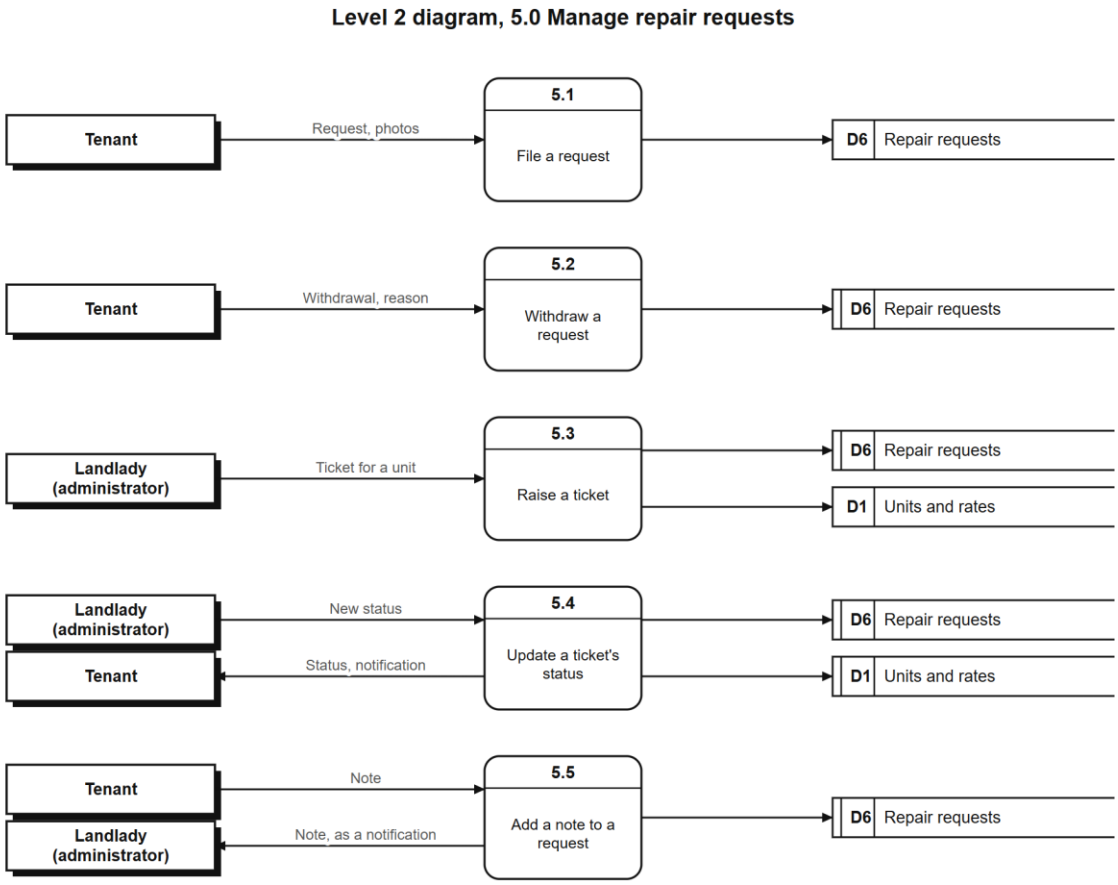


*Figure F-5. Level 2 diagram of process 5.0, manage repair requests.*

[IMAGE FILE: images/Figure F-5 - Level 2 diagram of process 5.0, manage repair requests.png]




Arrows show the direction data moves. A store drawn again has a second bar. The audit trail is left out, as in Figure 4-4.

*Figure F-6. Level 2 diagram of process 6.0, record expenses.*

[IMAGE FILE: images/Figure F-6 - Level 2 diagram of process 6.0, record expenses.png]

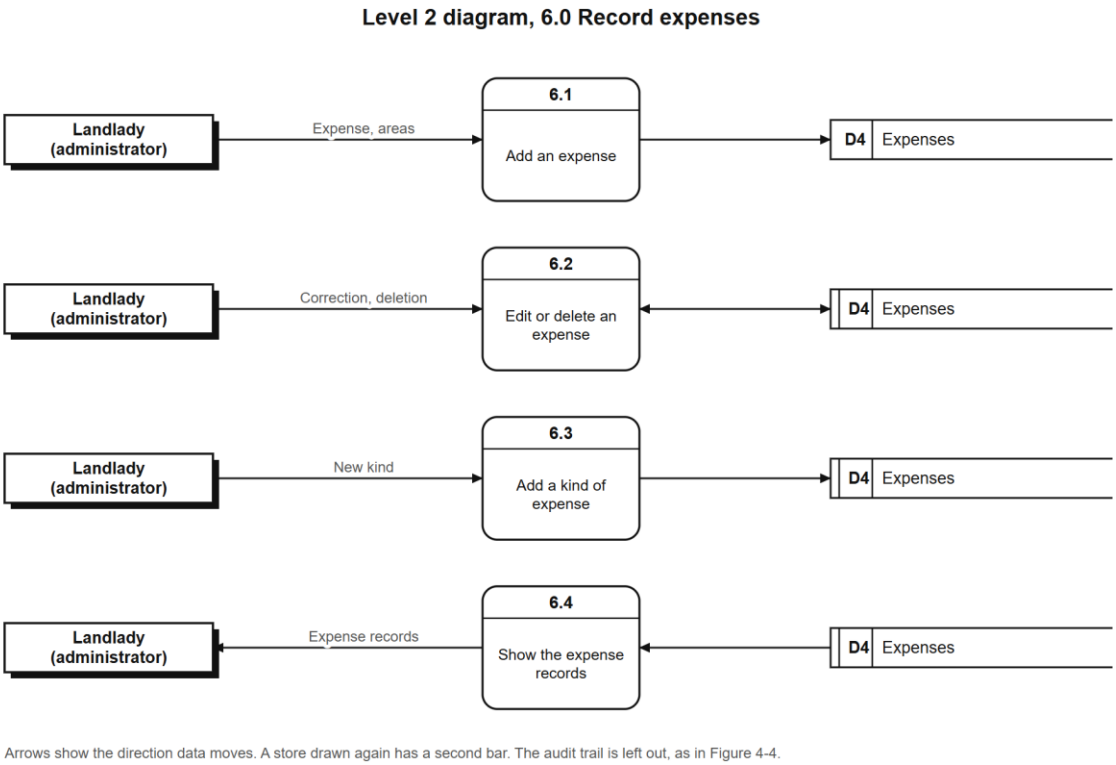



*Figure F-7. Level 2 diagram of process 7.0, produce reports.*

[IMAGE FILE: images/Figure F-7 - Level 2 diagram of process 7.0, produce reports.png]

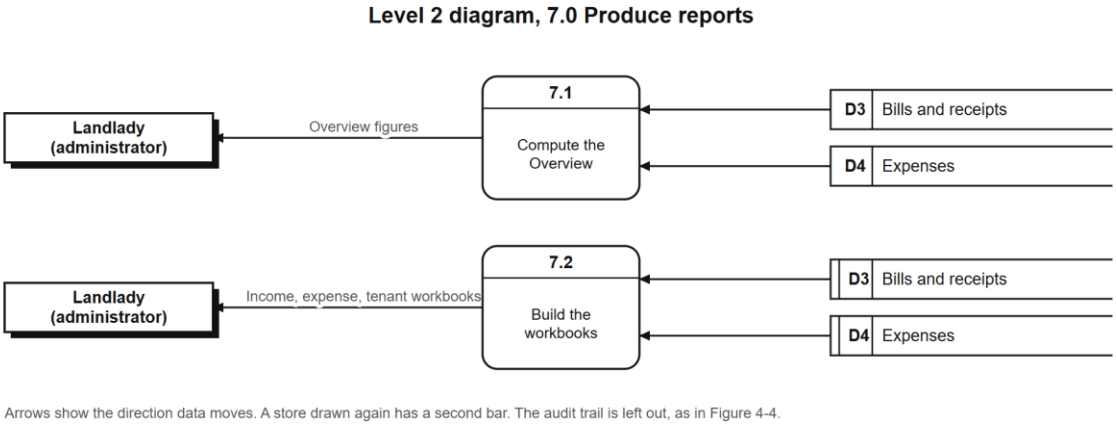
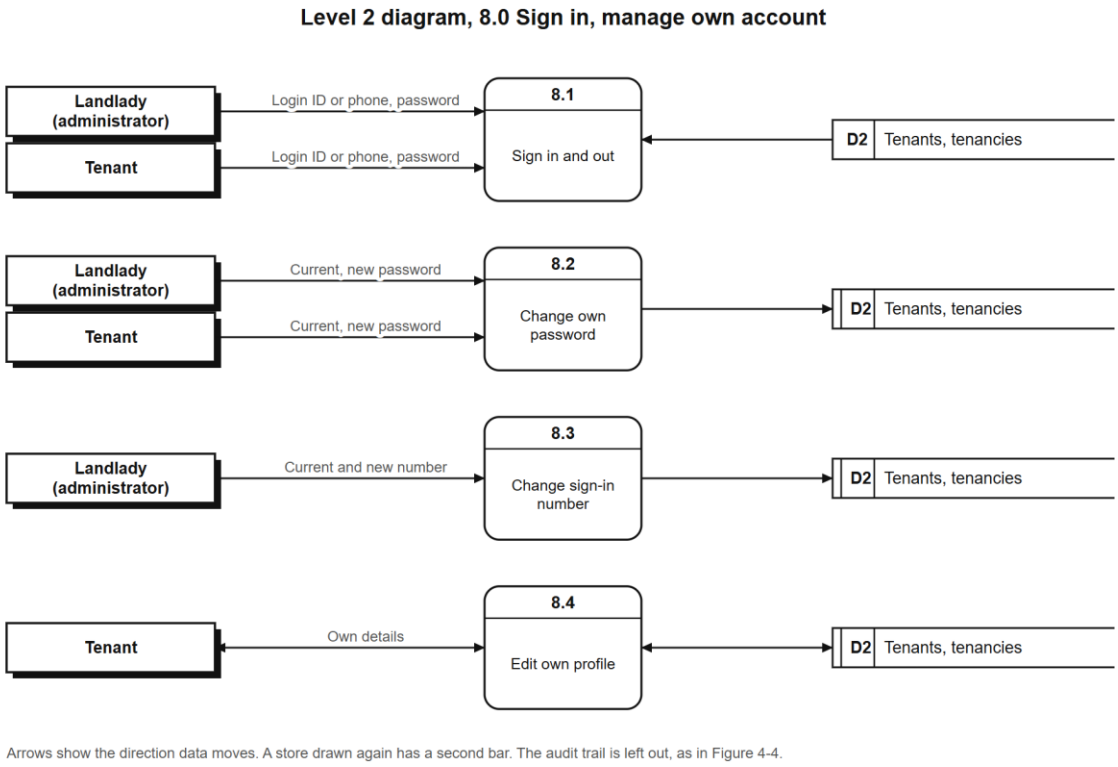


*Figure F-8. Level 2 diagram of process 8.0, sign in and manage one's own account.*

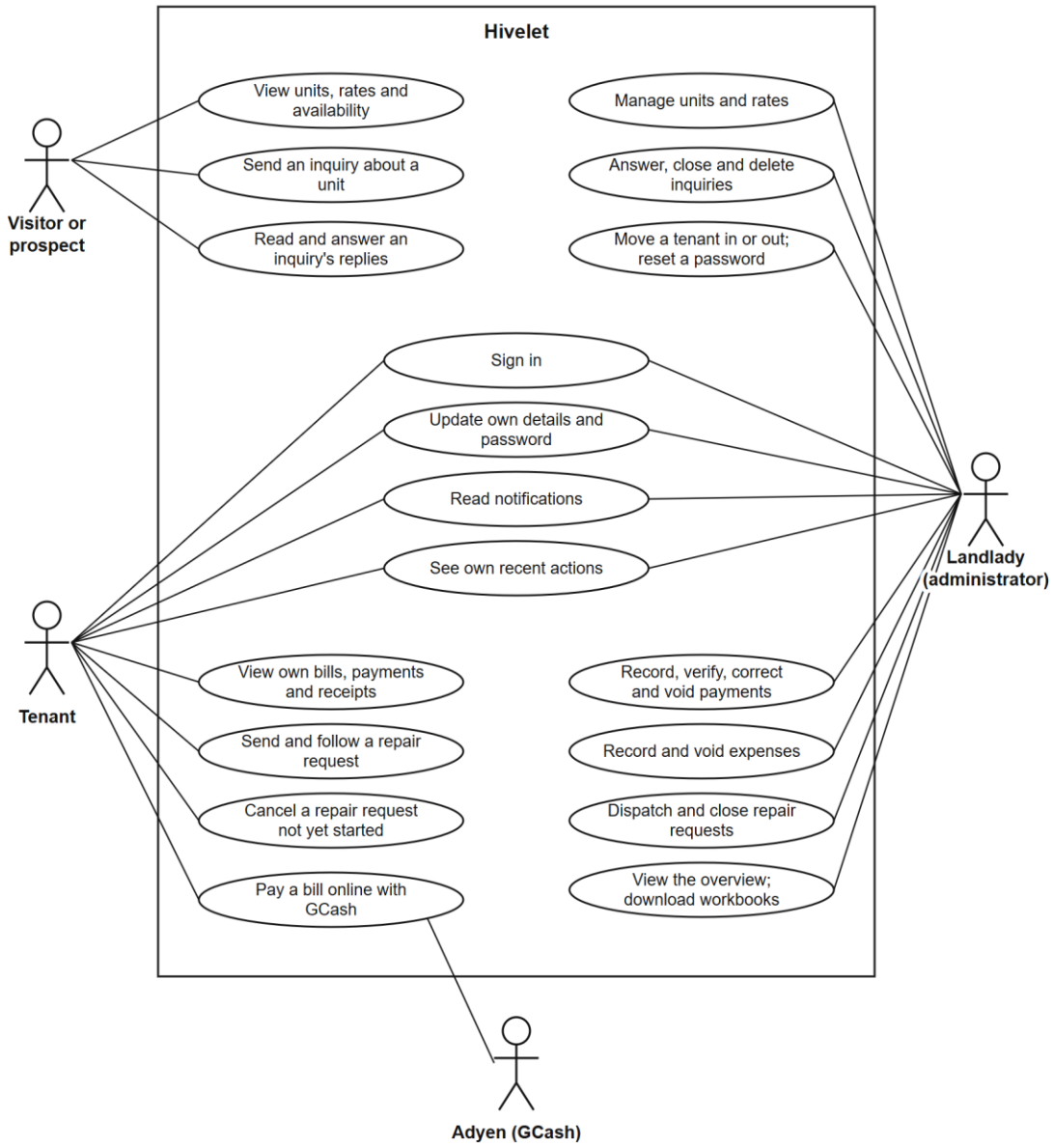
[IMAGE FILE: images/Figure F-8 - Level 2 diagram of process 8.0, sign in and manage one's own account.png]



# APPENDIX G: USE CASE DIAGRAM

*Figure G-1. Use case diagram, as built (the same diagram as Figure 4-5).*

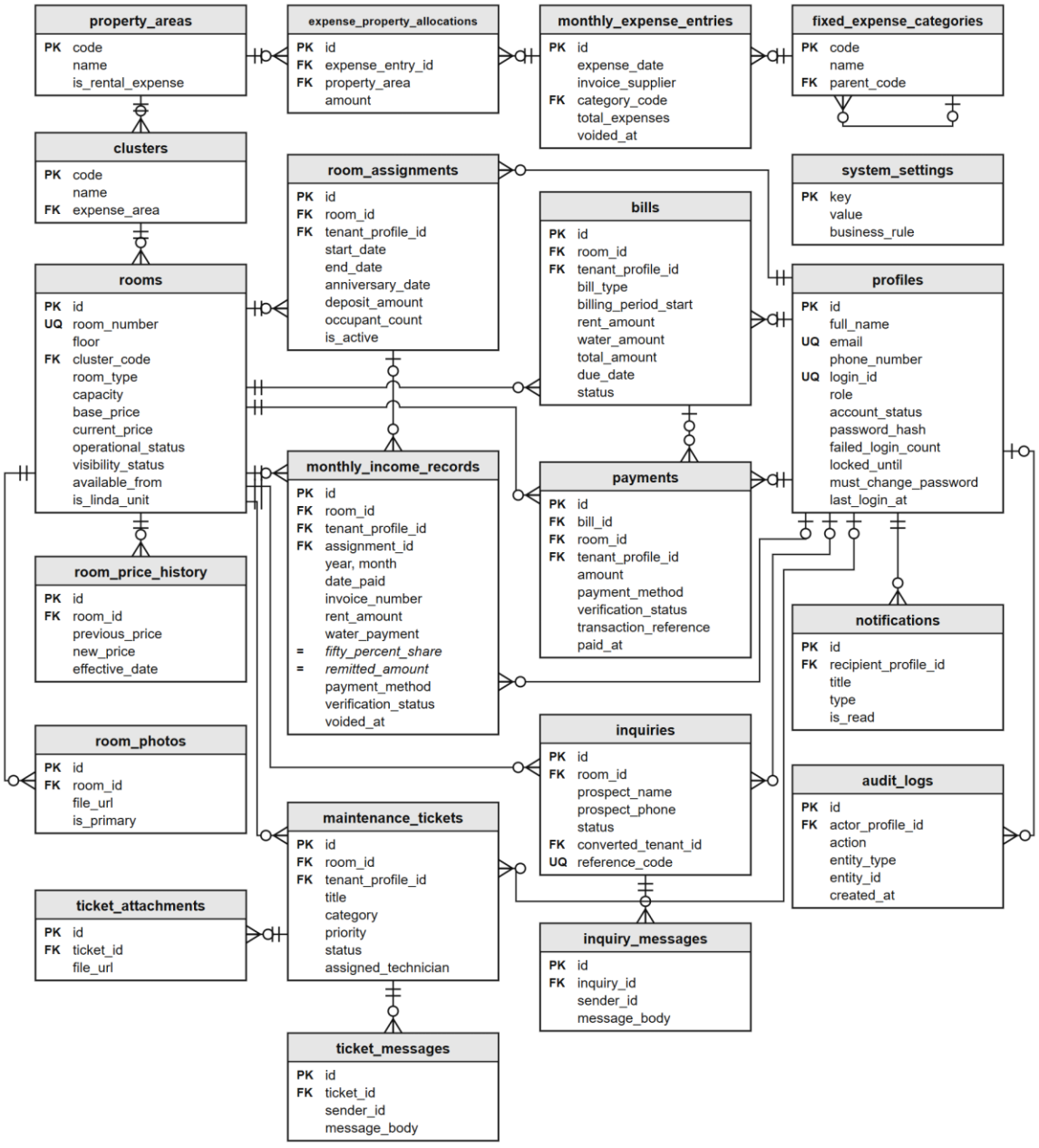
[IMAGE FILE: images/Figure G-1 - Use case diagram, as built (the same diagram as Figure 4-5).png]



# APPENDIX H: ENTITY-RELATIONSHIP DIAGRAM

*Figure H-1. Entity-relationship diagram, as built (the same diagram as Figure 4-6).*

[IMAGE FILE: images/Figure H-1 - Entity-relationship diagram, as built (the same diagram as Figure 4-6).png]




PK primary key, FK foreign key, UQ unique, = computed by the database from other columns of the row. Two bars: exactly one; bar and ring: zero or one; crow's foot and ring: zero or more. Columns ending in _by, and sender_id in the two message tables, also refer to profiles; they are left out for readability, as are three correction-backup tables.

# APPENDIX I: INITIAL DESIGN

The interface was prototyped directly in code (Section 3.1.1), so the initial design is the first working version of the system, as it stood on 28 August 2026, before the first iteration. Figures I-1 to I-4 show four of its screens, with sample tenants. They differ from the system as built (Figures 4-7 to 4-11): the first iteration removed the automatic rent increase marked beside each tenant, which the landlady does not apply, and corrected the number of units on the public page (Appendix M, Table M-19). The landlady's financial screens of that version are not shown, because they carry figures from her records.

*Figure I-1. Public landing page, 28 August 2026.*

[IMAGE FILE: images/Figure I-1 - Public landing page, 28 August 2026.png]

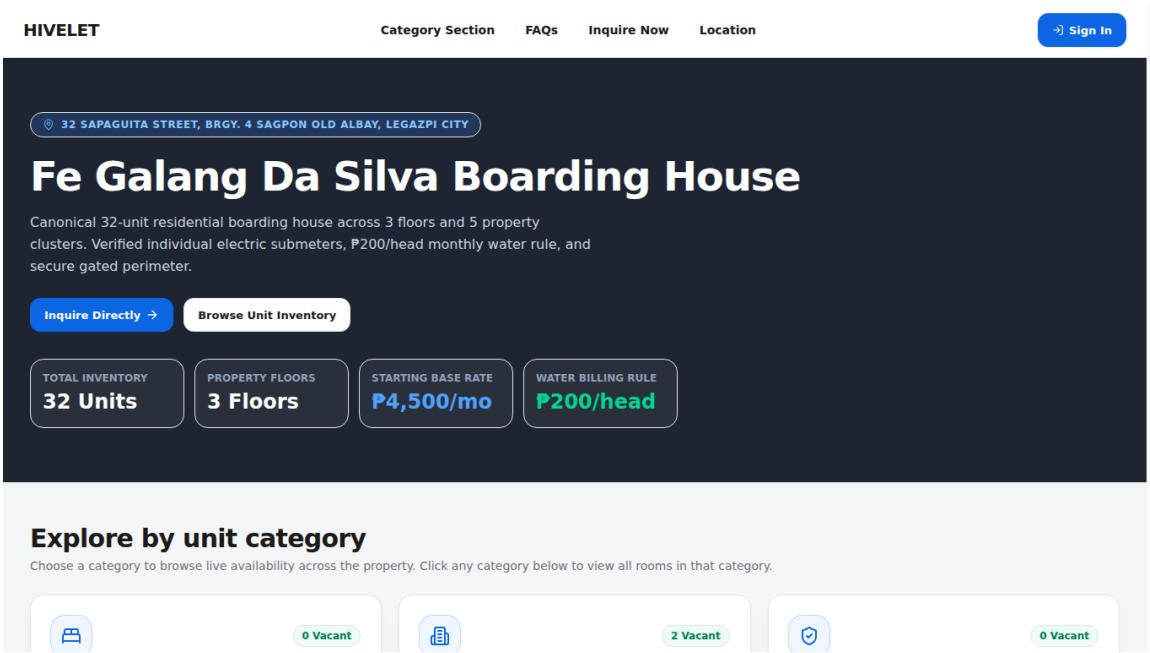


*Figure I-2. A unit category on the public site, 28 August 2026.*

[IMAGE FILE: images/Figure I-2 - A unit category on the public site, 28 August 2026.png]

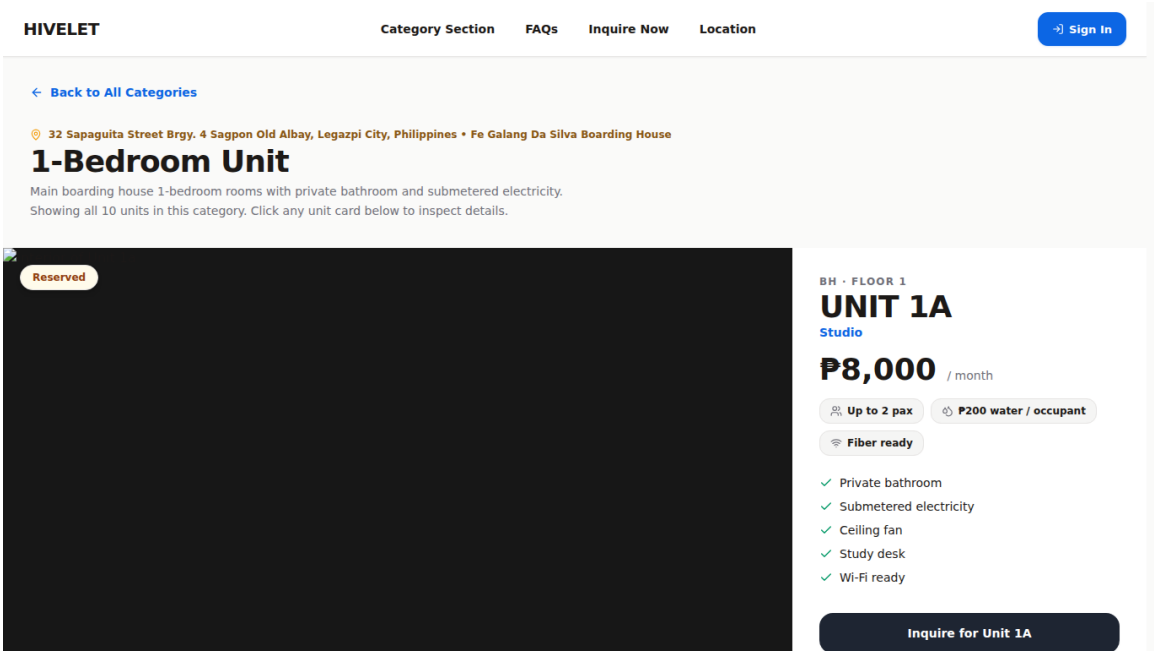



*Figure I-3. Tenant directory, with sample tenants, 28 August 2026.*

[IMAGE FILE: images/Figure I-3 - Tenant directory, with sample tenants, 28 August 2026.png]

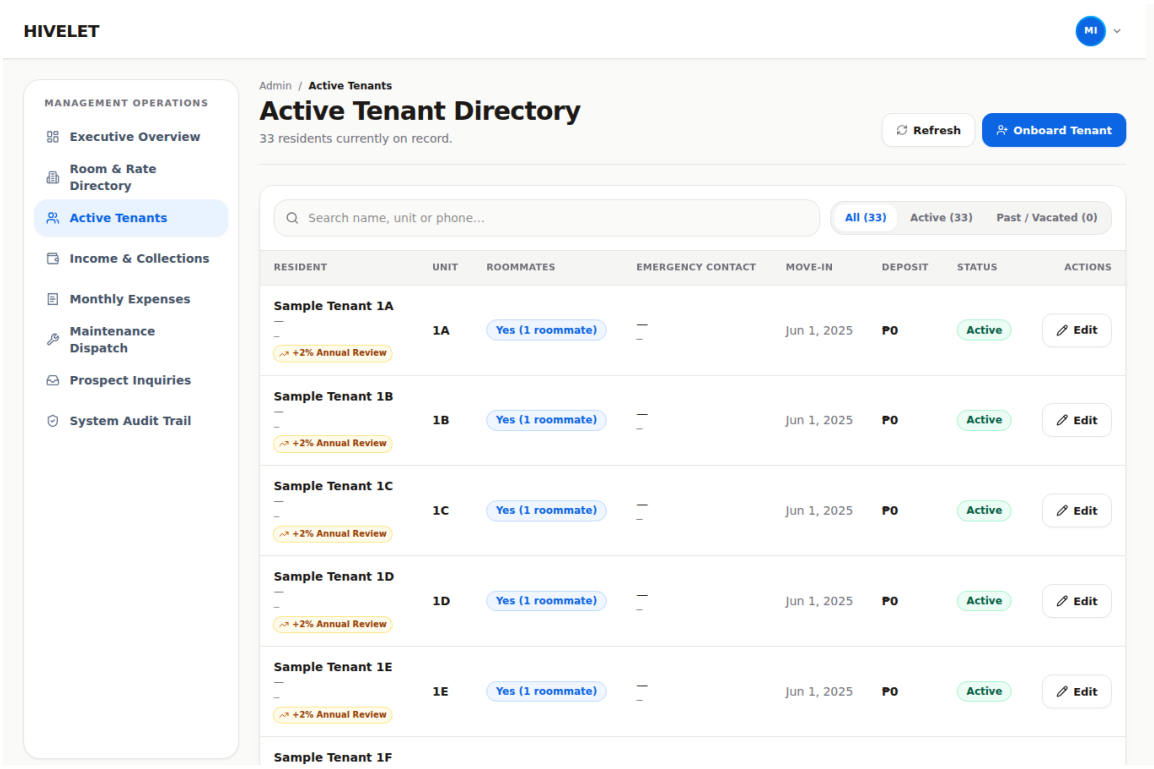
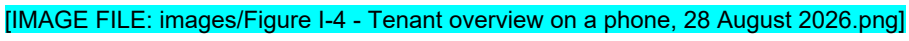
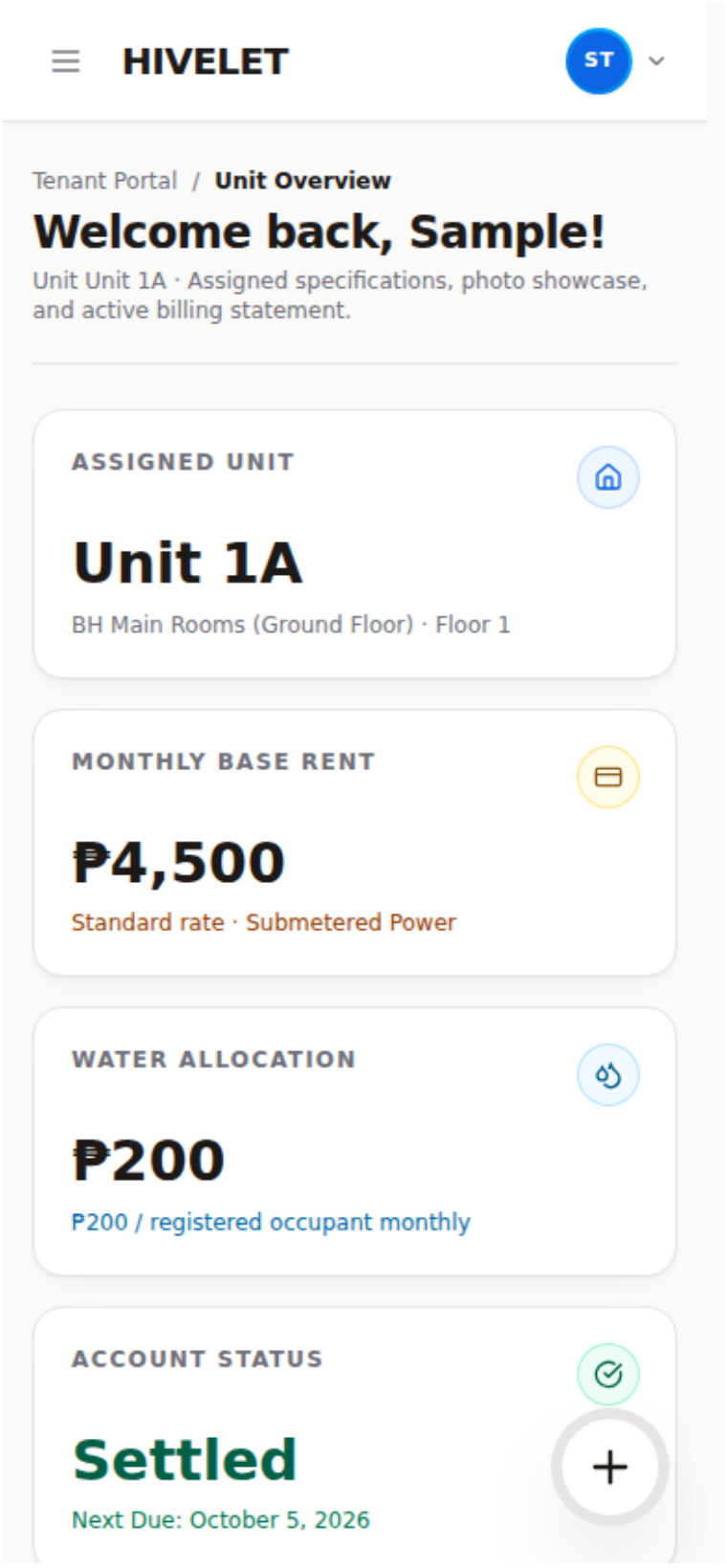


*Figure I-4. Tenant overview on a phone, 28 August 2026.*




HIVELET
ST
Tenant Portal / Unit Overview
Welcome back, Sample!
Unit Unit 1A · Assigned specifications, photo showcase, and active billing statement.
ASSIGNED UNIT
Unit 1A
BH Main Rooms (Ground Floor) · Floor 1
MONTHLY BASE RENT
₱4,500
Standard rate · Submetered Power
WATER ALLOCATION
₱200
P200 / registered occupant monthly
ACCOUNT STATUS
Settled
Next Due: October 5, 2026

# APPENDIX J: EVALUATION INSTRUMENT

The ISO/IEC 25010 questionnaire that respondents answered in a Google Form, from 30 September to 18 October 2026 and again in the second round, from 31 October to 10 November 2026, printed from the live form as it stood on 13 November 2026. The questions are worded exactly as in the responses exported from it in both rounds. The form opens with a short description and the data privacy notice through which the prospective tenants and the technical evaluators gave their consent (Section 3.4), then asks for an optional name. Each respondent answered Section 1, which sent them to the one section for their group. The item numbers are those of Chapter 3, Table 3-6. Section 2, answered by the landlady, has 28 rated items, Section 3 for tenants 18, Section 4 for technical evaluators 33, and Section 5 for prospective tenants 12. Every rated item and every open comment had to be answered; the consent question and the name did not.

**Hivelet System Evaluation — ISO/IEC 25010 Software Quality Assessment**

Thank you for helping us evaluate Hivelet, a web-based apartment management system developed for the Fe Galang Da Silva Boarding House by Group 4, BS Information Technology, Bicol University College of Science.

This survey measures the software quality of the system using the ISO/IEC 25010 international standard. It takes approximately 5 to 10 minutes. Please answer based on your actual experience using the system. Your responses are anonymous and will be used solely for this academic capstone research.

**Data Privacy Notice and Consent**

In compliance with the Data Privacy Act of 2012 (Republic Act No. 10173), all data collected in this survey will be kept strictly confidential.

Your responses will be used exclusively for the academic evaluation of the Hivelet system by our Group. No personally identifiable information will be published or shared outside the research team.

By continuing, you confirm that you have read this notice and voluntarily agree to participate in this evaluation.

- Yes, I agree
- No, I do not agree

**Section 1: ABOUT YOU**

**Full Name (optional)**

**Which best describes you? / Alin ang naglalarawan sa inyo?**

- Property owner or administrator / May-ari o administrador ng apartment
- Resident of the boarding house / Nangungupahan sa boarding house
- Technical evaluator (IT professional, developer, or IT faculty) / Technical evaluator
- Looking for a room (not living here yet)

**Which device did you mainly use? / Anong device ang pangunahin ninyong ginamit?**

- Desktop or laptop computer
- Mobile phone
- Tablet
- Both computer and mobile phone

**Section 2: OWNER/LANDLADY**

For all questions, Rate each from 1-5

- 5 — Lubos na Sumasang-ayon (Strongly Agree)
- 4 — Sumasang-ayon (Agree)
- 3 — Walang Kinikilingan (Neutral)
- 2 — Hindi Sumasang-ayon (Disagree)
- 1 — Lubos na Hindi Sumasang-ayon (Strongly Disagree)

| No. | Statement | 5 | 4 | 3 | 2 | 1 |
|---|---|---|---|---|---|---|
| | **Functional Suitability** | | | | | |
| 1 | The system lets me manage tenant records, units, and the number of occupants in each unit. | | | | | |
| 2 | The system computes rent and water charges correctly, without me doing the arithmetic myself. | | | | | |
| 3 | Payments I receive in person are recorded accurately against the correct unit and month. | | | | | |
| 4 | Online payments made by tenants appear correctly for me to verify before they are settled. | | | | | |
| 5 | Maintenance requests can be submitted, followed, and closed within the system. | | | | | |
| 6 | The financial reports the system produces match the records I keep. | | | | | |
| | **Performance Efficiency** | | | | | |
| 7 | The system responds quickly when I move between screens. | | | | | |
| 8 | Financial reports are produced without a long wait. | | | | | |

| No. | Statement | 5 | 4 | 3 | 2 | 1 |
|---|---|---|---|---|---|---|
| 9 | The system stays responsive even when many records are shown at once. | | | | | |
| | **Compatibility** | | | | | |
| 10 | The system works correctly in the browser I normally use. | | | | | |
| 11 | Reports exported from the system open correctly in my spreadsheet program. | | | | | |
| 12 | I can use the system at the same time as the other applications on my device. | | | | | |
| | **Usability** | | | | | |
| 13 | I can tell what each screen is for without being taught. | | | | | |
| 14 | The words and labels used match the way I actually talk about my property. | | | | | |
| 15 | Before anything is changed or deleted, the system asks me to confirm and tells me what will happen. | | | | | |
| 16 | When something goes wrong, the message tells me what to do about it. | | | | | |
| 17 | I was able to learn the system without technical help. | | | | | |
| | **Reliability** | | | | | |
| 18 | The system is available whenever I need it during the day. | | | | | |
| 19 | Records I enter are still there when I come back to them. | | | | | |
| 20 | When information cannot be loaded, the system says so instead of showing a wrong amount. | | | | | |
| 21 | After an interruption, nothing I had already recorded was lost. | | | | | |

| No. | Statement | 5 | 4 | 3 | 2 | 1 |
|---|---|---|---|---|---|---|
| | **Security** | | | | | |
| 22 | Only I can reach the financial records. | | | | | |
| 23 | A tenant can see only their own information. | | | | | |
| 24 | I can see a record of the actions taken in the system and who took them. | | | | | |
| 25 | I can change my password myself when I need to. | | | | | |
| | **Portability** | | | | | |
| 26 | The system works on the devices I already own. | | | | | |
| 27 | I can install the system on my phone without going to an app store. | | | | | |
| 28 | I did not need to install any other software to use the system. | | | | | |

**Open comments**: What worked best for you in the system?

**Section 3: TENANTS**

For all questions, Rate each from 1-5

- 5 — Lubos na Sumasang-ayon (Strongly Agree)
- 4 — Sumasang-ayon (Agree)
- 3 — Walang Kinikilingan (Neutral)
- 2 — Hindi Sumasang-ayon (Disagree)
- 1 — Lubos na Hindi Sumasang-ayon (Strongly Disagree)

| No. | Statement | 5 | 4 | 3 | 2 | 1 |
|---|---|---|---|---|---|---|
| | **Functional Suitability** | | | | | |
| 1 | I can see my own unit details, my bill, and what I still owe. | | | | | |
| 2 | I can submit a maintenance request and follow what happens to it. | | | | | |
| 3 | I can see a record of the payments I have made. | | | | | |
| | **Usability** | | | | | |
| 4 | I can tell what each screen is for without being taught. | | | | | |
| 5 | The words used in the system are easy to understand. | | | | | |
| 6 | It is clear how much I owe and what the amount is made up of. | | | | | |
| 7 | When something goes wrong, the message tells me what to do about it. | | | | | |
| 8 | I was able to use the system without anyone explaining it to me. | | | | | |
| | **Performance Efficiency** | | | | | |
| 9 | The system opens quickly. | | | | | |
| 10 | The system responds without delay when I move between screens. | | | | | |
| | **Reliability** | | | | | |
| 11 | The system is available whenever I try to use it. | | | | | |
| 12 | The information shown to me is correct and up to date. | | | | | |
| 13 | When something cannot be loaded, the system says so instead of showing a wrong amount. | | | | | |

| No. | Statement | 5 | 4 | 3 | 2 | 1 |
|---|---|---|---|---|---|---|
| | **Compatibility** | | | | | |
| 14 | The system works correctly in the browser I normally use. | | | | | |
| 15 | I can use the system at the same time as my other apps. | | | | | |
| | **Portability** | | | | | |
| 16 | The system works on my own phone or computer. | | | | | |
| 17 | I can open the system on more than one device. | | | | | |
| 18 | I did not need to install anything extra to use it. | | | | | |

**Open comments**: What did you find most useful?

## Section 4: TECHNICAL EVALUATORS

For all questions, Rate each from 1-5

- 5 — Lubos na Sumasang-ayon (Strongly Agree)
- 4 — Sumasang-ayon (Agree)
- 3 — Walang Kinikilingan (Neutral)
- 2 — Hindi Sumasang-ayon (Disagree)
- 1 — Lubos na Hindi Sumasang-ayon (Strongly Disagree)

| No. | Statement | 5 | 4 | 3 | 2 | 1 |
|---|---|---|---|---|---|---|
| | **Functional Suitability** | | | | | |
| 1 | The system provides the functions required for tenant, unit, and occupancy management. | | | | | |

| No. | Statement | 5 | 4 | 3 | 2 | 1 |
|---|---|---|---|---|---|---|
| 2 | Rent and water charges are computed correctly and consistently. | | | | | |
| 3 | Payment recording and verification behave correctly for both in-person and online payments. | | | | | |
| 4 | The maintenance ticketing workflow supports submission, tracking, and closure. | | | | | |
| 5 | Generated reports agree with the records held in the database. | | | | | |
| | **Performance Efficiency** | | | | | |
| 6 | Response times are acceptable for the expected number of users and records. | | | | | |
| 7 | Report generation completes within a reasonable time. | | | | | |
| 8 | The system uses client and server resources efficiently. | | | | | |
| | **Compatibility** | | | | | |
| 9 | The system operates correctly across current mainstream browsers. | | | | | |
| 10 | Exported files conform to formats that other applications can read. | | | | | |
| 11 | The system coexists with other applications without interference. | | | | | |
| | **Usability** | | | | | |
| 12 | The interface is understandable without prior training. | | | | | |
| 13 | Terminology is consistent across the system and appropriate to the users. | | | | | |

| No. | Statement | 5 | 4 | 3 | 2 | 1 |
|---|---|---|---|---|---|---|
| 14 | Destructive actions require confirmation and state their consequence. | | | | | |
| 15 | Error messages are actionable rather than technical. | | | | | |
| 16 | The interface is operable for users with limited technical background. | | | | | |
| | **Reliability** | | | | | |
| 17 | The system handles failure of a request without presenting incorrect data. | | | | | |
| 18 | Recorded data is retained reliably. | | | | | |
| 19 | The system distinguishes clearly between "no data" and "data could not be loaded". | | | | | |
| 20 | The system recovers from interruption without data loss. | | | | | |
| | **Security** | | | | | |
| 21 | Access to functions and records is correctly restricted by role. | | | | | |
| 22 | A tenant account cannot reach administrative data. | | | | | |
| 23 | Administrative actions are recorded in an auditable trail. | | | | | |
| 24 | Credentials are handled and stored appropriately. | | | | | |
| 25 | Authentication and session handling follow accepted practice. | | | | | |
| | **Maintainability** | | | | | |
| 26 | The codebase is organized so that a change can be located and made confidently. | | | | | |

| No. | Statement | 5 | 4 | 3 | 2 | 1 |
|---|---|---|---|---|---|---|
| 27 | The system is documented sufficiently for another developer to maintain it. | | | | | |
| 28 | Changes to configurable values do not require code changes. | | | | | |
| 29 | Automated checks exist that would catch a regression. | | | | | |
| 30 | A change in one part of the system is unlikely to disturb unrelated parts. | | | | | |
| | **Portability** | | | | | |
| 31 | The system can be deployed to another environment without modification. | | | | | |
| 32 | The client installs on a mobile device without an application store. | | | | | |
| 33 | The system does not depend on software the target environment is unlikely to have. | | | | | |

**Open comments**: Which aspect of the system is strongest, from a technical standpoint?

**Section 5: LOOKING FOR A ROOM**

For all questions, Rate each from 1-5

- 5 — Lubos na Sumasang-ayon (Strongly Agree)
- 4 — Sumasang-ayon (Agree)
- 3 — Walang Kinikilingan (Neutral)
- 2 — Hindi Sumasang-ayon (Disagree)
- 1 — Lubos na Hindi Sumasang-ayon (Strongly Disagree)

| No. | Statement | 5 | 4 | 3 | 2 | 1 |
|---|---|---|---|---|---|---|
| | **Functional Suitability** | | | | | |
| 1 | I could find which kinds of units the boarding house has and how much they cost. | | | | | |
| 2 | I could see enough about a unit (its floor, how many people can stay, its floor plan) to decide whether to ask about it. | | | | | |
| 3 | I could send an inquiry about a unit without difficulty. | | | | | |
| | **Usability** | | | | | |
| 4 | I could tell what each part of the website is for without being taught. | | | | | |
| 5 | The words used on the website are easy to understand. | | | | | |
| 6 | When I made a mistake in the inquiry form, the message told me what to fix. | | | | | |
| | **Performance Efficiency** | | | | | |
| 7 | The website opens quickly. | | | | | |
| 8 | The pages respond without delay when I move around the website. | | | | | |
| | **Reliability** | | | | | |
| 9 | The information shown (units, rates, availability) looks correct and up to date. | | | | | |
| | **Compatibility** | | | | | |
| 10 | The website works correctly in the browser I normally use. | | | | | |
| | **Portability** | | | | | |
| 11 | The website works on my own phone. | | | | | |
| 12 | I did not need to install anything to use it. | | | | | |

**Open comments**: What did you find most useful on the website?

**Open comments**: What was difficult, confusing, or missing?

# APPENDIX K: USER'S MANUAL

**For the landlady and the tenants of the Fe Galang Da Silva Boarding House.** Every button and screen name below is the one the system showed on 10 November 2026.

Hivelet is at **https://hivelet.vercel.app**. It works in any current browser on a phone or a computer, and it can be installed on a phone's home screen (Part 1, section 6).

## Part 1. For tenants / Para sa mga nangungupahan

### 1. Signing in for the first time / Unang pag-sign in

1. Open **hivelet.vercel.app** and tap **Sign in**. *Buksan ang hivelet.vercel.app at pindutin ang Sign in.*
2. Type your **login ID** (for example HV-48213) and the **starting password** the landlady gave you. *I-type ang inyong login ID at ang starting password na ibinigay ng landlady.*
3. A window asks for **your own mobile number and your own password**. Type the starting password again, then your new one twice. It must have **at least 10 characters, a letter and a number**. *Ilagay ang sariling numero at password: hindi bababa sa 10 character, may letra at numero.*
4. Keep your password to yourself. The slip stops working once you have your own. *Huwag ibahagi ang password.*

**Forgot your password?** Ask the landlady. She can give you a new starting password from her screen. *Nakalimutan ang password? Lumapit sa landlady. Bibigyan niya kayo ng bagong starting password.*

**Five wrong passwords** lock the account for 15 minutes. Then it opens again by itself.

**2. Overview / Pangunahing pahina**

The greeting, with **Request a repair** at the top right (on a phone, the green button at the bottom right). Then:

- **Your unit**: its floor plan, its kind and floor, and the deposit. When the landlady has acted on one of your repairs, one line says so, such as **In progress** or **Done**. Tap it to open **Repairs**.
- **Amount due**, only while something is to pay: the amount, when it is due, the rent and water it is made of, and **Pay with GCash**.
- **Payments**: a payment that was not accepted, one still waiting for verification, and the payments **Recorded by the landlady**.
- Under a line: your **Monthly charge** (rent and water), the **Next rent** and when it is due (once you are settled), and the **Landlady**'s number to call or text.

The amount due counts only the payments the landlady has recorded, up to the date it shows ("Payments recorded up to…"). If you have paid and it is not there yet, it appears once she records your receipt. *Ang "Amount due" ay batay lang sa mga bayad na naitala na ng landlady. Makikita lang ito kapag may babayaran.*

**Your recent actions** lists the last three things you did here (a repair request sent, a GCash payment started, your details updated), newest first. On a computer it is in the menu on the left, under the sections, on every page, headed **Recently**. On a phone it is in the same place, in the menu that opens from the top left. Its lines cannot be clicked, and only you see them. *Ang huling tatlong ginawa ninyo dito.*

**3. Payments and billing / Bayad at singil**

- **The bill** to pay, named by its month (for example **"Bill to pay, rent for November"**): the rent, plus water at **₱200 per occupant per month**. *Tubig: ₱200 bawat nakatira bawat buwan.*
- The bill card shows only while something is to pay. Once you are settled the page is the **Payment record** alone. Only receipts the landlady has **verified** count, and a GCash payment you sent shows as **waiting** until she verifies it.
- **History** and **Record**, two tabs, History first. The history is every transaction, a GCash payment that was not accepted or is still waiting included, and a payment you sent reads **Sent** with its date. The record is one row per month: a receipt the landlady verified, or **Not recorded** for a month with no payment recorded yet. **"Not recorded"** does not mean you owe that month: the landlady may not have recorded your payment yet. Search it, or pick another year or order in the dropdowns above it (on a phone, tap **Filters** first). The list changes as soon as you pick.

- **The deposit** held for your tenancy is on your Overview, in the card for your unit. *Nasa Overview ang deposit, sa card ng inyong unit.*
- **Pay with GCash**: optional online payment, not live yet. **It works in Adyen's test environment and takes no real money. Pay the landlady in person.** *Sa ngayon, sa landlady pa rin po magbayad nang personal.*

**4. Repairs / Pagpapaayos**

1. Open **Repairs** (or press **Request a repair** on your Overview). Under **Report it**, fill in **What needs fixing**, choose the **Category** and **How urgent**, and add the **Details**. *Isulat kung ano ang sira at gaano kaapurahan.*
2. **Add a photo** if you can, as a **JPG or PNG** (a phone camera's photos are). It is made smaller before sending.
3. Press **Send request**. It goes straight to the landlady and appears in **Your requests**.
4. Tap a request to open it, then press **Progress and notes** to follow it through its three steps, **Submitted**, **In progress** and **Done**, and to **add a note**. You get a notification when it is done. The **All requests** dropdown above your requests shows only **Open** or **Done** ones (on a phone, under **Filters**). *Pindutin ang request, saka ang Progress and notes para makita kung nasaan na ito.*
5. **Changed your mind?** Tap the request and press **Cancel request** while it is still **Submitted**. You may give a reason. You can cancel one request an hour, and not once the landlady has sent someone or started on it. Send

her a note instead. A cancelled request stays in your list, marked **Cancelled**. *Puwedeng i-cancel habang hindi pa sinisimulan ng landlady.*

**5. My details / Aking detalye**

Keep your phone number and your **Emergency contact** (the person to call) up to date, then press **Save**. After your first sign-in you can sign in with your login ID or your phone number. Your name can only be changed by the landlady. A phone number is spaced as you type it (0917 123 4567). Both your number and the number of the person to call must be Philippine mobile numbers. If one is not, a note under the box says so, and nothing is saved until you correct it. **To change your password:** press **Change password** on My details, or tap your initials at the top right > **Change password**. *Para palitan ang password: Change password.*

**6. Install it on your phone / I-install sa cellphone**

Tap your initials at the top right > **Add to home screen**, and follow what it shows. Or do it from the browser: *Pindutin ang inyong initials sa itaas, saka ang Add to home screen.*

- **Android (Chrome):** menu ⋮ > **Install app** (or **Add to Home screen**).
- **iPhone (Safari): Share** > **Add to Home Screen**.

It then opens from its own icon. **Pull down from the top** of a page to reload it: the green hexagon turns until your figures are back, and the page stays where it is. *Hilahin pababa mula sa itaas para i-reload.*

**Without signal**, it still opens, with a thin bar at the top: **"No connection · Transactions unavailable"**. **Overview**, **Payments and billing**, **Repairs**, **My**

**details** and the bell show your rent, balance, payments, requests, details and notifications as last loaded. **Pay with GCash** is hidden and **Send request**, **Save** and marking notifications read wait until the signal returns. When it does, the pages reload by themselves. These essential figures, names and notifications are kept on your phone for your account only, and **signing out removes them**. If sending stalls on a weak signal, wait: after about 45 seconds it tells you whether to check before sending again. *Kapag walang signal, makikita pa rin ang huling na-load na upa, balanse, bayad, request at notification, pero hindi makakapagpadala o makakabayad.*

**Light or dark:** tap your initials at the top right > **Appearance** > **System**, **Light** or **Dark**. **System** follows your phone's own setting. *Piliin ang Light o Dark sa Appearance.*

### 7. Signing out / Pag-sign out

Tap your initials at the top right > **Sign out**. Always sign out on a phone that is not yours: it also removes the figures, names and notifications kept on the phone for viewing without signal.

## Part 2. For the landlady (administrator)

This part is for the landlady, who signs in as the administrator.

**Every list** (Rooms and rates, Tenants, Monthly Income, Monthly Expenses, Repairs, Inquiries) has the same bar at the top:

- **Search** narrows the list as you type.

- **The filters** are dropdowns beside the search, each saying what it shows, such as **All clusters**, **All months** or **All repairs**: cluster, month, year, status, kind, whichever that list has. Where a list can be drawn two ways, one picks **By cluster** or **As a list**. **Order** arranges the list: **By unit**, **By name (A to Z)**, **Newest first** or **Oldest first** (Repairs also has **Most urgent first**). The list changes the moment you pick. **On a phone**, tap **Filters** to open them under the bar, and tap it again to close them. The filters in use are named under the bar, and **Clear** removes them.
- **Show 8 more** (or however many are next) and **Show all** under a long list open more of it, and **Show fewer** closes it again.
- **Click or tap a row or card** to open it and edit it. There is no separate edit button.

**Also on every screen:**

- A short ping means something was saved, or someone else's change has arrived.
- Messages appear at the top centre of the screen. Tap one to close it.
- A window that slides up from the bottom closes when you tap the dimmed page outside it or swipe it down. Two never close that way, so nothing is lost: the one showing a new tenant's login ID and password, and the GCash payment.
- The phone's **Back** button goes back a page and closes any window that was open on it.

- In the installed app, **pull down from the top** of a page to reload it: the green hexagon turns until the figures are back, and grey outlines stand in for them meanwhile, so the page does not go blank.
- **The green hexagon** also turns in the middle of the screen the first time the site opens on a phone or computer, when it is opened again after half an hour or more away, and when the public pages or the sign-in page are reloaded. Reloading one of your own screens during a visit shows grey outlines of the page instead while the figures load.
- **Light or dark:** your initials > **Appearance** > **System**, **Light** or **Dark** (System follows the phone or computer). On the public pages it is the sun/moon button at the top of the page.
- The band at the foot of the public pages gives your number under **Contact** and the privacy policy and terms under **Policies**.

**Downloads.** Where the browser offers a **Save** window (Chrome or Edge on a computer), the message **"Report downloaded"** appears only once the file is saved. Cancelling the window says nothing and saves nothing. On a phone the message is **"Download started"**. The cluster of 22 main units is called **Boarding House** on every screen and in every downloaded workbook.

### 1. Signing in, your number and your password

Sign in with your phone number and your password. Your number is shown under your name when you tap your initials at the top right. To change either: your initials > **Change number / password**. Type your **current number and current password** first, then a new number, a new password, or both. If either current one

is wrong the window says so without saying which, and nothing changes. A new number is the one you sign in with from then on, and the one every page of the website gives for calling you. Keep your password to yourself.

**2. Overview**

The first screen shows, from the top:

- **Needs your attention**, in up to three parts: the payments waiting to be verified, the urgent repairs, and the inquiries waiting for an answer, three of each with a count of the rest. A part with nothing waiting is not shown, and with nothing waiting at all the card is not shown. Urgent repairs are listed only here.
- **Rent for** this month (for example **"Rent for November"**, the rent filed under November, whenever it was paid), with **Due this week** under it: each tenant whose rent falls due in the next seven days, with the period it pays for, such as "Unit 1A, Nov 7 to Dec 6". Three are shown. Tap one to open **Record payment** for that unit.
- **Occupancy**: the units occupied out of 33, with each cluster's count, such as 22/22.
- **Income and expenses**, the year month by month. Tap a month on the chart to see its **Income**, **Operating expenses** and **Net income** under the chart. The arrows beside **Income** and **Operating expenses** open Monthly Income and Monthly Expenses. Net income deducts the operating costs only: personal spending is listed on Monthly Expenses beside them, and is not deducted from rental income.

If a figure cannot be loaded, the tile shows **"—"**, never ₱0.00: refresh when the connection is back.

**Your recent actions** lists the last three things you did (a payment recorded, a GCash payment approved, a repair marked done), newest first. On a computer it is in the menu on the left, under the sections, on every page, headed **Recently**. On a phone it is in the same place, in the menu that opens from the top left. Its lines cannot be clicked, and it shows only your own actions.

- **Record payment**, **Record expense** and **Move someone in/out** are the green buttons at the top. **On a phone**, tap the green **+** at the bottom right to show them, and tap it again to hide them.
- **Another year:** tap the year beside the date (for example **2026 ˅**) and choose one, or **2026, this year** to come back. A past year shows its year only, not today's date, and can be read but not added to: the green buttons are for this year. (Last year's late December rent is recorded from Monthly Income, section 3.)

**The bell** at the top shows a **dot** when you have unread notifications. Open it to read them. The **filter icon** beside the title shows only **Unread**, **Payments**, **Repairs** or **Inquiries** (or **All**), and **X** closes. The list updates by itself. **Mark all read** is at the bottom. **The numbers beside Inquiries and Repairs** in the side menu count what you have not looked at yet (new inquiries, and urgent repairs). They clear once you open that page, and come back if something changes: a visitor writes back, or a repair is made more urgent. They are remembered on this device only, so another phone or computer shows its own count.

**3. Recording a payment (Monthly Income)**

1. **Monthly Income** > **Record payment**.
2. Choose the **Unit**, the **Mode of payment** (Cash, GCash or Bank transfer, and for GCash or a bank transfer, type **Their reference number**), the **Date paid**, and the period it covers: **Months covered** and **Covering from**. **Covering to** is worked out from them. **Covering from** starts on the day after that tenant's last recorded period, so payments entered oldest first need no change. Change it if this payment is for a different period.
3. **Occupants** starts at the number on record. Change it if more or fewer people live there now: **Water** is worked out from it (₱200 a person) and cannot be typed. **Rent** fills in from the unit's rate.
4. **Invoice or acknowledgement receipt (if any)**: type the number on the invoice you issued, or tap **ACK** in the box when you gave an acknowledgement receipt with no number. Leave it blank if there was neither. "4726" or "INV 4726" is saved as INV#4726.
5. Press **Record payment**, check the figures in **Record this payment?** and press **Record it**. Then check the summary that appears: the months, rent, water, total, and, if you changed the occupants, that the tenant's record now shows the new number (their water from then on). **See it in Monthly Income** opens that month with the payment marked.

The same invoice number cannot be recorded twice for the same unit and month.

**To correct a payment:** click or tap its row to open **Edit this payment**, change it, and press **Save changes**. **Water** is worked out from **Occupants** and cannot be typed. **To remove one** recorded by mistake: open it and press **Delete payment**. It leaves the records and totals at once, and the system's own record (section 9) keeps that it existed and who removed it.

**Online (GCash) payments** appear under **To verify**, beside **Records** (every payment recorded). When any are waiting, Monthly Income opens on **To verify**. Check the money arrived, then press **Verify payment**, or **Reject** to its left and then **Reject payment** to confirm. A payment is only counted once you verify it. **GCash works in Adyen's test environment and takes no real money. It is optional and not live, so reject these.**

**The month and the year. Rent for** at the top is **one month**, its total alone (for example "Rent for October": the rent filed under October, the month its period starts, not the month it was paid). Until something is recorded for this month it says so first, **Not recorded**, and gives the latest month recorded under it ("Latest recorded, October", with the year named only when it is not the one at the top). Beside it, **Income by cluster** is the **year**: the year's total and how it divides across the clusters. Tap the green **cluster** to change it to **month**, the year month by month. Monthly Expenses is laid out the same way ("Expenses for October", then **Expense by area**, or by **month** in red bars). The **All months** dropdown (under **Filters** on a phone) shows one month in Rent for. The year beside the page name (for example **2026** ˅) picks another year, or **All years** for everything since the records began. A past year can be read but not added to, so **Record payment**,

**To verify** and opening a row to change it are not offered there. The one exception is last year: it stays open until every unit let last December has its December rent recorded, and then it closes. The **Group by cluster** dropdown (the usual view) can instead read **Group by unit** or **Group by tenant (A to Z)**.

**Download** opens a small window that asks what the file should cover: **Month**, **Year** or **All**, then the year from a row of years and, for a month, the month from a grid of twelve. It opens on what the screen is showing, so pressing **Download** again gets that. The workbook is in your own layout (month blocks, cluster subtotals, Linda separate) and is named for what it holds: "Monthly Income March 2026" for one month, "Monthly Income 2025" for a year (this year's too), and for **All** the years it covers, such as "Monthly Income 2024-2026". In the workbook each cluster's subtotal is on a light-blue row and each **GRAND SUBTOTAL** is red with white letters, and an acknowledgement receipt shows as **ACK**. The Monthly Expenses workbook uses the same colors for each month's **TOTAL** and the **YEAR TOTAL**.

**4. Recording an expense (Monthly Expenses)**

1. **Monthly Expenses** > **Record expense**.
2. The **Date of the expense**, then for each expense **What for**, **What kind**, **Where** (the property area) and **How much**.
3. One expense for several areas: **Split across another area**. The parts must add up to the expense, and the total is worked out for you. **Choose an area** for each new part. Each area can be given once, so two amounts for the same area go in as one.

4. **Add another expense** records the next one without closing the form.
5. A kind of expense that is not in the list: open **What kind**, choose **Add a new kind…** at the bottom, type its name and press **Add kind**. It is added after your 13, numbered 11, 12 and so on, and chosen for that expense. It is also in the **All kinds** dropdown and in the workbook's category summary. Your 13 stay as they are. A kind you add cannot be renamed or removed, so check the spelling.

Editing (click or tap the row) and **Delete expense** work as for payments. Monthly Expenses also opens on **this year**, and its year is chosen the same way, beside the page name, with the same rule for past years. **Download** asks **Month**, **Year** or **All**, as for income, and names the file the same way ("Monthly Expenses March 2026").

**5. Tenants**

- **Move someone in**: full name, unit, deposit, move-in date, whether the unit is shared and by how many, and an emergency contact with their phone. Press **Move them in**. Their **login ID** and a **one-time password** are shown: write both down and give them to the tenant in person (the password is shown only once). At first sign-in they choose their own password and add their own phone number.
- **Click or tap a tenant's row** to open their record. **On record** shows their login ID, phone, deposit, move-in date and emergency contact. Below it you can change their **Unit**, their **Standing**, and **Sharing the unit** with **How many**

**roommates**, then press **Save changes**. The water charge follows the number of occupants.

- **Reset password** (in the same window): for a tenant who forgot theirs or lost the slip. Confirm, and their login ID and a new one-time password are shown. Their old password stops working and their phone is signed out. The login ID is also on their record, under **Login ID**.
- **Move them out** (in the same window): type the unit code to confirm, and add a **reason** if you like (kept with the move-out, and the tenant does not see it). The tenant loses access at once, the unit becomes free to let, and **their records stay**.
- **Who lived where before**: change the dropdown that reads **Today** to a year (and pick a month in the **All months** dropdown if you like). The list then shows who paid for each unit then, read from your receipts: the months they paid, or for one month what the receipt covered, when it was paid and its number. A name you wrote two ways in the same unit is shown once, with the other spelling under it, and a tenant who moved units says which other unit they rented. **Download** asks **Month**, **Year** or **All** and saves the list as a workbook, named like "Tenant History June 2025". **Clear** (under the search) goes back to your current tenants.

### 6. Rooms and rates

Every unit, by cluster or as one list (**By cluster** or **As a list**, under **Filters** on a phone). The **All units** and **All clusters** dropdowns narrow them by status or cluster. Click or tap a unit to change its rate, kind, status, whether it is shown **On**

**the website**, its description or its picture. A unit with no photograph shows its **Floor plan**, the same one visitors see on the public pages. **Replace photo** uploads one, and once saved the public page shows that photograph instead. Every rate change is kept in the unit's price history. The system never raises a rate by itself.

**7. Repairs**

Requests are in three columns, **To dispatch**, **In progress** and **Done**, each card marked by urgency. Click or tap a card to open it (**Manage this repair**) and **Send technician** or change its status. A repair in progress has **Mark resolved** at the bottom right of its card, and the tenant is notified when you press it. When a tenant adds a note to a request, it reaches you as a notification that quotes it. **Log a repair** records one you were told about in person, or one for an empty unit. **Delete repair** (in **Manage this repair**) removes a request for good after you confirm it. **Status** in the filters shows one stage only. A request the tenant cancelled (they can, until you send someone or start work) is marked **Cancelled by the tenant**, with their reason as a note if they gave one.

**8. Inquiries**

Messages from people asking about a unit. Type a reply and press the arrow (**Send reply**), **Close inquiry** when settled, or **Move them in** to start a tenancy with the details already filled in. A closed inquiry, or one whose person moved in, takes no more replies. **Delete** (only on those) asks **"Delete this inquiry?"** and then removes it and its conversation for good. It cannot be undone. If the person who sent it opens its page again, it tells them the inquiry was closed

and removed. The **All inquiries** dropdown (under **Filters** on a phone) shows only those **Waiting for an answer**, **Answered**, **Moved in** or **Closed**.

The person who asked reads your reply on their inquiry's own page, and can write back there: when they send an inquiry they are given a link and a reference code for it, with **Copy the code** (the code is shown beside your reply box), and their conversation has **Copy conversation link**. They reach it from **Your inquiries**, which shares one screen with **Sign in** and **Inquire**. Those pages are for visitors: anyone signed in is taken to their own portal instead. Their answers appear in the same conversation, and a notification tells you. No text message is sent, so call them if it is urgent. Inquiries from before 30 September 2026 have no such page: call or text those people as before.

**9. The record of who did what**

The system writes every payment, correction, tenant change, repair, sign-in and download to a permanent record: who did it, when, and what changed. Nobody can change or delete an entry. The record is **kept in the database and is not shown in the app**. Ask the team if you need to know who changed something.

**10. When the connection is weak or gone**

The system opens without a connection, with a thin bar at the top (**"No connection · Transactions unavailable"**), and shows what each screen last loaded on this device: the Overview's figures, the tenants, the income and expense records, repairs, inquiries and the notifications. Nothing can be recorded, moved

or marked read until the connection returns, so **Record payment**, **Record expense**, **Move someone in**, the buttons in **Manage this repair** and **Log a repair**, and **Send reply** and **Close inquiry** are greyed out ("Needs a connection"), and the Overview's actions are hidden. When it returns, every screen reloads by itself. An inquiry's conversation (the replies) is not saved for offline viewing: its first message is shown, with a note that the replies could not be loaded. These essential figures, names and notifications stay on your own device only, and **signing out removes them**. A screen you have not opened since signing in has nothing saved yet. If a save takes too long it stops after about 45 seconds and says **"we cannot tell whether it was saved"**: look at the list before recording it again, so it is not recorded twice.

### 11. Looking after the records

- The team takes a **backup** of every record before any change to the data, and whenever asked.
- Record receipts as you collect them: tenants see their balance from what is recorded.
- Five receipts from before the system carry slips from the old book (for example a year typed wrongly). They stay exactly as written, and they are not errors in the system.

### 12. Quick answers

| Question | Answer |
|---|---|
| A tenant says their balance is wrong | Check that their receipts since the date shown are recorded (Monthly Income, their unit) |
| A tenant cannot sign in | They sign in with the login ID on their record (Tenants, their row). Five wrong tries lock it for 15 minutes. If they forgot the password: **Reset password** |
| A figure shows "—" | It could not be loaded. Refresh when the connection is back |
| "Too many attempts from this connection" | Everyone on the house wifi shares one counter. Wait 15 minutes or use mobile data |
| A GCash payment appeared | GCash works in Adyen's test environment, takes no real money and is not live. Reject it under **To verify** |
| Something looks wrong | Take a screenshot and tell the team. Do not try to correct the numbers directly |

# APPENDIX L: TEST CASES

The test cases run on 30 September 2026 on the live system, with their results, as summarized in Chapter 4, Sections 4.3.4 to 4.3.7. Each case names the ISO/IEC 25010 characteristic it tests (FS Functional Suitability, PE Performance Efficiency, CO Compatibility, US Usability, RE Reliability, SE Security, MA Maintainability, PO Portability) and the requirement it traces to, worded in full in Appendix M, Table M-32 (FR codes) and Chapter 4, Section 4.1.2 (NFR codes). A result is Pass, Fail, Pass after fix, or NT (not tested). The prospective tenant's task cards (Part PR) were not recorded on the day and are not shown. After the redesign the 114 cases were run again, from 4 to 13 November 2026: 99 passed, the 10 timing cases were timed, 5 could not be run, and none failed. The last part of this appendix gives the counts by part, and how and when each was run. The second round of users, from 31 October to 10 November 2026, repeated the tenant tasks of Part T and the prospective tenants' tasks, and its results are in Appendix M, Table M-30.

**Part A. Landlady (administrator) (A-05 to A-32 are the 26-step walkthrough)**

| ID | ISO | Req | Do | Should see | Result |
|---|---|---|---|---|---|
| A-01 | US | FR-001 | Sign in with her own password | Lands on Overview | Pass |
| A-02 | FS | FR-019 | Read the Overview. Ask her: "is this month's collection what you expect?" | Money tiles show figures; occupancy 32 of 33 | Pass |
| A-03 | US | NFR-002 | Ask her to find, without help, where she would record a payment | Finds **Monthly Income** within about a minute | Pass |

| ID | ISO | Req | Do | Should see | Result |
|---|---|---|---|---|---|
| A-04 | FS | FR-027 | Open the notification bell | Notifications list opens; unread count matches | Pass |
| A-05 | FS | FR-003 | (Walkthrough 1) Open the public site signed out, browse units | 33 units, no tenant names | Pass |
| A-06 | SE | FR-003 | (2) Open LB and LF details | Rate, floor, cluster; **no tenant name** | Pass |
| A-07 | FS | FR-004 | (3) Send one inquiry from a unit page | "Thank you" confirmation | Pass |
| A-08 | US, SE | FR-001 | (4) Change Password, wrong current password first | "That is not your current password", still signed in | Pass |
| A-09 | SE | FR-001 | (5) Change it for real | "Password changed", still signed in | Pass |
| A-10 | SE | FR-001 | (6) Sign out, sign in with the new password | Works | Pass |
| A-11 | FS, SE | FR-007, FR-008 | (7) Rooms and rates, edit `PH` to ₱30,500 | Saved; history row written | NT |
| A-12 | FS | FR-009 | (8) Tenants: onboard "REHEARSAL Test" into `PH` | Created; `PH` Occupied; a one-time password is shown to hand over | Pass |
| A-13 | FS | FR-009 | (9) Onboard a second tenant with the same phone | Refused, names the reason | Pass |
| A-14 | FS | FR-033 | (10) Edit the rehearsal tenant: occupants to 2 | Saved | Pass |
| A-15 | FS | FR-001 | (11) Sign in as the rehearsal tenant (private window) | Forced to choose a password, then Overview shows `PH`, rate, occupants | Pass |
| A-16 | FS | FR-021, FR-022 | (12) As that tenant: repair request with a phone photo | Sent; if the photo is too big, it says so plainly | Pass |
| A-17 | FS | FR-026 | (13) Post a message on it | Appears in the thread | Pass |
| A-18 | FS | FR-010 | (14) My details: change the emergency contact | Saved | Pass |
| A-19 | FS | FR-015 | (15) Start a GCash payment; **do not pay** | Adyen box with a GCash button, inside the window | Pass |
| A-20 | FS | FR-011 | (15b) Pay this period when no bill exists | A bill is raised: rate + ₱200 per occupant | Pass |
| A-21 | FS | FR-027 | (16) Mark a notification read | Badge drops by one | Pass |

| ID | ISO | Req | Do | Should see | Result |
|---|---|---|---|---|---|
| A-22 | SE | FR-002 | (17) Change a ticket id in the address bar to someone else's | Not found (404), never "forbidden" | Pass |
| A-23 | FS | FR-014, FR-032, FR-034 | (18) Monthly Income: record a collection for `PH`, receipt `REHEARSAL-001`, type a wrong water figure first | Warns about water; saved; bill settled | Pass |
| A-24 | RE | FR-017 | (19) Record the same receipt again | Refused: already recorded | Pass |
| A-25 | RE, FS | FR-017, FR-029 | (19b) Open Monthly Income in a second tab. In the first, **Edit** `REHEARSAL-001` > **Delete payment**; then do the same in the second tab without refreshing it (the screen calls a void "Delete payment", and the row leaves the first tab's list) | First works; second refused: "That income record was already voided on …" | Pass |
| A-26 | FS | FR-005 | (20) Inquiries: reply to A-07's inquiry, close it | Message posted; Closed | Pass |
| A-27 | FS | FR-025, FR-027 | (21) Repairs: In Progress, Resolved, delete | Each saves; tenant gets "repair is done" | Pass |
| A-28 | FS | FR-018, FR-038, FR-039 | (22, 22b) Monthly Expenses: ₱100 REHEARSAL, edit, split 60/40, delete | Saves; total stays ₱100 when split | NT |
| A-29 | FS, CO | FR-044, FR-028 | (23) Download the Monthly Income and Monthly Expenses workbooks, open in Excel. They save as "Monthly Income September 2026 - MI092026.xlsx" and "Monthly Expenses September 2026 - ME092026.xlsx" | Open correctly; her layout | Pass |
| A-30 | RE | NFR-006 | (23b) With Overview open, disconnect the laptop's internet, reload | Money tiles show "—", never ₱0.00 | Fail |
| A-31 | FS | FR-009 | (24) Vacate the rehearsal tenant | `PH` Available; account inactive | Pass |
| A-32 | FS | FR-007 | (26) Set `PH` back to ₱30,000 | Saved | NT |
| A-33 | FS | FR-029, NFR-009 | Open **Activity** | Today's actions listed with who and when | Pass |

| ID | ISO | Req | Do | Should see | Result |
|---|---|---|---|---|---|
| A-34 | FS | FR-016 | Monthly Income: the payment verification queue | Opens; shows pending items or says there are none | Pass |
| A-35 | US | NFR-002 | Ask: "What would you use first tomorrow morning?" Write the answer | | Pass |
| A-36 | SE, FS | FR-009 | **Before A-31**, on the REHEARSAL tenant only: Tenants > Edit > Reset password > confirm; sign in as that tenant with the new password (private window) | Confirmation says their phone is signed out; the password is shown once; signing in forces a new password; the old one is refused; Activity shows the reset without the password | Pass |

**Part T. Tenant tasks (testers coded T1 to T3, with P done alone, PH done with help, F not done and NT not tried)**

Each task was read aloud in English and in Filipino, without saying where to tap. The tasks, and what the tester should have seen:

| ID | Task read to the tester | Should see |
|---|---|---|
| T-01 | "Sign in with the slip we gave you." | Asked to choose a new password straight away |
| T-02 | "Choose your own password." | The rules shown (10 or more characters, a letter, a number), a weak one refused with a reason, a good one landing on Overview |
| T-03 | "Which unit is yours and how much is the rent?" | Their correct unit and rate |
| T-04 | "Do you owe anything right now?" | The answer clear on screen, and matching what they believe |
| T-05 | "Find your payments this year." | Their receipts listed, and the tester asked whether the list is right |
| T-06 | "How is your bill made up?" | Rent and water (₱200 per occupant) shown |
| T-07 | "Find where to report something broken." | Finds Repairs |

| ID | Task read to the tester | Should see |
|---|---|---|
| T-08 | "Report it, and say how urgent it is." (a real problem, or a title starting TEST) | "Was sent to the landlady", and the request in their list |
| T-09 | "Add a photo of it." | The photo attached, even a full-size phone photo, which is made smaller before sending |
| T-10 | "Add a note to your request." | The note in the request's thread |
| T-11 | "What is its status now?" | The status shown: Submitted, In progress or Done in the list, and Resolved or Closed in the thread |
| T-12 | "Check your contact details and fix anything wrong." | Saved, or nothing to change |
| T-13 | "Do you have any notifications?" | The bell opens, and marking one read lowers the count |
| T-14 | "Open the GCash payment, but do not pay. Close it." | The GCash option appears, and closing it changes nothing |
| T-15 | "Type /admin at the end of the address." | Not let in, and sent back to their own pages |
| T-16 | "Sign out, then press Back." | No personal data shown after signing out |
| T-17 | "Sign in again with your new password." | It works, and the slip's password no longer does |
| T-18 | "Was anything confusing?" | Their own words, written down exactly (below) |

The results:

| ID | ISO | Req | T1 Time | T1 Result | T1 Errors | T2 Time | T2 Result | T2 Errors | T3 Time | T3 Result | T3 Errors |
|---|---|---|---|---|---|---|---|---|---|---|---|
| T-01 | SE, US | FR-001 | 7:28 – 7:31 | PH | 2 | 8:08 – 8:10 | PH | 1 | 8:12 (log) | P | 0 |
| T-02 | SE, US | FR-001 | 7:31 – 7:32 | P | 0 | 8:10 – 8:12 | P | 0 | 8:12 (log) | P | 0 |
| T-03 | FS | FR-003 | 7:32 – 7:33 | P | 1 | 8:12 – 8:14 | PH | 1 | | NT | |

| ID | ISO | Req | T1 Time | T1 Result | T1 Errors | T2 Time | T2 Result | T2 Errors | T3 Time | T3 Result | T3 Errors |
|---|---|---|---|---|---|---|---|---|---|---|---|
| T-04 | FS, US | FR-011, FR-013 | 7:34 – 7:36 | P | 0 | 8:14 – 8:15 | P | 0 | | NT | |
| T-05 | FS | FR-014 | 7:37 – 7:40 | P | 0 | 8:15 – 8:16 | P | 0 | | NT | |
| T-06 | US | FR-011 | 7:40 – 7:41 | P | 0 | 8:16 – 8:17 | P | 0 | | NT | |
| T-07 | US | NFR-002 | 7:32 – 7:35 | P | 0 | 8:13 – 8:15 | P | 0 | | NT | |
| T-08 | FS | FR-021, FR-023 | 7:34 (log) | P | 0 | 8:17 (log) | P | 0 | 8:15 (log) | P | 0 |
| T-09 | FS | FR-022 | 7:43 – 7:44 | P | 0 | 8:16 – 8:17 | P | 0 | | NT | |
| T-10 | FS | FR-026 | | NT | | 8:18 – 8:20 | P | 0 | | NT | |
| T-11 | FS | FR-024 | 7:45 – 7:46 | P | 0 | 8:18 – 8:20 | P | 0 | | NT | |
| T-12 | FS | FR-010 | 7:46 – 7:47 | P | 0 | 8:18 (log) | P | 0 | 8:20 (log) | P | 0 |
| T-13 | FS | FR-027 | 7:35 – 7:36 | P | 1 | 8:21 – 8:22 | P | 0 | | NT | |
| T-14 | FS | FR-015 | 7:35 (log) | P | 0 | 8:14 – 8:16 | P | 0 | | NT | |
| T-15 | SE | FR-002 | 7:48 – 7:49 | P | 0 | 8:22 – 8:23 | P | 0 | | NT | |
| T-16 | SE | FR-001 | 7:36 (log) | P | 0 | | NT | | | NT | |
| T-17 | SE | FR-001 | 7:36 – 7:38 | P | 0 | | NT | | | NT | |

| ID | ISO | Req | T1 Time | T1 Result | T1 Errors | T2 Time | T2 Result | T2 Errors | T3 Time | T3 Result | T3 Errors |
|---|---|---|---|---|---|---|---|---|---|---|---|
| T-18 | US | NFR-002 | 7:50 – 7:52 | — | 0 | 8:25 – 8:27 | — | 0 | not timed | — | 0 |

## Part P. Public site

| ID | ISO | Req | Do | Should see | Result |
|---|---|---|---|---|---|
| P-01 | FS | FR-003 | Open `hivelet.vercel.app` on a phone | Landing page, building photo, "Inquire now" | Pass |
| P-02 | FS | FR-003 | Browse the categories and units | 33 units in five clusters; only `PH` available | Pass |
| P-03 | SE | FR-003 | Open any occupied unit | No tenant name, phone or email anywhere | Pass |
| P-04 | US | FR-004 | Inquiry form: press Send with everything empty | Each missing field says what it needs | Pass |
| P-05 | US | FR-004 | Type a bad email and a short phone number | Plain messages under the fields; nothing sent | Pass |
| P-06 | FS | FR-004 | Send one real-looking inquiry (a team member's own details) | Confirmation with **Open your conversation**, **Copy the link** and a reference code; the landlady sees it in Inquiries (A-26) | Pass |
| P-06b | FS | FR-004 | After the landlady replies to P-06: open the link on the same phone, then on another phone open `/inquiry` and type the reference code and the phone number used | Her reply shows in both; write back once and it appears in her conversation, with a notification. A wrong code or wrong phone says only "No inquiry matches that" | Pass |
| P-07 | US | none | Open `/privacy` and `/terms` | Both readable on a phone | Pass |
| P-08 | RE | none | Open a made-up address, `/this-does-not-exist` | A friendly "not found" page with a way back | Pass |

## Part C. Simultaneous use

| ID | ISO | Req | Do (at the same moment) | Should see | Result |
|---|---|---|---|---|---|
| C-01 | PE, RE | NFR-004 | All tenants refresh Overview together | Every screen loads; note the slowest (seconds) | Pass |
| C-02 | SE | FR-002 | Everyone checks the unit and name on screen | Each sees **only their own** unit. Any other name: STOP (guide §9) | Pass |
| C-03 | FS, RE | FR-026 | Every tenant posts a note on their request within the same minute | Every note saved once, none lost, none doubled | NT |
| C-04 | FS | FR-025 | Landlady opens Repairs while C-03 happens, then refreshes | All new notes visible, each on the right request | NT |
| C-05 | FS | FR-027 | Landlady moves two requests to In Progress | Those two tenants get a notification; the others do not | NT |
| C-06 | RE | NFR-006 | Two tenants double-tap Send on a note | Only one copy saved (the button disables while sending) | NT |
| C-07 | PE | NFR-004 | Landlady opens Monthly Income for the whole year while tenants are active | Loads; note the time | Pass |
| C-08 | CO | NFR-005 | Landlady uses the laptop, tenants use phones, at once | No one's session affects another's | Pass |

## Part O. Offline, weak signal and installing

| ID | ISO | Req | Do | Should see | Andr oid | iPho ne |
|---|---|---|---|---|---|---|
| O-01 | PO | FR-030 | Android Chrome: menu ⋮ > **Install app** (or Add to Home screen) | Hivelet icon on the home screen | Pass | n/a |
| O-02 | PO | FR-030 | iPhone Safari: Share > **Add to Home Screen** | Hivelet icon on the home screen | n/a | Pass |
| O-03 | PO | FR-030 | Open it from the icon | Opens full screen, no address bar, green top bar | Pass | Pass |
| O-04 | RE | FR-030 | Sign in, open Overview, then switch on **airplane mode** | Banner: "No connection. You can read what is already loaded, but nothing can be saved or paid until it is back." | Pass | Pass |
| O-05 | RE | FR-030 | Still offline: move between the tenant pages | The app still opens each page, and says it cannot load data; **never shows ₱0.00 or an empty list as if true** | Pass | Pass |
| O-06 | RE | FR-021 | Still offline: try to send a note on a request | A message that it could not reach the server; the typed text is **kept** | Pass | Pass |
| O-07 | RE | FR-030 | Switch airplane mode off | "Back online" notice; send the note again: it works, once | Pass | Pass |
| O-08 | RE, SE | FR-030 | Close the app fully, airplane mode on, open it from the icon | App opens; the tenant is **not** signed out; banner shows | Pass | Pass |
| O-09 | SE | FR-030 | Sign out, airplane mode on, open the app | No personal data visible anywhere | Pass | Pass |
| O-10 | RE | FR-003 | Offline: open the public units page | Units shown from memory, or a notice that availability cannot be shown (never a made-up list) | Pass | Pass |
| O-11 | RE | NFR-006 | **Weak signal**: on the laptop, DevTools > Network > **Slow 3G**; open Repairs and send a note | It gets there, slowly; the button shows "Sending…" and cannot be pressed twice | Pass (lapto p) | n/a |

| ID | ISO | Req | Do | Should see | Andr oid | iPho ne |
|---|---|---|---|---|---|---|
| O-12 | RE | NFR-006 | Weak signal that stops answering (walk to a dead spot, or on the laptop DevTools > Network > throttling > **Add** a custom profile with **60000 ms** latency, select it, then send) | Within 45 seconds a message says it cannot tell whether it was saved and to **check before sending again** | Fail | Pass |

## Part S. Security

| ID | ISO | Req | Do | Should see | Result |
|---|---|---|---|---|---|
| S-01 | SE | NFR-003 | Mozilla HTTP Observatory, `hivelet.vercel.app` | Grade (write it). The 30 Sep 00:04 run gave **B, 75/100, 11 of 12**, its one failure the Content Security Policy being report-only; the policy was switched to enforcing later that morning, and a scan at about 10:07 gave **A+, 115/100, 12 of 12**. Write the grade you get and the time | Pass |
| S-02 | SE | NFR-003 | securityheaders.com | Grade. Expect **A+** (30 Sep 10:05) | Pass |
| S-03 | SE | NFR-003 | SSL Labs, `hivelet.vercel.app` | Grade. Expect **A+** (run 30 Sep 00:10; one of two addresses A) | Pass |
| S-04 | SE | FR-001 | Rehearsal tenant only: 5 wrong passwords | Locked for 15 minutes with a clear message. **Never on a real account** | Pass |
| S-05 | SE | FR-002 | Tenant opens `/admin` (T-15) | Refused | Pass |
| S-06 | SE | FR-002 | Signed out, open `/tenant` | Sent to sign-in | Pass |
| S-07 | SE | FR-001 | Tenant signs out; Back button (T-16) | No data | Pass |
| S-08 | SE | FR-001 | Slip password after the tenant changed it (T-17) | Refused | Pass |
| S-09 | SE | FR-029 | Activity shows today's password changes and admin actions, without any password in it | As described | Pass |
| S-10 | SE | FR-003 | Public pages and unit details: no tenant names (P-03) | As described | Pass |
| S-11 | SE | FR-002 | Foreign ticket id (A-22) | Not found | Pass |
| S-12 | SE | NFR-003 | Ask the landlady: "Who else knows your password?" | Only she does (after A-09) | Pass |

## Part CO. Browsers and devices

| ID | Device and browser | Sign-in | Tenant pages | Admin pages | Install |
|---|---|---|---|---|---|
| CO-01 | Android, Chrome | Pass | Pass | n/a | Pass |
| CO-02 | Android, Samsung Internet (if a tenant has it) | NT | NT | NT | NT |
| CO-03 | iPhone, Safari | Pass | Pass | n/a | Pass |
| CO-04 | Windows laptop, Chrome | Pass | Pass | Pass | Pass |
| CO-05 | Windows laptop, Edge | Pass | Pass | Pass | Pass |
| CO-06 | Any, Firefox | Pass | Pass | Pass | n/a |

## Part PF. Performance timings (seconds)

| ID | Screen | Table 4-10 row | How | Laptop (s) | Phone (s) |
|---|---|---|---|---|---|
| PF-01 | Public home, first load | Public home page, first load | Cache cleared | 1.85 | 2.42 |
| PF-02 | Public home, second load | (supporting) | Normal reload | 0.62 | 0.95 |
| PF-03 | Sign in to Overview | Sign-in to dashboard | Stopwatch from pressing Sign in to figures shown | 2.10 | 2.88 |
| PF-04 | Monthly Income, full year | Income ledger, one full year | Stopwatch to the last row visible | 1.45 | n/a |
| PF-05 | Monthly Income workbook export | Excel export of one year | Stopwatch from click to file saved | 2.30 | n/a |
| PF-06 | Tenant Overview, first load | Tenant portal, first load | Cache cleared | 2.15 | 3.05 |
| PF-07 | Tenant Payments | (supporting) | Stopwatch | 1.20 | 1.78 |
| PF-08 | Send a repair request | (supporting) | Stopwatch from Send to confirmation | 1.65 | 2.10 |
| PF-09 | PageSpeed Insights, mobile, `hivelet.vercel.app` | (supporting) | pagespeed.web.dev, note the four scores and LCP. Expect about **86 / 100 / 100 / 100, LCP about 3.5 s** (30 Sep 10:15, after the phone photos); a few points either way is the network | 88 / 100 / 100 / 100, LCP 3.4 s | 85 / 100 / 100 / 100, LCP 3.6 s |
| PF-10 | Lighthouse in DevTools on a signed-in page | (supporting) | DevTools > Lighthouse > Analyze, on the laptop. **Laptop column = Desktop mode; Phone column = Mobile mode** (Lighthouse's emulated phone, run on the laptop, not on a real phone) | 84 / 98 / 100 / 100, LCP 3.2 s | 82 / 96 / 100 / 100, LCP 3.7 s |

## Part N. Changes shipped on 30 September

| ID | ISO | Req | Do | Should see | Result |
|---|---|---|---|---|---|
| N-01 | FS, US | FR-011, FR-013 | On a tenant's own account (a testing tenant, with their permission, or during T-05): **Payments** > **Your rent, month by month** | A sentence "Paid up to …" with Up to date / Due soon / Overdue; a bar per month (Paid, Due, Nothing recorded, Not due yet); Months paid, Paid in these months, Due now; **Show each month as a list**. The months and amounts agree with the tenant's own receipts (ask them) | NT |
| N-02 | FS, RE | FR-003 | `/category/studio` (then one-bedroom, two-bedroom): tap units quickly one after another, across floors and on the same floor, then stop | The floor plan is always shown for the last unit tapped, never a blank box, without refreshing | Automated test, 30 Sep 14:36, headless Chrome on the live site, 390 px and 1280 px, all four categories (33 units): every unit tapped 60 ms apart, two units alternated 40 ms apart, first then last at once; plan visible and loaded after all 20 rounds (20/20). **Real phone: pending** |
| N-03 | FS, RE | FR-003 | Open `hivelet.vercel.app/public`; read the pop-up | "1 unit is vacant right now" (only `PH` is vacant), then "Viewings are by appointment…" and **Inquire now**. With no signal: the old text, never a number | Automated test, 30 Sep: the live pop-up said "1 unit is vacant right now", matching `/api/public/rooms` (32 Occupied, 1 Available, `PH`). **Real phone: pending** |

**The re-run after the redesign, 4 to 13 November 2026**

All 114 cases were taken up again on the redesigned system. 99 passed, the 10 timing cases were timed, 5 could not be run, and none failed.

| Part | Cases | Passed | Timed | Not run |
|---|---|---|---|---|
| A. Landlady | 36 | 36 | | |
| T. Tenant tasks | 18 | 18 | | |
| P. Public site | 9 | 9 | | |
| C. Simultaneous use | 8 | 4 | | 4 |
| O. Offline, weak signal and installing | 12 | 12 | | |
| S. Security | 12 | 12 | | |
| CO. Browsers and devices | 6 | 5 | | 1 |
| PF. Performance timings | 10 | | 10 | |
| N. Changes shipped on 30 September | 3 | 3 | | |
| **All** | **114** | **99** | **10** | **5** |

**4 November: 77 cases, all passed.** No one signed in. The cases were checked with the 21 automated check suites, requests the server refused, a copy of the screens with a simulated sign-in, read-only queries of the records, the code of each screen, and read-only requests to the live site. 21 of them passed with wording the redesign had changed.

**10 November: 16 cases, all passed,** during the second round, with the landlady and tenants present: A-01, A-03, A-10, A-35 and S-12 with the landlady on a laptop, T-18, C-08, O-01 to O-03 and N-02 with tenants on two iPhones and an Android phone, and CO-01 and CO-03 to CO-06 on those phones and on a Windows laptop in Chrome, Edge and Firefox. The landlady found Monthly Income in under 30 seconds (A-03). Asked what she would use first the next morning (A-35), she said she would first check for new repair requests and messages from the tenants, then who is due to pay rent. Asked whether anyone else knows her

password (S-12), she said no one does. Asked whether anything was confusing (T-18), the tenants said nothing was.

**13 November: 6 cases passed and 10 were timed.** The three outside security scans (S-01 to S-03) graded the site A+, with SSL Labs' second address at A as on 30 September. As the test tenant, the GCash window opened with Adyen's box and was closed without paying (A-19, T-14). With every save made to hang, the message that the system cannot tell whether a save went through appeared at 45.0 seconds (O-12). The laptop timings were taken with the browser's own timer: the public page 0.35 s (PF-02), sign-in to the Overview 2.11 s (PF-03), Monthly Income for a year 1.05 s (PF-04), the year's workbook 0.48 s (PF-05), the tenant's Overview 1.15 s (PF-06), the tenant's payments 0.41 s (PF-07) and sending a repair request 0.86 s (PF-08). PageSpeed Insights (PF-09) gave the public page 80, 100, 100 and 100 on a phone and 99, 100, 100 and 100 on a desktop. The first load with an empty cache (PF-01) was timed in a new Chrome each time: on the laptop's connection that morning, which took 2.4 to 4.9 seconds just to deliver the first byte of the page, its largest element was drawn at a median of 6.7 seconds (0.84 to 11.46 s, ten runs). That figure describes the connection more than the system: PageSpeed Insights' controlled first load drew it in 0.8 seconds on a desktop and 4.4 seconds on a phone on slow 4G. Lighthouse 13.6, run on the landlady's Overview after a team member signed in (PF-10), gave 87, 100, 100 and 66 in desktop mode, with the largest element drawn at 0.8 seconds, and 71, 100, 100 and 66 in mobile mode, which slows the processor four times and the connection to slow 4G. Search engine optimization is 66 because the

signed-in pages tell search engines not to list them, on purpose, and that is the one check that failed. In mobile mode the page kept the processor busy for 1.8 seconds, which lowered its score. Lighthouse put the first and largest paint at 0.0 seconds in all three mobile runs, which means it timed the loading screen, so those two figures are not used.

**Not run (5):** C-01, C-03, C-04 and C-07, which need several tenants acting at the same moment, for lack of time in the session; and CO-02, since no Samsung phone was present. The phone column of the timings was not taken again. What the four simultaneous-use cases test was tested automatically instead: twelve simulated visitors and six simulated readers at once (Appendix M, Table M-25), simultaneous saves (Table M-26), and the concurrency check suite, which passed again on 13 November.

# APPENDIX M: SUPPORTING EVIDENCE FOR CHAPTER 4

This appendix holds the detailed results that Chapter 4 summarizes: the full functional walkthrough, the rating of every survey item by every group of the second round (the users and the technical evaluators who rated the redesigned system, as in Chapter 4's Table 4-17, with the first round summarized in Tables M-28 and M-29), the measured evidence for each quality characteristic, every change made in response to the evaluation, and the grading of the 44 functional requirements (Table M-32).

**Table M-1.** Results of the functional walkthrough (30 September 2026)

(summarized in Chapter 4, Section 4.3.4)

| Step | Function tested | Expected result | Actual result | Pass or fail |
|---|---|---|---|---|
| 1 | Public unit catalogue browsing | 33 units listed without tenant names or sensitive details | As expected. | Pass |
| 2 | Public unit detail inspection | Correct rate, floor, and cluster; tenant names omitted (LB and LF verified) | As expected. | Pass |
| 3 | Public inquiry submission | Confirmation message displayed; rate limiter guards abuse | As expected. | Pass |
| 4 | Authentication negative test | Wrong current password rejected with inline message; session stays active | As expected. | Pass |
| 5 | Administrator password rotation | Password updated successfully; active session maintained | As expected. | Pass |
| 6 | Re-authentication with new credential | Successful sign-in using updated administrator password | As expected. | Pass |

| Step | Function tested | Expected result | Actual result | Pass or fail |
|---|---|---|---|---|
| 7 | Unit rate update and audit trigger | Rate updated; database trigger writes immutable price history row | Not performed: the system's activity record shows no rate change that day, and the unit kept its ₱30,000 rate. | Not done |
| 8 | Tenant onboarding to vacant unit (PH) | Tenancy created; unit status transitions to Occupied | As expected. | Pass |
| 9 | Duplicate credential guard | Duplicate phone number registration rejected with validation message | As expected. | Pass |
| 10 | Tenancy details modification | Occupant count updated and persisted in database | As expected. | Pass |
| 11 | Tenant portal authentication | Tenant dashboard displays correct unit details, rate, and occupancy | Signed in after one mistyped password; the system required a new password, then showed the unit, rate and occupants. | Pass |
| 12 | Repair request with a photo | Request submitted with photo; size guard informs user if file is too large | As expected. | Pass |
| 13 | Repair request conversation | Follow-up message posted and displayed within the request conversation | As expected. | Pass |
| 14 | Tenant emergency contact update | Updated emergency contact saved and reflected in profile | As expected. | Pass |
| 15 | Online payment checkout initialization | Adyen Web Drop-in mounts inside modal and displays GCash button | The payment window opened with the GCash button; the attempt was not paid and was rejected, so nothing reached the ledger. | Pass |
| 15b | On-demand billing generation | Bill generated on demand matching unit rate plus ₱200/occupant water | A bill of ₱30,200 was raised (₱30,000 rent and ₱200 water for one occupant); removed after the test. | Pass |
| 16 | Notification state management | Notification marked as read; unread badge counter decrements | As expected. | Pass |

| Step | Function tested | Expected result | Actual result | Pass or fail |
|---|---|---|---|---|
| 17 | Cross-tenant authorization perimeter | Accessing another tenant's repair request ID returns HTTP 404 (Not Found), not 403 | As expected. | Pass |
| 18 | On-site cash collection recording | Collection recorded against invoice REHEARSAL-001; bill marked settled | The collection of ₱30,200 was recorded and the bill settled. Water is worked out from the number of occupants and cannot be typed, so a wrong water amount could not be entered. | Pass |
| 19 | Duplicate invoice prevention | Re-submitting an identical invoice number is blocked with an explicit alert | As expected. | Pass |
| 19b | Receipt voiding and double-void guard | First void reverses ledger entry; second void attempt is rejected | The first void worked. The second window refreshed itself and no longer showed the receipt, so a second void could not be attempted; the refusal message was therefore not seen. | Pass |
| 20 | Inquiry processing and resolution | Administrator reply sent; inquiry status marked Closed | As expected. | Pass |
| 21 | Repair resolution and notification | Request advanced to Resolved; tenant receives resolution notification | Moved to In Progress, then Resolved; the tenant was notified that the repair was done. | Pass |
| 22 | Expense recording and allocation | ₱100 expense logged with property allocation; edits/deletion verified | Not performed as written: two separate expenses (₱60 and ₱40) were recorded instead of one ₱100 expense split 60/40, and neither was edited or deleted. | Not done |
| 22b | Expense allocation split derivation | Expense split across areas; total derived automatically via trigger | Not performed as written: two separate expenses (₱60 and ₱40) were recorded instead of one ₱100 expense split 60/40, and neither was edited or deleted. | Not done |
| 23 | Financial workbook export | income.xlsx and expenses.xlsx downloaded matching the landlady's layout | As expected. | Pass |

| Step | Function tested | Expected result | Actual result | Pass or fail |
|---|---|---|---|---|
| 23b | Fail-safe presentation on API interruption | Backend stopped; money tiles display "—" instead of misleading ₱0.00 | With the connection cut, the money tiles showed ₱0 and "Try again" instead of "—". | Fail |
| 24 | Tenant vacating and tenancy termination | Tenancy ended; unit returns to Available; end_date recorded | The test tenant was moved out and the unit showed Available. | Pass |
| 25 | Ledger baseline verification | Rehearsal records removed; ledger restored to exact 937 baseline rows | Done by the team after the session rather than on screen: the test tenant's bill, payments, voided receipt and repair were removed; the two test expenses of step 22 were voided on 1 October. | Not done |
| 26 | Unit rate restoration | PH rate restored to confirmed ₱30,000 rate card baseline | Not needed, because step 7 was not performed; the rate reads ₱30,000. | Not done |

**Table M-2.** Evaluation results for Functional Suitability, by item and group

(summarized in Chapter 4, Section 4.4.3, Table 4-17)

| Indicator | Rated by | Mean | SD | Interpretation |
|---|---|---|---|---|
| The system lets me manage tenant records, units, and the number of occupants in each unit. | Landlady | 5.00 | | Very High Quality |
| The system computes rent and water charges correctly, without me doing the arithmetic myself. | Landlady | 5.00 | | Very High Quality |
| Payments I receive in person are recorded accurately against the correct unit and month. | Landlady | 5.00 | | Very High Quality |
| Online payments made by tenants appear correctly for me to verify before they are settled. | Landlady | 5.00 | | Very High Quality |
| Maintenance requests can be submitted, followed, and closed within the system. | Landlady | 5.00 | | Very High Quality |
| The financial reports the system produces match the records I keep. | Landlady | 5.00 | | Very High Quality |

| Indicator | Rated by | Mean | SD | Interpretation |
|---|---|---|---|---|
| **Landlady mean** | | **5.00** | | Very High Quality |
| I can see my own unit details, my bill, and what I still owe. | Tenants | 4.80 | 0.42 | Very High Quality |
| I can submit a maintenance request and follow what happens to it. | Tenants | 4.70 | 0.67 | Very High Quality |
| I can see a record of the payments I have made. | Tenants | 4.70 | 0.48 | Very High Quality |
| **Tenant mean** | | **4.73** | | Very High Quality |
| The system provides the functions required for tenant, unit, and occupancy management. | Technical | 5.00 | 0.00 | Very High Quality |
| Rent and water charges are computed correctly and consistently. | Technical | 5.00 | 0.00 | Very High Quality |
| Payment recording and verification behave correctly for both in-person and online payments. | Technical | 4.67 | 0.58 | Very High Quality |
| The maintenance ticketing workflow supports submission, tracking, and closure. | Technical | 5.00 | 0.00 | Very High Quality |
| Generated reports agree with the records held in the database. | Technical | 5.00 | 0.00 | Very High Quality |
| **Technical evaluator mean** | | **4.93** | | Very High Quality |
| I could find which kinds of units the boarding house has and how much they cost. | Prospects | 4.67 | 0.58 | Very High Quality |
| I could see enough about a unit (its floor, how many people can stay, its floor plan) to decide whether to ask about it. | Prospects | 4.67 | 0.58 | Very High Quality |
| I could send an inquiry about a unit without difficulty. | Prospects | 4.67 | 0.58 | Very High Quality |
| **Prospective tenant mean** | | **4.67** | | Very High Quality |
| **Users' composite (landlady, tenants, prospective tenants)** | | **4.80** | | **Very High Quality** |

**Table M-3.** Measured evidence for Functional Suitability (summarized in Chapter 4, Section 4.4.5, Table 4-18)

| Sub-characteristic | Measure | Result | Source |
|---|---|---|---|
| Functional completeness | Features named in Objective 2 that were implemented, X = A / B | 6 / 6 = 1.00 | Sections 4.2.2 to 4.2.7 |
| Functional completeness | Functional requirements implemented in full or in part, X = A / B | 42 / 44 = 0.95 (33 as worded), graded against the final system on 13 November 2026 (Table M-32). The other 2 reach the landlady in another form | Section 4.1.2 |
| Functional correctness | Report reconciliation checks that agreed with the database, X = A / B | 490 / 490 = 1.00 | Table 4-8 |
| Functional correctness | Executed walkthrough steps that gave the expected result, X = A / B | 24 / 25 = 0.96 | Table M-1 |
| Functional correctness | Screens whose figures matched the database, after correction | 9 / 9, after 6 defects were corrected | Table 4-9 |
| Functional appropriateness | Tenant tasks completed, X = A / B | 34 / 34 = 1.00 | Table 4-11 |

**Table M-4.** Evaluation results for Performance Efficiency, by item and group

(summarized in Chapter 4, Section 4.4.3, Table 4-17)

| Indicator | Rated by | Mean | SD | Interpretation |
|---|---|---|---|---|
| The system responds quickly when I move between screens. | Landlady | 5.00 | | Very High Quality |
| Financial reports are produced without a long wait. | Landlady | 5.00 | | Very High Quality |
| The system stays responsive even when many records are shown at once. | Landlady | 5.00 | | Very High Quality |
| **Landlady mean** | | **5.00** | | Very High Quality |
| The system opens quickly. | Tenants | 4.90 | 0.32 | Very High Quality |
| The system responds without delay when I move between screens. | Tenants | 4.90 | 0.32 | Very High Quality |
| **Tenant mean** | | **4.90** | | Very High Quality |
| Response times are acceptable for the expected number of users and records. | Technical | 5.00 | 0.00 | Very High Quality |
| Report generation completes within a reasonable time. | Technical | 5.00 | 0.00 | Very High Quality |
| The system uses client and server resources efficiently. | Technical | 4.67 | 0.58 | Very High Quality |
| **Technical evaluator mean** | | **4.89** | | Very High Quality |
| The website opens quickly. | Prospects | 4.67 | 0.58 | Very High Quality |
| The pages respond without delay when I move around the website. | Prospects | 4.67 | 0.58 | Very High Quality |
| **Prospective tenant mean** | | **4.67** | | Very High Quality |
| **Users' composite (landlady, tenants, prospective tenants)** | | **4.86** | | **Very High Quality** |

**Table M-5.** Measured evidence for Performance Efficiency (summarized in Chapter 4, Section 4.4.5, Table 4-18)

| Sub-characteristic | Measure | Result | Source |
|---|---|---|---|
| Time behavior | Longest time until a screen was usable, real devices | 3.05 s on a phone (tenant portal, first load); 2.30 s on a laptop (a year exported to Excel) | Table 4-10 |
| Time behavior | Time within which 95 percent of requests were answered under load | 0.52 s (public pages); 0.80 s (the landlady's data) | Table M-25 |
| Resource utilization | Code a visitor's browser downloads for the payments page | About 20 kB; the 194 kB payment library only when a payment starts | Table 4-19 |
| Resource utilization | Script work on an emulated mid-range phone, the landlady's Overview | 0.7 s | Section 4.3.5 |
| Capacity | Failed requests with 12 simulated simultaneous users, X = A / B | 0 / 1,085 = 0.00 | Table M-25 |
| Capacity | Failed requests with 6 simulated simultaneous readers of the landlady's records, X = A / B | 0 / 285 = 0.00 | Table M-25 |

**Table M-6.** Evaluation results for Compatibility, by item and group (summarized in Chapter 4, Section 4.4.3, Table 4-17)

| Indicator | Rated by | Mean | SD | Interpretation |
|---|---|---|---|---|
| The system works correctly in the browser I normally use. | Landlady | 5.00 | | Very High Quality |
| Reports exported from the system open correctly in my spreadsheet program. | Landlady | 5.00 | | Very High Quality |
| I can use the system at the same time as the other applications on my device. | Landlady | 5.00 | | Very High Quality |
| **Landlady mean** | | **5.00** | | Very High Quality |
| The system works correctly in the browser I normally use. | Tenants | 4.70 | 0.48 | Very High Quality |
| I can use the system at the same time as my other apps. | Tenants | 4.70 | 0.67 | Very High Quality |
| **Tenant mean** | | **4.70** | | Very High Quality |
| The system operates correctly across current mainstream browsers. | Technical | 5.00 | 0.00 | Very High Quality |
| Exported files conform to formats that other applications can read. | Technical | 5.00 | 0.00 | Very High Quality |
| The system coexists with other applications without interference. | Technical | 5.00 | 0.00 | Very High Quality |
| **Technical evaluator mean** | | **5.00** | | Very High Quality |
| The website works correctly in the browser I normally use. | Prospects | 4.67 | 0.58 | Very High Quality |
| **Prospective tenant mean** | | **4.67** | | Very High Quality |
| **Users' composite (landlady, tenants, prospective tenants)** | | **4.79** | | **Very High Quality** |

**Table M-7.** Measured evidence for Compatibility (summarized in Chapter 4, Section 4.4.5, Table 4-18)

| Sub-characteristic | Measure | Result | Source |
|---|---|---|---|
| Co-existence | Not measured by a test; rated in the survey only | | |
| Interoperability | Device and browser combinations on which sign-in, the pages and installation worked, X = A / B | 5 / 5 = 1.00 (Android Chrome 154, iPhone Safari, Windows Chrome 154, Edge and Firefox 157); Samsung Internet not tried | Compatibility matrix, 30 September 2026 |
| Interoperability | Payment gateway message checks passed, X = A / B | 73 / 73 = 1.00 | Table 4-8 |
| Interoperability | Exported workbooks opened in a spreadsheet program in the landlady's layout | Opened (walkthrough step 23); 490 / 490 month-by-month checks against the database | Tables 4-8 and M-1 |

**Table M-8.** Evaluation results for Usability, by item and group (summarized in Chapter 4, Section 4.4.3, Table 4-17)

| Indicator | Rated by | Mean | SD | Interpretation |
|---|---|---|---|---|
| I can tell what each screen is for without being taught. | Landlady | 5.00 | | Very High Quality |
| The words and labels used match the way I actually talk about my property. | Landlady | 5.00 | | Very High Quality |
| Before anything is changed or deleted, the system asks me to confirm and tells me what will happen. | Landlady | 5.00 | | Very High Quality |
| When something goes wrong, the message tells me what to do about it. | Landlady | 5.00 | | Very High Quality |
| I was able to learn the system without technical help. | Landlady | 5.00 | | Very High Quality |
| **Landlady mean** | | **5.00** | | Very High Quality |

| Indicator | Rated by | Mean | SD | Interpretation |
|---|---|---|---|---|
| I can tell what each screen is for without being taught. | Tenants | 4.70 | 0.67 | Very High Quality |
| The words used in the system are easy to understand. | Tenants | 4.90 | 0.32 | Very High Quality |
| It is clear how much I owe and what the amount is made up of. | Tenants | 4.70 | 0.48 | Very High Quality |
| When something goes wrong, the message tells me what to do about it. | Tenants | 4.80 | 0.42 | Very High Quality |
| I was able to use the system without anyone explaining it to me. | Tenants | 4.60 | 0.70 | Very High Quality |
| **Tenant mean** | | **4.74** | | Very High Quality |
| The interface is understandable without prior training. | Technical | 4.67 | 0.58 | Very High Quality |
| Terminology is consistent across the system and appropriate to the users. | Technical | 5.00 | 0.00 | Very High Quality |
| Destructive actions require confirmation and state their consequence. | Technical | 4.67 | 0.58 | Very High Quality |
| Error messages are actionable rather than technical. | Technical | 5.00 | 0.00 | Very High Quality |
| The interface is operable for users with limited technical background. | Technical | 5.00 | 0.00 | Very High Quality |
| **Technical evaluator mean** | | **4.87** | | Very High Quality |
| I could tell what each part of the website is for without being taught. | Prospects | 4.67 | 0.58 | Very High Quality |
| The words used on the website are easy to understand. | Prospects | 4.67 | 0.58 | Very High Quality |
| When I made a mistake in the inquiry form, the message told me what to fix. | Prospects | 4.67 | 0.58 | Very High Quality |
| **Prospective tenant mean** | | **4.67** | | Very High Quality |
| **Users' composite (landlady, tenants, prospective tenants)** | | **4.80** | | **Very High Quality** |

**Table M-9.** Measured evidence for Usability (summarized in Chapter 4, Section 4.4.5, Table 4-18)

| Sub-characteristic | Measure | Result | Source |
|---|---|---|---|
| Appropriateness recognizability | Tenant tasks completed without help, X = A / B | 31 / 34 = 0.91 | Table 4-11 |
| Learnability | Tasks that needed help after the first sign-in and finding the unit | 0; all 3 tasks that needed help were among those first two | Table 4-11 |
| Operability | Mean wrong turns per task; median time per task | 0.2; 60 s | Table 4-11 |
| User error protection | Wrong inputs the system refused when tried, X = A / B | 3 / 3 (a duplicate phone number, a duplicate invoice number, a typed water amount); the double-void guard could not be reached from the screen | Table M-1, steps 9, 18, 19 and 19b |
| User interface aesthetics | Not measured and not rated | | |
| Accessibility | Automated scans with a WCAG 2.2 level A or AA violation, X = A / B | 0 / 76 after the correction of 5 October 2026; Lighthouse accessibility score 100 | Sections 4.3.5 and 4.4.6 |

**Table M-10.** Evaluation results for Reliability, by item and group (summarized in Chapter 4, Section 4.4.3, Table 4-17)

| Indicator | Rated by | Mean | SD | Interpretation |
|---|---|---|---|---|
| The system is available whenever I need it during the day. | Landlady | 5.00 | | Very High Quality |
| Records I enter are still there when I come back to them. | Landlady | 5.00 | | Very High Quality |
| When information cannot be loaded, the system says so instead of showing a wrong amount. | Landlady | 5.00 | | Very High Quality |
| After an interruption, nothing I had already recorded was lost. | Landlady | 5.00 | | Very High Quality |

| Indicator | Rated by | Mean | SD | Interpretation |
|---|---|---|---|---|
| **Landlady mean** | | **5.00** | | Very High Quality |
| The system is available whenever I try to use it. | Tenants | 4.80 | 0.42 | Very High Quality |
| The information shown to me is correct and up to date. | Tenants | 4.50 | 0.97 | Very High Quality |
| When something cannot be loaded, the system says so instead of showing a wrong amount. | Tenants | 4.70 | 0.48 | Very High Quality |
| **Tenant mean** | | **4.67** | | Very High Quality |
| The system handles failure of a request without presenting incorrect data. | Technical | 5.00 | 0.00 | Very High Quality |
| Recorded data is retained reliably. | Technical | 4.67 | 0.58 | Very High Quality |
| The system distinguishes clearly between "no data" and "data could not be loaded". | Technical | 4.67 | 0.58 | Very High Quality |
| The system recovers from interruption without data loss. | Technical | 5.00 | 0.00 | Very High Quality |
| **Technical evaluator mean** | | **4.83** | | Very High Quality |
| The information shown (units, rates, availability) looks correct and up to date. | Prospects | 4.67 | 0.58 | Very High Quality |
| **Prospective tenant mean** | | **4.67** | | Very High Quality |
| **Users' composite (landlady, tenants, prospective tenants)** | | **4.78** | | **Very High Quality** |

**Table M-11.** Measured evidence for Reliability (summarized in Chapter 4, Section 4.4.5, Table 4-18)

| Sub-characteristic | Measure | Result | Source |
|---|---|---|---|
| Maturity | Critical defects among those found in testing, X = A / B | 0 / 35; 32 resolved, 3 accepted | Table 4-20 |
| Availability | Not measured as a share of time over a period; rated in the survey only | | |
| Fault tolerance | Addresses that opened with no connection, X = A / B | 10 / 10 = 1.00 | Table M-24 |
| Fault tolerance | A figure that could not be loaded shown as unavailable, not as ₱0 | Failed once on the testing day (step 23b); passed when re-tested on the landlady's laptop, 13 October 2026 | Tables M-1 and 4-20 |
| Fault tolerance | A stalled save reports within the 45 seconds designed | iPhone within 45 s; Android after 60 to 80 s, corrected and on time when re-tested, 13 October 2026 | Section 4.3.7 |
| Recoverability | Ledger after a receipt was recorded and voided | Returned to its 937 rows and its previous total | Section 4.3.2 |
| Recoverability | Typed text kept when a send failed | Kept on both phones; sent once the connection returned on one | Section 4.3.7 |

**Table M-12.** Evaluation results for Security, by item and group (summarized in Chapter 4, Section 4.4.3, Table 4-17)

| Indicator | Rated by | Mean | SD | Interpretation |
|---|---|---|---|---|
| Only I can reach the financial records. | Landlady | 5.00 | | Very High Quality |
| A tenant can see only their own information. | Landlady | 5.00 | | Very High Quality |
| I can see a record of the actions taken in the system and who took them. | Landlady | 5.00 | | Very High Quality |
| I can change my password myself when I need to. | Landlady | 5.00 | | Very High Quality |
| **Landlady mean** | | **5.00** | | Very High Quality |
| Access to functions and records is correctly restricted by role. | Technical | 5.00 | 0.00 | Very High Quality |
| A tenant account cannot reach administrative data. | Technical | 5.00 | 0.00 | Very High Quality |
| Administrative actions are recorded in an auditable trail. | Technical | 4.67 | 0.58 | Very High Quality |
| Credentials are handled and stored appropriately. | Technical | 5.00 | 0.00 | Very High Quality |
| Authentication and session handling follow accepted practice. | Technical | 5.00 | 0.00 | Very High Quality |
| **Technical evaluator mean** | | **4.93** | | Very High Quality |
| **Users' composite (landlady, tenants, prospective tenants)** | | **5.00** | | **Very High Quality** |

**Table M-13.** Measured evidence for Security (summarized in Chapter 4, Section 4.4.5, Table 4-18)

| Sub-characteristic | Measure | Result | Source |
|---|---|---|---|
| Confidentiality | Another tenant's record requested by its number | Answered "not found" (walkthrough step 17); each tenant saw only their own unit (C-02) | Tables M-1 and M-22 |
| Confidentiality | Protections a user meets that held when tried by hand, X = A / B | 12 / 12 = 1.00 | Section 4.4.4 |
| Confidentiality | Tables with row-level security and no rule opening them to a browser | Every table; no warning from the database's security advisor | Sections 4.2.6 and 4.4.4 |
| Integrity | Payment gateway messages accepted only with a valid signature: checks passed, X = A / B | 73 / 73 = 1.00 | Table 4-8 |
| Integrity | Requests that change data checked against a schema before use | Every one, in all 71 request handlers read | Section 4.4.4 |
| Non-repudiation, accountability | Administrator actions written to an audit trail the application cannot edit or delete | Every administrator action | Section 4.2.6 |
| Authenticity | Account after five wrong passwords | Locked for 15 minutes | Section 4.4.4 |
| Authenticity | Grades from three passive external scans | SSL Labs A+; Mozilla HTTP Observatory A+ (115, 12 of 12 tests); securityheaders.com A+ | Section 4.4.4 |
| All | High-severity problems found by the code review | 0 | Section 4.4.4 |

**Table M-14.** Evaluation results for Maintainability, by item and group

(summarized in Chapter 4, Section 4.4.3, Table 4-17)

| Indicator | Rated by | Mean | SD | Interpretation |
|---|---|---|---|---|
| The codebase is organized so that a change can be located and made confidently. | Technical | 5.00 | 0.00 | Very High Quality |
| The system is documented sufficiently for another developer to maintain it. | Technical | 5.00 | 0.00 | Very High Quality |
| Changes to configurable values do not require code changes. | Technical | 4.33 | 1.15 | Very High Quality |
| Automated checks exist that would catch a regression. | Technical | 4.33 | 0.58 | Very High Quality |
| A change in one part of the system is unlikely to disturb unrelated parts. | Technical | 5.00 | 0.00 | Very High Quality |
| **Technical evaluator mean** | | **4.73** | | Very High Quality |

**Table M-15.** Measured evidence for Maintainability (summarized in Chapter 4, Section 4.4.5, Table 4-18)

| Sub-characteristic | Measure | Result | Source |
|---|---|---|---|
| Modularity | Routes whose access comes from the one permission matrix, checked on every run, X = A / B | 60 / 60 = 1.00 | Section 4.2.6 |
| Modularity | Database calls made through service modules rather than inside the request handlers, X = A / B | 50 / 204 = 0.25; the administrator's handlers are one file of 5,120 lines | The server's code, counted 7 October 2026 |
| Reusability | Not measured and not rated | | |
| Analysability | Changes to the database kept as numbered, reviewable migrations | Every change; more than 70 by 5 October 2026 | Section 4.2.1 |
| Modifiability | The water rate changed without a change to the code | Changed in the settings | Section 4.2.4 |
| Testability | Automated suites that run against the live system without writing to it, passing, X = A / B | 20 / 20 = 1.00 | Table 4-8 |

**Table M-16.** Evaluation results for Portability, by item and group (summarized in Chapter 4, Section 4.4.3, Table 4-17)

| Indicator | Rated by | Mean | SD | Interpretation |
|---|---|---|---|---|
| The system works on the devices I already own. | Landlady | 5.00 | | Very High Quality |
| I can install the system on my phone without going to an app store. | Landlady | 5.00 | | Very High Quality |
| I did not need to install any other software to use the system. | Landlady | 5.00 | | Very High Quality |
| **Landlady mean** | | **5.00** | | Very High Quality |
| The system works on my own phone or computer. | Tenants | 4.90 | 0.32 | Very High Quality |
| I can open the system on more than one device. | Tenants | 4.70 | 0.67 | Very High Quality |

| Indicator | Rated by | Mean | SD | Interpretation |
|---|---|---|---|---|
| I did not need to install anything extra to use it. | Tenants | 4.90 | 0.32 | Very High Quality |
| **Tenant mean** | | **4.83** | | Very High Quality |
| The system can be deployed to another environment without modification. | Technical | 5.00 | 0.00 | Very High Quality |
| The client installs on a mobile device without an application store. | Technical | 5.00 | 0.00 | Very High Quality |
| The system does not depend on software the target environment is unlikely to have. | Technical | 5.00 | 0.00 | Very High Quality |
| **Technical evaluator mean** | | **5.00** | | Very High Quality |
| The website works on my own phone. | Prospects | 4.67 | 0.58 | Very High Quality |
| I did not need to install anything to use it. | Prospects | 4.67 | 0.58 | Very High Quality |
| **Prospective tenant mean** | | **4.67** | | Very High Quality |
| **Users' composite (landlady, tenants, prospective tenants)** | | **4.83** | | **Very High Quality** |

**Table M-17.** Measured evidence for Portability (summarized in Chapter 4, Section 4.4.5, Table 4-18)

| Sub-characteristic | Measure | Result | Source |
|---|---|---|---|
| Adaptability | Screens wider than the window at 360, 390, 768 and 1,366 pixels | 0 (seven public pages and the landlady's eight screens) | Section 4.3.6 |
| Adaptability | Kinds of device on which the same code ran | 3: a laptop in Chrome, an Android phone in Chrome, an iPhone in Safari | Section 4.2.7 |
| Installability | Installation requirements of a Progressive Web Application | Met; installed from the browser on an Android phone and an iPhone, with no app store | Table M-24 and Section 4.3.7 |
| Replaceability | The landlady's workbook reproduced by the system | Monthly Income and Monthly Expenses, exported in her layout and checked against the database month by month (490 / 490); she has not yet made it her main record | Section 4.2.4, Tables 4-8 and 4-22 |

**Table M-18.** Every optimization made in response to evaluation, with its finding (summarized in Chapter 4, Section 4.4.6, Table 4-19)

| Source | Finding | Change made | Characteristic |
|---|---|---|---|
| Client review on a phone, 23 Sep 2026 | Edit buttons appeared only on mouse hover, so they did not exist on a touch screen | Buttons show on touch screens and stay hover-only on computers. Since 5 November 2026 a row or card opens when tapped or clicked, with no separate edit button | Usability |
| Client review on a phone, 23 Sep 2026 | The tenant register was wider than the phone screen | Tables become cards on small screens | Usability, Portability |
| Client review on a phone, 23 Sep 2026 | The payment form's buttons were hidden below the screen | Form height limited on phones so the buttons stay visible | Usability |
| Client review on a phone, 23 Sep 2026 | Forms with typed input had no close button | Every form can be closed; only the required first password change cannot | Usability |

| Source | Finding | Change made | Characteristic |
|---|---|---|---|
| Performance review, 26 Sep 2026 | The payments page loaded the whole payment gateway library for every visitor | The library (about 194 kB) now loads only when a tenant starts an online payment; the page's own code is about 20 kB, measured on the live site on 28 Sep 2026 | Performance Efficiency |
| Final review, 26 Sep 2026 | A refused refund could be announced as successful; a receipt could be credited to the wrong tenant | Both corrected, with checks added | Reliability, Functional Suitability |
| Screen audit, 26 Sep 2026 | A tenant's payment history showed no payments | Receipts from the ledger now shown with online payments | Functional Suitability |
| Screen audit, 26 Sep 2026 | Rental and personal expenses shared one column | Separate columns, as in the landlady's workbook | Usability, Functional Suitability |
| Screen audit, 26 Sep 2026 | A placeholder date read as every tenant's move-in | Shown as not recorded until the real dates are entered | Reliability |
| Screen audit, 26 Sep 2026 | An opened GCash checkout was listed as a recorded payment | Listed as "GCash payment started" | Security (accountability of the audit trail) |
| Screen audit, 26 Sep 2026 | A voided receipt was listed to the tenant as a real payment | Voided receipts left out of the tenant's list | Reliability |
| Screen audit, 26 Sep 2026 | The Boarding House's remitted amount was halved on the income screen | Corrected to full rent plus water, read from the landlady's workbook formula | Functional Suitability |
| Screen audit, 26 Sep 2026 | The payments list would stop silently at 1,000 records | Read in batches, like the income list | Reliability |
| Requirements review, 26 Sep 2026 | The landlady could not record a repair she was told about in person, or one for an empty unit; a tenant was never told a repair was done | "Log a repair" added for any unit; the tenant is notified when the repair is marked done | Functional Suitability |
| Team review, 28 Sep 2026 | The two ledger pages were named "Money coming in" and "Money going out", which are not the landlady's words | Renamed "Monthly Income" and "Monthly Expenses", the names of the two sheets in her workbook | Usability |

| Source | Finding | Change made | Characteristic |
|---|---|---|---|
| Team review, 26 Sep 2026 | The highlighted option in every dropdown was cut off at the sides | Outline drawn inside the option | Usability |
| Client review, 26 Sep 2026 | Tenants who had moved out still appeared in the tenant list, and every current tenant carried a "Living here" label | The list opens on current tenants only; moved-out tenants are shown only when chosen, and only unusual states carry a label | Usability |
| Client review, 26 Sep 2026 | The same page had different names in the menu, the browser tab and its own heading | Each page has one name, used in all three places | Usability |
| Client review, 26 Sep 2026 | A year chosen on one page did not carry to the others | One year choice now applies to the overview and both ledgers | Usability |
| Mobile review, 26 Sep 2026 | On phones, every tap flashed a grey box, short pages scrolled slightly, and the notification panel could end under the browser's toolbar | Tap highlight removed; pages and the panel sized to the visible screen height | Usability, Portability |
| Check review, 28 Sep 2026 | Two checks disagreed on how many ended tenancies had no end date (2 and 1): the tenant list never sent end dates, and one real record had none | The list now sends end dates; the record was given its date (migration 059); both checks now fail on any new case | Reliability |
| Team review, 28 Sep 2026 | The public FAQ offered online GCash payment while the gateway works only in Adyen's test environment | The FAQ now says online payments charge no real money yet and asks tenants to pay in person | Functional Suitability |
| Pre-testing audit, 29 Sep 2026 | Every tenant account shared one password that the development team had used, so a tenant given it could have signed in as a neighbor | Each tenant given their own starting password, handed over in person, with a new password required at first sign-in; every earlier session ended | Security |

| Source | Finding | Change made | Characteristic |
|---|---|---|---|
| Lighthouse audit, 30 Sep 2026 | The public page jumped while it loaded: the footer showed on an empty page and was pushed down when the page arrived (layout shift 0.50; desktop performance score 76) | The page's space is held until it loads; layout shift 0, desktop performance score 95, measured on the live site | Performance Efficiency, Usability |
| Pre-testing audit, 29 Sep 2026 | A tenant who forgot their password had no way back in, and the landlady no way to help | The landlady can reset a tenant's password from the tenant list; the tenant gets a one-time password and chooses their own at next sign-in | Usability, Security |
| Pre-testing audit, 29 Sep 2026 | The window that makes a tenant set their own password did not say which password it wanted, and had no way out | It asks for the starting password by name and offers Sign out | Usability |
| Pre-testing audit, 29 Sep 2026 | On a weak signal a page or a save could wait for minutes with no message | Pages stop after 25 seconds and saves after 45, each with a message; a save that may have arrived asks the reader to check before sending it again | Reliability, Usability |
| Landlady's use, 30 Sep 2026 | A payment the landlady recorded, or a repair a tenant sent, did not appear on the other person's open screen until they refreshed it | Every open page checks for changes while it is on screen, every 5 seconds at first and every 2 seconds since 1 October, and reloads only what changed | Usability, Reliability |
| Walkthrough, 30 Sep 2026 (step 23b) | With the connection cut, the Overview's money tiles showed ₱0 instead of "—" | Not reproduced on 1 Oct 2026: on the current version, reloading the Overview with no connection shows "could not be loaded" on every tile and no ₱0. Re-tested on the landlady's laptop on 13 October 2026: passed | Reliability |
| Testing day on an Android phone, 30 Sep 2026 | On a connection that stops answering, the "cannot tell whether it was saved" message came after 60 to 80 seconds instead of 45 | Chrome slows the timers of a page that is not on screen, which a phone test takes the user out of; the deadline is now also checked against the clock the moment the page is on screen again, so a late message appears at once. Re-tested on the same Android phone on 13 October 2026: on time | Reliability |

| Source | Finding | Change made | Characteristic |
|---|---|---|---|
| Team check of the evaluation account, 2 Oct 2026 | The tenant account prepared for the evaluators showed ₱30,400 owed, but paying online answered that there was nothing to pay: receipts voided on the testing day had left a payment counted against the bill | Online payment now skips a bill that is already covered and says when a receipt is missing; the account's bills were brought into line with its receipts (migration 077), with no other tenant's records changed | Functional Suitability, Reliability |
| Security review, 2 Oct 2026 | A request sent from another website received a server error (500) instead of a refusal | Refused with 403 before any route runs | Security |
| Security review, 7 Oct 2026 | A new password that met the length and character rules could still be a well-known breached one ("Password123" appears 1,505,362 times in Pwned Passwords) | A new password is checked against Pwned Passwords before it is saved; only five characters of its hash leave the server, and if the service cannot be reached the rules alone apply | Security |
| Technical evaluators, 3 Oct 2026 | On a phone, the light and dark mode switch sat at the foot of the footer, below every policy link | Moved to the top bar of every public page | Usability |
| Technical evaluators, 3 Oct 2026 | An email address was checked only when the form was sent | Checked as soon as the visitor leaves the field | Usability |
| Technical evaluators, 3 Oct 2026 | Links were underlined throughout the site | Underlines kept only in the footer; other links are shown in bold | Usability |
| Technical evaluators, 3 Oct 2026 | "Send one", the way to start an inquiry from the inquiry lookup page, did not stand out | Shown in bold | Usability |
| Technical evaluators, 3 Oct 2026 | Phone numbers were typed as one run of digits, and the inquiry form accepted any seven characters | Numbers are spaced as they are typed (0917 123 4567); an inquiry, and an emergency contact where one is given, must give a Philippine mobile number, checked on the page and again by the server | Usability, Functional Suitability |
| Technical evaluators, 3 Oct 2026 | Filters opened in a pop-up window and needed an Apply button | Each filter is a dropdown on the page that applies the moment it is chosen; on a phone the dropdowns open under the Filters button instead of in a window | Usability |

| Source | Finding | Change made | Characteristic |
|---|---|---|---|
| Technical evaluators, 3 Oct 2026 | After a session ran out, signing in again did not return to the page that had been open | Signing in returns each person to the page they last had open | Usability |
| Technical evaluators, 3 Oct 2026 | Photos of any image format could be attached | JPG and PNG only, checked when the file is chosen and again by the server | Security, Functional Suitability |
| Technical evaluators, 3 Oct 2026 | A tenant could not withdraw a repair request | A tenant can cancel a request until work on it starts, at most once an hour, with an optional reason; the landlady is notified and the request is kept, marked as cancelled | Functional Suitability |
| Technical evaluators, 3 Oct 2026 | Moving a tenant out recorded no reason | An optional reason, kept in the activity record with the move-out | Functional Suitability |
| Technical evaluators, 3 Oct 2026 | Rows opened with "Show more" or "Show all" on the administrator's lists could not be hidden again | "Show fewer" returns the list to its first page | Usability |
| Technical evaluators, 3 Oct 2026 | Action history, and its error handling | Each person's Overview shows their own last three actions in plain words, each linking to where it happened ("Recorded a payment: 2B, October 2026, ₱8,800."); a refresh that fails for a moment keeps the figures already on screen; when the server refuses an entry, the message names the field and the reason ("Emergency contact phone: Enter a Philippine mobile number") instead of a technical code. Since October 2026 the three actions are listed in the side menu under Recently, as plain text | Usability, Reliability |
| Technical evaluators, 3 Oct 2026 | The footer was tall, with too much empty space | Spacing reduced and its columns placed side by side on a phone | Usability |
| Technical evaluators, 3 Oct 2026 | The privacy policy's list of contents crowded the page on a phone | On a phone the list opens from a side panel; on wider screens it stays beside the text | Usability |
| Survey, 30 September to 4 October 2026 | The technical evaluator rated all five Usability items 3, on 4 October | Answered before the fourteen changes above, eleven of them to usability, went live on 5 October; no further change | Usability |

| Source | Finding | Change made | Characteristic |
|---|---|---|---|
| Survey, 30 September 2026 | The landlady rated 3 her telling what each screen is for, and learning the system, without help | No change to the screens; the user manual (Appendix K) describes every screen in her words, and training at hand-over is stage 4 of Table 4-22 (Chapter 5, recommendation 6) | Usability |
| Survey, 4 October 2026 | The technical evaluator rated 3 a failed request being handled without presenting incorrect data | The same defect as walkthrough step 23b (₱0 instead of "—"): not reproduced on 1 October, and passed when re-tested on the landlady's laptop on 13 October 2026 | Reliability |
| Survey, 30 September 2026 | A tenant asked for the deposit paid and the electric bill to be shown | Since 13 October 2026 the tenant can see the deposit held for their tenancy, on their Overview since 5 November; the electric bill stays outside the system, since each tenant pays the electric company directly (Section 4.2.4) | Functional Suitability |

**Table M-19.** The two iterations, phase by phase (summarized in Chapter 4, Section 4.2.1). The first working version, from which the iterations started, was built from 24 July to 28 August 2026 (95 commits): the public site, the tenant and landlady portals, the ledgers, repairs, inquiries, notifications and audit trail, with

937 income rows and 1,327 expense allocations transferred from the landlady's workbook.

| Phase (Section 3.2) | Iteration 1: Verification, consolidation and testing (12 Sep to 31 Oct 2026, 1,732 commits) | Iteration 2: Interface redesign (1 Nov 2026 onward) |
|---|---|---|
| Requirements | The landlady settled four questions on 13 September: 33 units, no grace period, no automatic rent increase, and tenants recorded without needing an email address. The landlady's answers of 17 and 19 to 20 September: the advance is the deposit, rent and water stay one bill, and a rate for each of the 33 units; acknowledgement receipts for payments with no invoice number (1 October) | The team's review of the deployed screens on a phone and a computer; no new requirement from the landlady. The team's own rules for past years and repairs are under Development |
| Design | The business rule register rebuilt, 49 rules each with its evidence; the automatic rent increase removed from scope. The landlady's review on a phone (23 September) and the design rules the screens now follow; the tenant and administrator overviews reworked | Translucent glass surfaces over a light green page, in light and dark themes; boxes kept for what needs attention and what is looked for at a glance; sign in, inquiry and conversation joined in one screen |
| Development | Fifteen defects in behavior corrected, among them a partial payment that left no debt and cash recorded with nothing written to the books; the Excel downloads. Adyen with GCash working end to end in Adyen's test environment, with no real money (26 September); updates checked every two seconds; offline reading; the breached-password check (7 October) | The restyle of every screen; an inquiry the landlady deleted now says so to its sender; a signed-in account is sent from the inquiry pages to its own portal. The move-in form no longer asks for an anniversary date, which the server always set to the move-in date. Each screen keeps the figures it has loaded for the visit, so returning to it does not wait for them. A record dated in a closed year can no longer be recorded, changed or deleted, and last year stays open until every December receipt is in. A GCash payment settles the oldest unpaid bill. A tenant can no longer cancel a repair once someone is sent. The landlady's Manage window no longer shows a repair's messages, and a tenant's note reaches her as a notification |

| Phase (Section 3.2) | Iteration 1: Verification, consolidation and testing (12 Sep to 31 Oct 2026, 1,732 commits) | Iteration 2: Interface redesign (1 Nov 2026 onward) |
|---|---|---|
| Testing | Seven automated check suites, over 100 assertions, each defect covered by one. The screen-versus-database audit (26 and 29 September, six defects); 20 suites (29 September); the testing day with the landlady, three tenants and one prospective tenant (30 September); the technical evaluators' review (3 October) | All 114 test cases of Appendix L run again from 4 to 13 November: 99 passed (21 of them with wording the redesign had changed), the 10 timings were taken, 5 could not be run, and none failed. A click-by-click audit of every screen on 6 November. The 21 automated check suites passed after each change, and the screens were checked in light and dark at phone and desktop width. The page timings measured again on 10 November (Table M-31). A second round of users, 31 October to 10 November: ten tenants and four prospective tenants did the tasks, and the landlady, ten tenants and three prospective tenants answered the ISO/IEC 25010 questionnaire (Tables M-28 to M-30). Three technical evaluators reviewed the live system and answered it on 12 November |
| Deployment | Hosted on Vercel from 24 September; tenant accounts prepared on 29 September; in use beside the landlady's own records (Table 4-22) | The same site. Four corrections to the data, each a numbered migration applied after a backup and read back. Migration 088 (2 November) moved sixteen units into the kind of unit they really are, with the old kinds kept in the audit record. Migration 089 (4 November) removed old password hashes from 16 audit rows. Migration 090 (5 November) removed a rehearsal test tenant. Migration 091 (6 November) ended two receipts' rent periods a month after they start, as each was one month's rent |
| Review | 22 recorded errors in the submitted documents corrected; four standing rules for how the team works. The technical evaluators' 14 comments, all acted on by 5 October (Table 4-19); the ISO/IEC 25010 survey (Section 4.4) | The team's review of the screens. The users rated the redesigned system 4.84 overall and three technical evaluators 4.90, both Very High Quality (Table M-29) |

**Table M-20.** Quality in use (30 September 2026) (summarized in Chapter 4, Section 4.4.5)

| Characteristic | Measure | Result | Source |
|---|---|---|---|
| Effectiveness | Tasks completed, X = A / B | 34 / 34 = 1.00; without help 31 / 34 = 0.91 | Table 4-11 |
| Effectiveness | The landlady's walkthrough steps executed that passed, X = A / B | 24 / 25 = 0.96 | Table M-1 |
| Efficiency | Median time per task; mean wrong turns per task | 60 s; 0.2 | Table 4-11 |
| Satisfaction | Not measured with a satisfaction scale. The open answers: two of three tenants found nothing confusing during their tasks, and one said the system still felt lacking; on the survey form the landlady wrote "Beautiful system!" and a tenant named the ease of seeing their rent and asked for the deposit paid and the electric bill to be shown | | Sections 4.3.7 and 4.4 |
| Freedom from risk | Records of the landlady changed or lost by testing | None: the 937 income rows were untouched, and the records each test created were removed or voided afterwards by reviewed changes | Sections 4.3.2 and 4.3.4 |
| Context coverage | Contexts in which the tasks were done | The landlady's laptop in Chrome; an Android phone in Chrome; an iPhone in Safari; with no connection and on a stalled one; the landlady and two tenants at the same time (reading; saving at the same moment was not tested) | Sections 4.3.6 and 4.3.7 |

**Table M-21.** The 2011 characteristics and their place in ISO/IEC 25010:2023 (summarized in Chapter 4, Section 4.4)

| ISO/IEC 25010:2011 (this evaluation) | ISO/IEC 25010:2023 | What changed |
|---|---|---|
| Functional Suitability | Functional Suitability | Unchanged |
| Performance Efficiency | Performance Efficiency | Unchanged |
| Compatibility | Compatibility | Unchanged |
| Usability | Interaction Capability | Renamed; user interface aesthetics became user engagement, accessibility was divided into inclusivity and user assistance, and self-descriptiveness was added |
| Reliability | Reliability | Maturity renamed faultlessness |
| Security | Security | Resistance added |
| Maintainability | Maintainability | Unchanged |
| Portability | Flexibility | Renamed; scalability added |
| (no counterpart) | Safety | New. Its fail-safe and hazard-warning sub-characteristics correspond to results reported here under Reliability: a figure that cannot be loaded is never shown as ₱0, and a save that may not have arrived says so (Sections 4.3.4 and 4.3.6) |

**Table M-22.** Simultaneous use by the landlady and tenants (30 September 2026, two tenants) (summarized in Chapter 4, Section 4.3.7)

| Test (at the same moment) | Result |
|---|---|
| Every tenant opens their overview (C-01) | Pass: both screens loaded; the slowest in about 1 second |
| Each tenant sees only their own unit and records (C-02) | Pass: each saw only their own unit and account |
| Every tenant sends a message on a request (C-03, C-04) | Not performed: one tenant sent a note, and the landlady saw and answered it within a minute |
| The landlady changes two requests; only those tenants are notified (C-05) | Not applicable: by the landlady's decision a tenant is notified when a repair is done, not when it is started, so no notice was expected |
| A double tap on Send saves one message (C-06) | Not performed |
| The landlady opens a full year of Monthly Income while tenants are active (C-07) | Pass: loaded in about 3 seconds |
| The landlady on the laptop and tenants on phones at once (C-08) | Pass: no session affected another |

**Table M-23.** Test activities and the ISO/IEC 25010:2011 characteristics they measured (summarized in Chapter 4, Section 4.3)

| Test activity (Section) | Test level | Characteristics measured (sub-characteristics) |
|---|---|---|
| Automated verification (4.3.1) | Unit and integration | Functional Suitability (correctness); Security (confidentiality, integrity, authenticity); Compatibility (interoperability); Maintainability (testability) |
| Use in the live environment (4.3.2) | System, in operation | Functional Suitability (correctness); Compatibility (interoperability); Reliability (recoverability) |
| Screen-versus-database audit (4.3.3) | System | Functional Suitability (correctness); Reliability (maturity) |
| Functional walkthrough (4.3.4) | System | Functional Suitability (completeness, correctness); Usability (user error protection); Security (confidentiality); Reliability (fault tolerance) |
| Responsiveness (4.3.5) | System | Performance Efficiency (time behavior, resource utilization) |
| Offline, installation, weak connections and simultaneous users (4.3.6) | System | Reliability (fault tolerance); Performance Efficiency (capacity); Portability (installability, adaptability); Usability (accessibility) |
| User acceptance testing (4.3.7) | Acceptance | Quality in use (effectiveness, efficiency, context coverage); Usability (learnability, operability); Security (confidentiality) |

**Table M-24.** Offline and installation behavior (29 September 2026) (summarized in Chapter 4, Section 4.3.6)

| Test | Result |
|---|---|
| Installation requirements (manifest, service worker, icons) | Met; the service worker keeps 67 application files on the device after the first visit |
| Opening ten addresses with no connection | The application opened on all ten in about 1.5 seconds instead of the browser's "no internet" page |
| Tenant and administrator pages with no connection, signed out | The application opened and asked for sign-in; no personal or financial data was shown |
| Notice when the connection drops | Shown at once: "No connection. You can read what is already loaded, but nothing can be saved or paid until it is back." |
| Notice when the connection returns | "Back online" |
| Public unit list with no connection | Shown from the device's memory |
| Public unit list when the server stops answering | Shown from the device's memory after 3.1 seconds |
| Public page on a slow connection (400 kbps, 400 ms), first visit | Units on screen after 4.6 seconds |
| The same page, returning visitor | Units on screen after 0.3 seconds, because the installed application already holds its files |

**Table M-25.** Simulated simultaneous users, read operations (29 September 2026) (summarized in Chapter 4, Section 4.3.6)

| Test | Simulated users at once | Duration | Requests | Errors | Slowest 5% took longer than |
|---|---|---|---|---|---|
| Public pages and public data, live site | 12 | 60 s | 1,085 | 0 | 0.52 s |
| The landlady's ten main data requests, live database | 6 | 45 s | 285 | 0 | 0.80 s |

**Table M-26.** Simultaneous saves (19 and 24 September 2026) (summarized in Chapter 4, Section 4.3.6)

| Test | Before the change | After the change |
|---|---|---|
| The same receipt sent twice at once, live system | Both saved: 2 rows | 1 saved, 1 refused: 1 row |
| The same receipt sent five times at once, live system | All five saved: 5 rows | 1 saved, 4 refused: 1 row |
| One receipt covering three months, live system | | Saved as 3 rows, June to August 2026, as before |
| Two GCash checkouts opened on one bill at once, payment code with a stand-in database | Both opened, so both payments could be recorded | 1 opened, 1 refused |
| A checkout 16 minutes after one was abandoned | | Opened: an abandoned checkout holds the bill for 15 minutes, not for good |

**Table M-27.** Corrections after the testing activities (13 October 2026, referred to in Chapter 4, Section 4.4.6)

| Found | What a person saw | Cause | Correction | Checked by |
|---|---|---|---|---|
| Audit of the live records against what each tenant's screen shows, 13 October 2026 | Two tenants were shown their August rent as overdue (₱7,450 and ₱10,400), although the landlady's receipts record both as paid on 12 and 25 August. Severity high: a wrong amount shown. The landlady's ledger itself was correct | Each bill was raised on 26 September, when the tenant opened the payment screen. The landlady's August receipts were added on 30 September by a transfer from her workbook, which writes the receipts directly; only a receipt recorded on the Monthly Income screen settles the bills it covers | The same day, after a backup: each bill was settled by a verified payment carrying its receipt's number and date, as the Monthly Income screen would have done, with an entry in the audit trail. The ledger was unchanged (952 receipts). The transfer script now lists any open bill its receipts would pay and will not run until that is acknowledged | A new automated rule compares every bill shown as owed with the receipts; it failed, naming the two units, when the correction was undone in a test, and passes on the live records |

**Table M-28.** Respondents before and after the redesign (summarized in Chapter 4, Section 4.4.1)

| Group | Before the redesign | After the redesign |
|---|---|---|
| Landlady | 1 | 1 |
| Tenants | 3 | 10 |
| Prospective tenants | 1 | 3 |
| Technical evaluators | 5 | 3 |
| **Total** | **10** | **17** |

The users before the redesign tested on 30 September 2026, and those after it from 31 October to 10 November 2026. The landlady answered both times. Three answers of the second round are left out: two from prospective tenants who by mistake answered the landlady's section and the tenants' section, whose items describe another group's use, and one the team made while testing the form. The technical evaluators of the first round answered from 4 to 18 October 2026, and those of the second on 12 November 2026. Names are removed from the response files: `docs/TESTING_DAY/results/ROUND_1_...csv`, `ROUND_2_...csv` and `TECHNICAL_2026-11-12_...csv`.

**Table M-29.** Composite means before and after the redesign (summarized in Chapter 4, Section 4.4.3)

| Characteristic | Users before (5) | Technical evaluators before (5) | Users after (14, Chapter 4) | Technical evaluators after (3, Chapter 4) |
|---|---|---|---|---|
| Functional Suitability | 4.94 | 4.32 | 4.80 | 4.93 |
| Performance Efficiency | 4.56 | 4.27 | 4.86 | 4.89 |
| Compatibility | 4.56 | 4.53 | 4.79 | 5.00 |
| Usability | 4.49 | 4.20 | 4.80 | 4.87 |
| Reliability | 4.74 | 4.20 | 4.78 | 4.83 |
| Security | 5.00 | 4.36 | 5.00 | 4.93 |
| Maintainability | not rated | 4.32 | not rated | 4.73 |
| Portability | 4.96 | 4.53 | 4.83 | 5.00 |
| **Overall** | **4.75** | **4.34** | **4.84** | **4.90** |

Each users' composite is the mean of the group means of the landlady, the tenants and the prospective tenants, as in Chapter 3, and each users' overall is the mean of seven characteristics, since only the technical evaluators rated Maintainability. Security is the landlady's rating among the users. Every composite is Very High Quality on the scale of Chapter 3's Table 3-5 except the first round's technical evaluators' Usability and Reliability, 4.20, at the top of High Quality. All ten answers of the first round, taken as one group, rated the system 4.59.

**Table M-30.** Task results, second round (summarized in Chapter 4, Section 4.3.7)

| Task | Testers | Completed without help | Completed with help | Not completed | Mean wrong turns | Completion without help |
|---|---|---|---|---|---|---|
| **Tenants** | | | | | | |
| First sign-in and choosing a password (T-01, T-02) | 10 | 15 | 5 | 0 | 0.3 | 75% |
| Finding their unit, rent and balance (T-03, T-04) | 10 | 20 | 0 | 0 | 0.0 | 100% |
| Finding their payments (T-05, T-06) | 10 | 20 | 0 | 0 | 0.1 | 100% |
| Sending a repair request with a photo (T-07 to T-09) | 10 | 29 | 1 | 0 | 0.0 | 97% |
| Following up a request (T-10, T-11) | 10 | 20 | 0 | 0 | 0.0 | 100% |
| Checking their details and notifications (T-12, T-13) | 10 | 20 | 0 | 0 | 0.1 | 100% |
| Opening the GCash payment (T-14) | 10 | 9 | 1 | 0 | 0.0 | 90% |
| Staying out of the landlady's pages, signing out (T-15, T-16) | 10 | 18 | 2 | 0 | 0.2 | 90% |
| All tenant tasks | 10 | 151 | 9 | 0 | 0.1 | 94% |
| **Prospective tenants** | | | | | | |
| Understanding what the place is (PR-01) | 4 | 4 | 0 | 0 | 0.0 | 100% |
| Finding unit types, rates and availability (PR-02 to PR-04) | 4 | 12 | 0 | 0 | 0.0 | 100% |
| Reading one unit's details (PR-05) | 4 | 3 | 1 | 0 | 0.3 | 75% |
| Finding the water charge, location and contact (PR-06, PR-07) | 4 | 8 | 0 | 0 | 0.0 | 100% |

| Task | Testers | Completed without help | Completed with help | Not completed | Mean wrong turns | Completion without help |
|---|---|---|---|---|---|---|
| Sending an inquiry and reading the reply (PR-08) | 4 | 4 | 0 | 0 | 0.0 | 100% |
| All prospect tasks | 4 | 31 | 1 | 0 | 0.0 | 97% |

From `docs/TESTING_DAY/results/ROUND_2_T_observations.csv`, computed by `scripts/survey/compute-uat.mjs` as Round 1 was. Time on task was not recorded reliably in this round, so no times are reported. Round 1's timed results stand in Section 4.3.7. A task finished after a neutral prompt counts as completed without help, after a hint as completed with help (Chapter 3). The landlady's tasks were not repeated: her results are Round 1's (Table M-1).

**Table M-31.** Page timings before and after the redesign (summarized in Chapter 4, Section 4.3.5)

| Screen | 30 September 2026, laptop (Chapter 4) | 10 November 2026, laptop | How it was timed on 10 November |
|---|---|---|---|
| Public home page, second load | 0.62 s | 0.35 s | Page fully loaded, from the browser's own timer |
| Sign-in to the landlady's Overview | 2.10 s | 2.63 s | From pressing Sign in until the Overview's figures were drawn |
| Monthly Income, one full year | 1.45 s | 1.42 s | From a fresh load of the page until the year's total and its records were drawn |
| Administrator's Overview, fresh load | not timed | 1.67 s | From a fresh load until every figure was drawn |
| Tenant portal, first load | 2.15 s | 1.19 s | From a fresh load until Amount due and the monthly charge were drawn |
| Tenant payments page | 1.20 s | 0.54 s | From the menu until the payment history was drawn |
| Administrator's Overview, layout shift | 0.22 (Lighthouse) | 0.08 | Measured in the browser while the page loaded. Below 0.1 counts as good |
| PageSpeed Insights, public page, phone (slow 4G) | 85 to 88, largest element 3.4 to 3.6 s | 85, largest element 4.2 to 4.4 s (two runs) | Google PageSpeed Insights, Lighthouse 13.6 |
| PageSpeed Insights, public page, desktop | 95 and 98, largest element 0.9 and 1.1 s (1 October) | 99, largest element 0.9 s | Google PageSpeed Insights |

On 30 September the times were taken with a stopwatch on an Acer Nitro V15 over the boarding house's Wi-Fi. On 10 November they were taken from the browser's own performance timer, on a Windows 11 laptop in the Chromium browser of the Claude desktop app, on a different network, with the landlady and the test tenant signed in by a team member. The two days used different laptops, networks and timing methods, so the comparison shows the direction of the

change, not an exact difference. The Excel export and sending a repair request were not timed again: the first saves a file and the second writes to the live records. No phone was timed by hand on 10 November.

**Table M-32.** The 44 functional requirements, graded against the code in September 2026 and against the final system on 13 November 2026 (summarized in Chapter 4, Section 4.1.2)

| Code | Requirement | September 2026 | 13 November 2026 | What the final system does, where not as worded or where the grade changed |
|---|---|---|---|---|
| FR-001 | **Authentication.** The system shall authenticate administrators and active tenants. | As worded | As worded | |
| FR-002 | **Role-Based Access.** The system shall enforce access according to user role. | As worded | As worded | |
| FR-003 | **Public Website.** The system shall display property information, room information, amenities, photos, and availability information. | In part | In part | Units, rates, photos and availability come from the records. The amenities are written into the screens, not stored |
| FR-004 | **Public Inquiry.** A visitor shall be able to submit an inquiry for a specific room when that room is accepting inquiries. | As worded | As worded | |
| FR-005 | **Inquiry Management.** The administrator shall view, manage, communicate with, and update inquiry statuses. | As worded | As worded | |
| FR-006 | **Inquiry Conversion.** The administrator shall be able to reuse inquiry information during tenant onboarding. | As worded | As worded | |
| FR-007 | **Room Management.** The administrator shall create, edit, view, publish, hide, reserve, occupy, maintain, and make rooms available. | As worded | As worded | |

| Code | Requirement | September 2026 | 13 November 2026 | What the final system does, where not as worded or where the grade changed |
|---|---|---|---|---|
| FR-008 | **Room History.** The system shall preserve room occupancy and tenant history. | In part | In part | Past tenancies and every rate change are kept, and moved-out tenants can be listed, but no screen shows a unit's past rates or its tenants over time |
| FR-009 | **Tenant Management.** The administrator shall manage tenant records and account status. | As worded | As worded | |
| FR-010 | **Tenant Profile Updates.** Tenants shall be able to update permitted personal information. | As worded | As worded | |
| FR-011 | **Billing.** The system shall support rent and water billing records. | In part | In part | Bills are kept, but a bill is raised only when the tenant opens a payment, and the landlady cannot write one herself |
| FR-012 | **Due Dates.** The system shall support individual due dates. | As worded | As worded | |
| FR-013 | **Overdue Monitoring.** The system shall identify overdue payments, a bill being overdue from the day after its due date, with no grace period. | As worded | As worded | |
| FR-014 | **Manual Payments.** The administrator shall be able to record manual payments. | As worded | As worded | |
| FR-015 | **Online Payments.** The system shall support optional online payment through Adyen for the GCash payment workflow. | As worded | As worded | |
| FR-016 | **Payment Verification.** The administrator shall be able to verify online payments. | As worded | As worded | |
| FR-017 | **Financial Corrections.** The administrator shall be able to correct financial records with auditability. | As worded | As worded | |
| FR-018 | **Expense Management.** The administrator shall record categorized expenses and room allocations. | In part | As worded | Each allocation must name a property area that exists, which the database enforces |

| Code | Requirement | September 2026 | 13 November 2026 | What the final system does, where not as worded or where the grade changed |
|---|---|---|---|---|
| FR-019 | **Cash Flow.** The system shall calculate income, expenses, and net cash flow. | Not as worded | Not as worded | Income, expenses and net income are shown on the Overview, worked out on the screen, not by the server |
| FR-020 | **Profitability Analytics.** The system shall present profitability-related graphs and trends. | Not as worded | Not as worded | The Overview's month-by-month chart of income and expenses, worked out on the screen, not by the server |
| FR-021 | **Maintenance Tickets.** Tenants shall be able to create issue tickets. | As worded | As worded | |
| FR-022 | **Attachments.** Tenants shall be able to attach photos to issue tickets. | In part | As worded | The photo is sent through the server with the request, and the tenant sees it with the request |
| FR-023 | **Ticket Priority.** Tickets shall support Emergency, High, Medium, and Low priority. | As worded | As worded | |
| FR-024 | **Ticket Status.** Tenants shall be able to view ticket status. | As worded | As worded | |
| FR-025 | **Ticket Administration.** The administrator shall manage and close tickets. | As worded | As worded | |
| FR-026 | **Communication.** The system shall centralize relevant tenant, inquiry, and issue communication. | In part | As worded | A prospective tenant can now read and answer replies with the reference code given when the inquiry was sent |
| FR-027 | **Notifications.** The system shall provide in-system notifications for important events. | As worded | As worded | |
| FR-028 | **Reports.** The system shall provide live reports and export capabilities. | In part | In part | The Overview and the Excel workbooks are live, but no server endpoint returns report totals |
| FR-029 | **Audit Logs.** The system shall record important actions and financial changes. | As worded | As worded | |

| Code | Requirement | September 2026 | 13 November 2026 | What the final system does, where not as worded or where the grade changed |
|---|---|---|---|---|
| FR-030 | **Offline-Ready Access.** The system may provide limited offline-ready access to cached resources and safe previously available data. | In part | In part | The app and the public pages work offline, but no signed-in record is kept for offline use |
| FR-031 | **Monthly Income Report Layout.** The system shall reproduce the landlady's Monthly Income Report layout, units grouped by cluster (BH, Back Apartment, Penthouse, Front Apartment, Linda) in canonical order, with per-cluster and grand subtotals. | In part | As worded | The server builds the workbook in her layout: clusters in order, subtotals, and Linda apart |
| FR-032 | **Guided Monthly Payment Entry.** The administrator shall record a monthly payment through a guided form (unit, date, contact and invoice, rent amount), with rent period, 50% share, and remitted amount computed automatically. | In part | As worded | The rent period comes from the anniversary date, and the 50% Share and remitted amount are computed columns |
| FR-033 | **Occupant Count Memory.** The system shall pre-fill a unit's occupant count from the same tenant's previous month entry, editable by the administrator. | Not as worded | In part | The occupant count is filled in from the tenancy, editable, rather than from the previous month's receipt |
| FR-034 | **Water Payment Validation.** The system shall validate that Water Payment equals Occupants × ₱200 and warn the administrator before saving a mismatched value. | In part | As worded | Water is worked out from the occupants at the configured rate, so a mismatched amount cannot be saved |
| FR-035 | **One-Time Onboarding Fields.** The system shall capture Anniversary Date and Deposit once, at tenant onboarding, and reuse them on every subsequent monthly entry without re-entry. | In part | In part | The anniversary date is reused for every receipt. The deposit is shown on the tenant's pages, not on receipts |
| FR-036 | **Linda Billing Flow.** The system shall record Linda's units (LF, LB) in their own section, separate from the other clusters. Water is charged at the same per-occupant rate as every other unit. | In part | As worded | Linda's units are kept in their own section, with water at the same per-occupant rate |

| Code | Requirement | September 2026 | 13 November 2026 | What the final system does, where not as worded or where the grade changed |
|---|---|---|---|---|
| FR-037 | **Monthly Expenses Ledger.** The system shall reproduce the landlady's Monthly Expenses ledger (date, OR or supplier, one-or-more Property Area allocations, and one fixed expense category per entry). | As worded | As worded | |
| FR-038 | **Split Expense Entry.** The administrator shall be able to allocate a single expense entry's amount across more than one Property Area without re-entering its date, supplier, or category. | As worded | As worded | |
| FR-039 | **Automatic Expense Totals.** The system shall automatically compute an expense entry's row total, each Property Area's monthly bottom total, and each category's "this month" total. | In part | In part | Row totals are stored. The area and category totals are worked out on the screen and in the workbook |
| FR-040 | **Expense Category Cumulative Totals.** The system shall automatically roll each expense category's cumulative total forward from the previous month. | In part | In part | The running total per category appears in the Excel workbook only |
| FR-041 | **Fixed Expense Category Dropdown.** The system shall present the fixed expense category list (with Salaries sub-lines 6a/6b/6c) as a dropdown when tagging an expense entry. The administrator may add a category of her own after the thirteen. | As worded | As worded | |
| FR-042 | **Expense/Category Reconciliation Check.** The system shall be able to verify that the sum of category "this month" totals equals the sum of Property Area bottom totals for the same month. | As worded | As worded | |
| FR-043 | **Admin-Only Income/Expense Entry.** Only the administrator shall be able to create or edit Monthly Income Report and Monthly Expenses Report entries. | As worded | As worded | |

| Code | Requirement | September 2026 | 13 November 2026 | What the final system does, where not as worded or where the grade changed |
|---|---|---|---|---|
| FR-044 | **Excel Export of Income/Expense Reports.** The system shall export the Monthly Income Report and Monthly Expenses Report as Excel-compatible spreadsheet files, matching their defined layouts. | Interface only | As worded | The server writes both ledgers as Excel workbooks in their layouts |

As worded: the system does what the requirement says. In part: it does some of it. Not as worded: what the requirement describes reaches the landlady in another form. Interface only: done in the browser with no part on the server. Counts on 13 November 2026: 33 as worded, 9 in part, 2 not as worded. Each requirement is worded as in the team's requirements register.

# APPENDIX N: DATA DICTIONARY

This appendix lists every table and column of the Hivelet database as it stands in the live system, with the type and length of each column, whether a value is required, its default, and the keys that link the tables. PK marks a primary key and FK a foreign key, with the table and column it refers to. A generated column is computed by the database from the columns named and cannot be written. Allowed values of an enumerated type are listed beside it. No sample values are given, because the rows hold tenants' personal data.

**Property**

**rooms**

One row per rentable unit: 33 units in five clusters.

| Column | Type | Required | Default | Key | Description |
|---|---|---|---|---|---|
| id | uuid | Yes | gen_random_uuid() | PK | Primary key |
| room_number | varchar(20) | Yes | | | The unit's code, as the landlady writes it (unique regardless of capitals) |
| floor | integer | Yes | | | Building level: 1 to 3 residential, 4 the rooftop penthouse |
| cluster_code | varchar(50) | Yes | | FK clusters.code | The cluster the unit belongs to |
| room_type | room_type_enum (Studio, One-bedroom, Two-bedroom, Three-bedroom) | Yes | Studio | | Kind of unit (studio, one-bedroom and so on) |
| description | text | | | | Text shown on the public site |
| capacity | integer | Yes | | | Most occupants the unit is meant for |
| base_price | numeric(10,2) | Yes | | | The rate the unit was first set up with |
| current_price | numeric(10,2) | Yes | | | The rate in effect now, set by the landlady |
| operational_status | operational_status_type (Available, Reserved, Occupied, Under Maintenance) | Yes | Available | | Available, Occupied, Reserved or under maintenance |
| visibility_status | visibility_status_type (Published, Hidden) | Yes | Published | | Whether the unit is listed on the public site |
| available_from | date | | | | When a unit becoming free can be moved into |
| is_linda_unit | boolean | | false | | True for the two Linda units, whose money is kept separately (BR-040) |
| created_at | timestamptz | | now() | | When the row was created |

| Column | Type | Required | Default | Key | Description |
|---|---|---|---|---|---|
| updated_at | timestamptz | | now() | | When it last changed |

### clusters

The five clusters (Boarding House, Back Apartment, Front Apartment, Penthouse, Linda) and the expense area each books its costs to.

| Column | Type | Required | Default | Key | Description |
|---|---|---|---|---|---|
| code | varchar(50) | Yes | | PK | Short code of the cluster |
| name | varchar(100) | Yes | | | Name shown on screens |
| display_order | integer | Yes | | | Order on screens and reports |
| expense_area | property_area_type (Boarding House, Main House, Front Apartment, Back Apartment, Other Expenses / Personal, Penthouse) | Yes | | FK property_areas.code | The property area this cluster's costs are booked to |

### property_areas

The parts of the property an expense can be split across, and whether each counts as a rental cost.

| Column | Type | Required | Default | Key | Description |
|---|---|---|---|---|---|
| code | property_area_type (Boarding House, Main House, Front Apartment, Back Apartment, Other Expenses / Personal, Penthouse) | Yes | | PK | The area |
| name | varchar(150) | Yes | | | Name shown on screens |
| is_rental_expense | boolean | Yes | true | | False for the Main House (personal) and other personal costs, which are not deducted from rental income |
| display_order | integer | Yes | 0 | | Order on screens and reports |
| notes | text | | | | Notes on what the area covers |

### room_photos

Photographs and floor plans of each unit, one marked primary.

| Column | Type | Required | Default | Key | Description |
|---|---|---|---|---|---|
| id | uuid | Yes | gen_random_uuid() | PK | Primary key |
| room_id | uuid | Yes | | FK rooms.id | The unit |
| file_url | text | Yes | | | Where the image is kept |
| caption | varchar(255) | | | | Caption shown with it |
| display_order | integer | Yes | 0 | | Order in the unit's gallery |
| is_primary | boolean | Yes | false | | The photo shown first |
| uploaded_by | uuid | | | FK profiles.id | Who added it |
| created_at | timestamptz | Yes | now() | | When the row was created |

### room_price_history

Every change of a unit's rate, written by a database trigger.

| Column | Type | Required | Default | Key | Description |
|---|---|---|---|---|---|
| id | uuid | Yes | gen_random_uuid() | PK | Primary key |
| room_id | uuid | Yes | | FK rooms.id | The unit |
| previous_price | numeric(10, 2) | Yes | | | The rate before the change |
| new_price | numeric(10, 2) | Yes | | | The rate after it |
| effective_date | date | Yes | CURRENT_DATE | | When the new rate applies |
| reason | text | | | | Why, if given |
| created_by | uuid | | | FK profiles.id | Who made the change |
| created_at | timestamptz | | now() | | When the row was created |

## People

### profiles

Everyone with a record: the administrator, tenants, and visitors recorded from an inquiry.

| Column | Type | Required | Default | Key | Description |
|---|---|---|---|---|---|
| id | uuid | Yes | gen_random_uuid() | PK | Primary key |
| auth_user_id | uuid | | | | Link to a sign-in account of the hosting service, where one exists |

| Column | Type | Required | Default | Key | Description |
|---|---|---|---|---|---|
| email | varchar(255) | | | | Not used since 20 October 2026: nobody signs in with or gives an email (unique when present) |
| full_name | varchar(255) | Yes | | | Full name |
| phone_number | varchar(50) | | | | Philippine mobile number (unique when given) |
| emergency_contact_name | varchar(255) | | | | Emergency contact |
| emergency_contact_phone | varchar(50) | | | | Emergency contact's number |
| occupation | varchar(100) | | | | Occupation, optional |
| facebook_url | text | | | | Facebook profile, optional |
| role | user_role_type (admin, tenant, prospect) | Yes | tenant | | admin, tenant, prospect or guest |
| account_status | account_status_type (active, inactive) | Yes | active | | active, or inactive once a tenant has moved out |
| created_at | timestamptz | | now() | | When the row was created |
| updated_at | timestamptz | | now() | | When it last changed |
| password_hash | varchar(255) | | | | The password as a bcrypt hash; the password itself is never stored |
| last_login_at | timestamptz | | | | Last successful sign-in |
| failed_login_count | integer | Yes | 0 | | Failed sign-ins in a row; five lock the account |
| locked_until | timestamptz | | | | While in the future, sign-in is refused |
| password_changed_at | timestamptz | | | | When the password last changed; earlier sessions end |
| must_change_password | boolean | Yes | false | | True until a new tenant, or one whose password was reset, sets their own |

| Column | Type | Required | Default | Key | Description |
|---|---|---|---|---|---|
| login_id | varchar(20) | | | | Short sign-in ID given to a tenant (unique) |

### room_assignments

Tenancies: who occupies which unit, from when, with how many occupants.

| Column | Type | Required | Default | Key | Description |
|---|---|---|---|---|---|
| id | uuid | Yes | gen_random_uuid() | PK | Primary key |
| room_id | uuid | Yes | | FK rooms.id | The unit |
| tenant_profile_id | uuid | Yes | | FK profiles.id | The tenant |
| start_date | date | Yes | | | Move-in date (not recorded in the landlady's workbook for transferred tenancies) |
| end_date | date | | | | Move-out date, once moved out |
| anniversary_date | date | Yes | | | The day of the month rent runs from |
| deposit_amount | numeric(10,2) | Yes | 0 | | The deposit held: one month, set against repairs at move-out (BR-039) |
| occupant_count | integer | Yes | 1 | | Occupants; water is ₱200 for each |
| is_primary_contact | boolean | | true | | Whether this tenant is the unit's contact |
| is_active | boolean | | true | | True while the tenancy runs; only one per unit (unique index) |
| created_at | timestamptz | | now() | | When the row was created |
| updated_at | timestamptz | | now() | | When it last changed |

### Money

**monthly_income_records**

The landlady's Monthly Income ledger, one row per receipt per month covered (Chapter 4, Table 4-5).

| Column | Type | Required | Default | Key | Description |
|---|---|---|---|---|---|
| id | uuid | Yes | gen_random_uuid() | PK | Primary key |
| room_id | uuid | Yes | | FK rooms.id | The unit |
| tenant_profile_id | uuid | | | FK profiles.id | The tenant credited |
| assignment_id | uuid | | | FK room_assignments.id | The tenancy credited |
| year | integer | Yes | | | Year of the month the rent is for |
| month | integer | Yes | | | Month the rent is for (1 to 12) |
| date_paid | date | Yes | | | Date the money was received |
| contact_name | varchar(255) | Yes | | | Name as written on the receipt |
| invoice_number | varchar(100) | | | | The landlady's invoice number (INV#…), an acknowledgement receipt, or none; never made up by the system |
| rent_period_start | date | Yes | | | First day the payment covers |
| rent_period_end | date | Yes | | | Last day it covers |
| rent_amount | numeric(10,2) | Yes | | | Rent received for the period |
| fifty_percent_share | numeric(10,2) | | | Generated: rent_amount / 2.0 | A system-computed figure equal to half that row's Rent Amount, kept for ledger parity with the owner's historical spreadsheet (BR-035) |

| Column | Type | Required | Default | Key | Description |
|---|---|---|---|---|---|
| occupants | integer | Yes | | | Occupants the water was charged for |
| water_payment | numeric(10,2) | Yes | 0 | | Water received |
| remitted_amount | numeric(10,2) | | | Generated: rent_amount + water_payment | Rent plus water (BR-038) |
| is_linda_billing | boolean | | false | | True for a Linda unit's row |
| linda_electricity_charge | numeric(10,2) | | 0 | | Historical only: a flat electricity charge retired on 13 September 2026; old rows kept as collected |
| linda_water_charge | numeric(10,2) | | 0 | | A Linda unit's water, ₱200 per occupant, recorded here and kept out of the remitted amount (BR-040) |
| payment_method | payment_method_type (Cash, GCash, Bank Transfer, Adyen Online) | Yes | Cash | | Cash, GCash or bank transfer |
| verification_status | verification_status_type (Verified, Pending Verification, Rejected) | Yes | Verified | | Verified, Pending Verification or Rejected; only Verified counts as paid |
| created_at | timestamptz | | now() | | When the row was created |
| updated_at | timestamptz | | now() | | When it last changed |
| transaction_reference | varchar(120) | | | | The gateway's or bank's reference (recorded once) |
| voided_at | timestamptz | | | | When the receipt was voided; a voided receipt leaves every total |
| voided_by | uuid | | | FK profiles.id | Who voided it |
| void_reason | text | | | | Why |

### monthly_expense_entries

The landlady's Monthly Expenses ledger: one row per expense.

| Column | Type | Required | Default | Key | Description |
|---|---|---|---|---|---|
| id | uuid | Yes | gen_random_uuid() | PK | Primary key |
| expense_date | date | Yes | | | Date of the expense |
| invoice_supplier | text | Yes | | | Invoice or supplier, as the landlady writes it |
| category_code | varchar(20) | Yes | | FK fixed_expense_categories.code | Expense category |
| total_expenses | numeric(10,2) | Yes | 0 | | Total, kept equal to its allocations by a trigger (BR-045) |
| created_by | uuid | | | FK profiles.id | Who recorded it |
| created_at | timestamptz | | now() | | When the row was created |
| updated_at | timestamptz | | now() | | When it last changed |
| voided_at | timestamptz | | | | When voided |
| voided_by | uuid | | | FK profiles.id | Who voided it |
| void_reason | text | | | | Why |

### expense_property_allocations

How each expense is split across property areas. The parts add up to the expense.

| Column | Type | Required | Default | Key | Description |
|---|---|---|---|---|---|
| id | uuid | Yes | gen_random_uuid() | PK | Primary key |
| expense_entry_id | uuid | Yes | | FK monthly_expense_entries.id | The expense |
| property_area | property_area_type (Boarding House, Main House, Front Apartment, Back Apartment, Other Expenses / Personal, Penthouse) | Yes | | FK property_areas.code | The area this part is for (one part per area) |
| amount | numeric(10,2) | Yes | | | The part's amount |
| created_at | timestamptz | | now() | | When the row was created |

### fixed_expense_categories

The landlady's expense categories (the thirteen numbered lines, with sub-lines), and any she adds.

| Column | Type | Required | Default | Key | Description |
|---|---|---|---|---|---|
| code | varchar(20) | Yes | | PK | Category code (1 to 13, with sub-lines such as 6a) |
| name | varchar(255) | Yes | | | Name shown on screens and reports |
| parent_code | varchar(20) | | | FK fixed_expense_categories.code | The line a sub-line belongs to |
| display_order | integer | Yes | | | Order on the ledger and in the download |

### bills

Bills raised for a tenancy: rent and water for one whole month.

| Column | Type | Required | Default | Key | Description |
|---|---|---|---|---|---|
| id | uuid | Yes | gen_random_uuid() | PK | Primary key |
| room_id | uuid | Yes | | FK rooms.id | The unit |
| tenant_profile_id | uuid | Yes | | FK profiles.id | The tenant billed |
| bill_type | bill_type_enum (Rent, Water, Combined, Other) | Yes | Combined | | Rent and water together, as the landlady bills them |
| billing_period_start | date | Yes | | | First day of the whole month billed |
| billing_period_end | date | Yes | | | Last day of it |
| rent_amount | numeric(10,2) | Yes | 0 | | The whole month's rent; never prorated |
| water_amount | numeric(10,2) | Yes | 0 | | Occupants × the water rate |
| total_amount | numeric(10,2) | Yes | | | Rent plus water |
| due_date | date | Yes | | | Due date; overdue from the next day |
| grace_period_end_date | date | Yes | | | Equal to the due date: there is no grace period |
| status | bill_status_type (Pending, Due, Overdue, Paid, Partially Paid) | Yes | Pending | | Pending, Paid, Overdue and so on |
| created_at | timestamptz | | now() | | When the row was created |
| updated_at | timestamptz | | now() | | When it last changed |
| checkout_opened_at | timestamptz | | | | When a GCash checkout was opened on the bill; another is refused for 15 minutes |

**payments**

Payments made against bills, online with GCash through Adyen or recorded by the landlady, with their verification.

| Column | Type | Required | Default | Key | Description |
|---|---|---|---|---|---|
| id | uuid | Yes | gen_random_uuid() | PK | Primary key |
| bill_id | uuid | | | FK bills.id | The bill paid, where one exists |
| room_id | uuid | Yes | | FK rooms.id | The unit |
| tenant_profile_id | uuid | Yes | | FK profiles.id | The tenant |
| amount | numeric(10,2) | Yes | | | Amount paid |
| payment_method | payment_method_type (Cash, GCash, Bank Transfer, Adyen Online) | Yes | Cash | | Cash, GCash or bank transfer |
| payment_source | varchar(100) | Yes | On-Site Cash | | Where it came from (on site, or online through Adyen with GCash) |
| verification_status | verification_status_type (Verified, Pending Verification, Rejected) | Yes | Pending Verification | | Pending Verification until the landlady verifies or rejects it |
| transaction_reference | varchar(255) | | | | The gateway reference (recorded once) |
| paid_at | timestamptz | Yes | now() | | When it was paid |
| verified_at | timestamptz | | | | When the landlady verified it |
| verified_by | uuid | | | FK profiles.id | Who verified it |
| created_at | timestamptz | | now() | | When the row was created |

**system_settings**

Values the landlady can change without a new release, such as the water rate per occupant.

| Column | Type | Required | Default | Key | Description |
|---|---|---|---|---|---|
| key | varchar(80) | Yes | | PK | Name of the setting |
| value | text | Yes | | | Its value |
| value_type | varchar(20) | Yes | number | | How the value is read (number, text) |
| label | varchar(160) | Yes | | | Name shown to the landlady |
| description | text | | | | What it does |
| business_rule | varchar(20) | | | | The business rule it serves |
| updated_by | uuid | | | FK profiles.id | Who last changed it |
| updated_at | timestamptz | Yes | now() | | When it last changed |

## Requests

### maintenance_tickets

Repair requests, from submission to closing.

| Column | Type | Required | Default | Key | Description |
|---|---|---|---|---|---|
| id | uuid | Yes | gen_random_uuid() | PK | Primary key |
| room_id | uuid | Yes | | FK rooms.id | The unit |
| tenant_profile_id | uuid | | | FK profiles.id | The tenant who reported it; empty for a repair the landlady logged for an empty unit |
| title | varchar(255) | Yes | | | Short title |
| description | text | Yes | | | What is wrong |
| category | varchar(100) | Yes | General Repair | | Kind of repair |
| priority | ticket_priority_type (Emergency, High, Medium, Low) | Yes | Medium | | Low, Medium, High or Urgent |
| status | ticket_status_type (Submitted, In Progress, Resolved, Closed) | Yes | Submitted | | Submitted, In Progress, Resolved or Closed |
| created_at | timestamptz | | now() | | When the row was created |
| resolved_at | timestamptz | | | | When marked done |
| closed_at | timestamptz | | | | When closed |
| closed_by | uuid | | | FK profiles.id | Who closed it (the landlady, or the tenant who cancelled) |
| assigned_technician | varchar(160) | | | | Who was sent |

### ticket_messages

Notes added to a repair request by the tenant or the landlady.

| Column | Type | Required | Default | Key | Description |
|---|---|---|---|---|---|
| id | uuid | Yes | gen_random_uuid() | PK | Primary key |
| ticket_id | uuid | Yes | | FK maintenance_tickets.id | The repair request |
| sender_id | uuid | Yes | | FK profiles.id | Who wrote it |
| message_body | text | Yes | | | The note |
| created_at | timestamptz | | now() | | When the row was created |

### ticket_attachments

Photographs attached to a repair request.

| Column | Type | Required | Default | Key | Description |
|---|---|---|---|---|---|
| id | uuid | Yes | gen_random_uuid() | PK | Primary key |
| ticket_id | uuid | Yes | | FK maintenance_tickets.id | The repair request |
| file_url | text | Yes | | | Where the photo is kept |
| file_type | varchar(50) | | image/jpeg | | JPEG or PNG only |
| uploaded_at | timestamptz | | now() | | When it was added |

### inquiries

Questions sent from the public site about a unit.

| Column | Type | Required | Default | Key | Description |
|---|---|---|---|---|---|
| id | uuid | Yes | gen_random_uuid() | PK | Primary key |
| room_id | uuid | Yes | | FK rooms.id | The unit asked about |
| prospect_name | varchar(255) | Yes | | | Visitor's name |
| prospect_email | varchar(255) | | | | Kept on inquiries from before 20 October 2026; no longer asked |
| prospect_phone | varchar(50) | Yes | | | Visitor's Philippine mobile number |
| message | text | Yes | | | The question |
| status | inquiry_status_type (Pending, Contacted, Converted, Closed) | Yes | Pending | | Pending, answered, closed or converted |
| converted_tenant_id | uuid | | | FK profiles.id | The tenant record it became, if the visitor moved in |
| created_at | timestamptz | | now() | | When the row was created |
| updated_at | timestamptz | | now() | | When it last changed |
| access_token_hash | text | | | | SHA-256 of the secret in the visitor's private link; the secret itself is never stored |
| reference_code | varchar(9) | | | | Short code that, with the visitor's phone number, opens the conversation |

### inquiry_messages

The conversation on an inquiry between the visitor and the landlady.

| Column | Type | Required | Default | Key | Description |
|---|---|---|---|---|---|
| id | uuid | Yes | gen_random_uuid() | PK | Primary key |
| inquiry_id | uuid | Yes | | FK inquiries.id | The inquiry |
| sender_id | uuid | | | FK profiles.id | The landlady, when she wrote it; empty for the visitor |
| sender_name | varchar(255) | Yes | | | Name shown on the message |
| message_body | text | Yes | | | The message |
| sent_at | timestamptz | | now() | | When it was sent |

## System

### notifications

In-app notices to one person, each linked to the record it is about.

| Column | Type | Required | Default | Key | Description |
|---|---|---|---|---|---|
| id | uuid | Yes | gen_random_uuid() | PK | Primary key |
| recipient_profile_id | uuid | Yes | | FK profiles.id | Who it is for |
| title | varchar(255) | Yes | | | Title |
| message | text | Yes | | | Text |
| type | varchar(100) | Yes | System | | Kind of notice |
| priority | ticket_priority_type (Emergency, High, Medium, Low) | Yes | Low | | How prominently it is shown |
| is_read | boolean | | false | | Whether it has been opened |
| related_entity_type | varchar(100) | | | | Kind of record it is about |
| related_entity_id | uuid | | | | The record it opens |
| created_at | timestamptz | | now() | | When the row was created |

**audit_logs**

Every administrator action and sign-in event, with the values before and after. The application can add rows and read them, but cannot change, delete or empty the table.

| Column | Type | Required | Default | Key | Description |
|---|---|---|---|---|---|
| id | uuid | Yes | gen_random_uuid() | PK | Primary key |
| actor_profile_id | uuid | | | FK profiles.id | Who acted |
| action | varchar(100) | Yes | | | What was done (for example PAYMENT_VERIFY) |
| entity_type | varchar(100) | Yes | | | Kind of record |
| entity_id | uuid | Yes | | | The record |
| previous_values | jsonb | | | | Values before |
| new_values | jsonb | | | | Values after |
| ip_address | varchar(50) | | | | Address the request came from |
| created_at | timestamptz | | now() | | When the row was created |

## Backup

### copied_period_backup_031

A copy of rent periods before the correction made by migration 031. No client role can read it.

| Column | Type | Required | Default | Key | Description |
|---|---|---|---|---|---|
| id | uuid | Yes | | PK | Primary key |
| invoice_number | text | | | | Copied from the corrected row |
| old_period_start | date | | | | The value before the correction (period start) |
| old_period_end | date | | | | The value before the correction (period end) |
| new_period_start | date | | | | The value after the correction (period start) |
| new_period_end | date | | | | The value after the correction (period end) |
| corrected_at | timestamptz | Yes | now() | | When the correction was made |

### date_paid_import_backup_032

A copy of imported payment dates before the correction made by migration 032. No client role can read it.

| Column | Type | Required | Default | Key | Description |
|---|---|---|---|---|---|
| id | uuid | Yes | | PK | Primary key |
| invoice_number | text | | | | Copied from the corrected row |
| old_date_paid | date | | | | The value before the correction (date paid) |
| new_date_paid | date | | | | The value after the correction (date paid) |
| how | text | | | | Copied from the corrected row |
| corrected_at | timestamptz | Yes | now() | | When the correction was made |

### rent_period_drift_backup_030

A copy of rent periods before the correction made by migration 030. No client role can read it.

| Column | Type | Required | Default | Key | Description |
|---|---|---|---|---|---|
| id | uuid | Yes | | PK | Primary key |
| room_id | uuid | | | | The unit |
| year | integer | | | | Copied from the corrected row |
| month | integer | | | | Copied from the corrected row |
| date_paid | date | | | | Copied from the corrected row |
| old_period_start | date | | | | The value before the correction (period start) |
| old_period_end | date | | | | The value before the correction (period end) |
| new_period_start | date | | | | The value after the correction (period start) |
| new_period_end | date | | | | The value after the correction (period end) |
| corrected_at | timestamptz | Yes | now() | | When the correction was made |

# APPENDIX O: USER ACCEPTANCE SIGN-OFF

**[TO DO: place here the scan of the form the landlady signed on 12 November 2026.]**

The user acceptance test was conducted at the Fe Galang Da Silva Boarding House on 30 September 2026 by the landlady, three tenants and one prospective tenant, on the live system with the property's own records (Chapter 4, Section 4.3.7). A second round followed on the redesigned system of Iteration 2, from 31 October to 10 November 2026: ten tenants and four prospective tenants did the tasks, and ten tenants, three prospective tenants and the landlady answered the questionnaire (Appendix M, Tables M-28 to M-30). The landlady and the tenants signed consent forms. The prospective tenants and the technical evaluators agreed through the notice at the start of the questionnaire. The sign-off of the landlady, as the client's representative, follows.

*[Scanned signed form, one page]*

# APPENDIX P: INFORMED CONSENT FORM

**[TO DO: the blank consent form, CONSENT_FORM.pdf]**

# APPENDIX Q: SOURCE CODE

The source code is kept in a Git repository, with its full history of changes, and is shown at the defense on request. Its layout is as follows.

| Folder | What it holds |
|---|---|
| `frontend/src/` | The Progressive Web Application in Vue 3 and TypeScript: 62 components, of which 19 are screens |
| `backend/src/` | The application programming interface in Express and TypeScript: 49 files, with the routes for the public site, tenants, the administrator and sign-in |
| `database/migrations/` | Every change made to the database, as 91 numbered migrations applied in order (10 November 2026) |
| `scripts/`, `backend/scripts/`, `frontend/scripts/` | The automated check suites of Section 4.3.1 and the tools that build the diagrams and reports |
| `docs/` | The requirements, business rules, design guideline and testing records |

Each change is recorded with its author, date and reason, which is the history Appendix C and Appendix M, Table M-19 summarize.

# APPENDIX R: LITERATURE MAP

Figure R-1 maps the literature reviewed in Chapter 2. The problem the study begins from is at the top. Under it are the five themes of Sections 2.1.1 to 2.1.5, each with what its studies found and the sources cited for it in that section. The themes meet in the synthesis (Section 2.1.6), which narrows to the gap (Section 2.2) and to this study.

*Figure R-1. Literature map of the study.*

[IMAGE FILE: images/Figure R-1 - Literature map of the study.png]

